

Código-fonte completo

Projeto **Diários Oficiais** — coleta, leitura e análise de diários oficiais municipais

Java puro (JDK 17+), sem biblioteca externa. Este PDF reúne o código **na íntegra**, arquivo por arquivo, com os **números de linha reais** (os mesmos que o IntelliJ mostra) — assim, quando o guia de explicação (PDF 2) diz “Limpeza.java:17”, você acha a linha 17 deste arquivo.

50 arquivos · 7234 linhas de código · organizados por pacote, na ordem em que vale a pena estudar (dados → extratores → etapas → análise → coleta → servidor → classes principais → testes), mais a documentação original (README e guias) em apêndice.

O código-fonte original **não tem comentários** (as razões das decisões estão no README e no PDF 2). Este PDF é **texto puro e pesquisável** (sem imagens, sem realce): cada linha do código é uma linha de texto, com o número da linha à esquerda, para que leitores e ferramentas de IA (como o NotebookLM) extraiam o código corretamente. Caracteres que a fonte do PDF não possui (um emoji no guia de estudo) aparecem como <U+XXXX>. Cada arquivo começa com a linha ARQUIVO: caminho/Arquivo.java.

Os favoritos (painel lateral do leitor de PDF) levam direto a cada arquivo; no índice da próxima página, clique no nome do arquivo.

Índice

Configuração

pom.xml	Configuração do Maven: Java 17, nenhuma dependência, classe principal = Servidor.	4
ABRIR_PAINEL.bat	Script do Windows que abre o painel com o Java portátil (_sistema/java) e o diarios.jar.	5
LEIA-ME.txt	Instruções curtas de uso para quem recebe o zip (abrir o painel, o que tem em cada pasta).	6

Pacote modelo

Fato.java	Um dado achado no texto (CNPJ, data, valor...) + a POSIÇÃO exata onde foi achado.	7
Ato.java	Unidade de trabalho: um decreto/portaria/extrato do diário, com seus fatos e a decisão do portão.	8
Diario.java	O arquivo de um dia inteiro: texto bruto e texto limpo lado a lado, mais os atos.	10
Contrato.java	Uma contratação (quem, quanto, processo) e o formatador de reais.	11
Relacao.java	Aresta do grafo: ato --(ALTERA REVOGA PRORROGA CITA)--> norma.	12
Achado.java	Uma frase da interpretação, com nível, título e evidência.	13

Pacote erros

DiariollegivelException.java	Exceção própria: diário sem texto aproveitável (carrega motivo e evidência).	14
ColetaException.java	Exceção própria: falha ao falar com a API (diz se vale tentar de novo).	15

Pacote extratores

Extrator.java	Classe ABSTRATA: contrato "extrair" + utilitários (casaMolde, semAcento...).	16
ExtratorCnpj.java	Filha: acha CNPJ pelo molde 99.999.999/9999-99.	17
ExtratorData.java	Filha: acha datas numéricas (dd/mm/aaaa) e por extenso ("24 de março de 2026").	18
ExtratorValor.java	Filha: acha valores em reais ("R\$ 1.234,56") e converte para número.	19
ExtratorProcesso.java	Filha: acha o número do processo depois da palavra "processo".	20
ExtratorReferencia.java	Filha: acha citações de normas (lei, decreto, portaria...) e as padroniza ("lei:11065").	21

Pacote etapas

Ingestao.java	Etapa 1: lista a pasta e cria um Diario por arquivo .txt.	23
Limpeza.java	Etapa 2: remove lixo de PDF e RECUSA o diário sem texto aproveitável.	24
Fragmentacao.java	Etapa 3: corta o diário em atos; só corta com evidência de corpo de texto.	25
Extracao.java	Etapa 4: roda todos os extratores em cada ato (polimorfismo).	27
Portao.java	Etapa 5: dá a nota a cada ato — ACEITO, DUVIDOSO ou REJEITADO — com motivo.	28
Privacidade.java	Mascara CPFs (***,***,***-20) sem mudar o tamanho do texto.	29
Relatorio.java	Grava arquivos de saída (BufferedWriter + try-with-resources).	30
Visualizador.java	Navegador de arquivos do menu de terminal (paginação, busca).	31
PaginaHtml.java	Monta as seções HTML do painel e o arquivo painel.html (concatenação de String).	34

Pacote analise

Assunto.java	Classifica o assunto de um ato (contratação, pessoal, norma...).	40
PainelContratos.java	Monta a lista de contratações, separa multas, faz o ranking por CNPJ.	41
GrafoNormas.java	Grafo de normas: decide ALTERA / REVOGA / PRORROGA / CITA; percorre vizinhança.	45
LeituraDoAto.java	Escreve, em português, o que cada ato faz.	47
Interpretacao.java	Gera as frases do "o que esses dados dizem" a partir de regras e limites.	49
Cobertura.java	Relatório do que foi lido, do que foi recusado e dos totais do portão.	54

Pacote coleta

Coletor.java	Fala com a API do Querido Diário (HttpClient), baixa os textos, tenta de novo.	55
Json.java	Leitor de JSON feito à mão (indexOf / substring).	58
Municipio.java	Código IBGE, nome e UF; gera o "apelido" usado nos nomes de arquivo.	60
Fonte.java	Catálogo das 27 capitais e do conjunto padrão de 5 fontes.	61
EstadoFonte.java	Estado de uma fonte: ligada?, status da última tentativa, período coberto.	62
Fontes.java	Lista de fontes + persistência em dados/estado/fontes.txt.	63
ResultadoColeta.java	Contadores e registro do que aconteceu com UMA fonte numa rodada.	65
Rodada.java	Uma coleta (ou descoberta) inteira; grava/relê uma linha em rodadas.txt.	66

Pacote servidor

Servidor.java	Sobe o HttpServer do JDK em localhost e liga cada endereço a um método.	68
Formulario.java	Lê parâmetros de formulário/URL (split, URLDecoder).	71
PainelWeb.java	Páginas "Painel" e "Coleta e fontes" (HTML montado em ArrayList<String>).	72
PaginaDiario.java	Páginas de um diário, de um ato (com prova de cada fato) e a busca.	77
Classes principais		
Aplicacao.java	O "cérebro": processarPasta(), coletar(), descobrir() e a coleta em segundo plano.	81
Principal.java	Menu de terminal (Scanner + switch); chama a Aplicacao.	85
Testes		
Testes.java	85 verificações com um mini-framework próprio (sem JUnit).	91
Documentação original do projeto (texto)		
README.md	README do projeto: como rodar, o pipeline e as 11 decisões de projeto (o porquê).	96
GUIA_DE_ESTUDO.md	Guia de estudo original: ordem de leitura do código e plano de estudo.	101
PREPARO_ARGUICAO.md	Preparo original para a arguição: perguntas por classe e exercícios.	105

ARQUIVO: pom.xml

raiz do projeto · 43 linhas · Configuração do Maven: Java 17, nenhuma dependência, classe principal = Servidor.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6
7     <groupId>br.edu.projeto</groupId>
8     <artifactId>diarios-oficiais</artifactId>
9     <version>1.0</version>
10    <packaging>jar</packaging>
11    <name>Diarios Oficiais</name>
12
13    <properties>
14        <maven.compiler.release>17</maven.compiler.release>
15        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
16        <classe.principal>diarios.servidor.Servidor</classe.principal>
17    </properties>
18
19    <build>
20        <plugins>
21            <plugin>
22                <groupId>org.apache.maven.plugins</groupId>
23                <artifactId>maven-jar-plugin</artifactId>
24                <version>3.4.2</version>
25                <configuration>
26                    <archive>
27                        <manifest>
28                            <mainClass>${classe.principal}</mainClass>
29                        </manifest>
30                    </archive>
31                </configuration>
32            </plugin>
33            <plugin>
34                <groupId>org.codehaus.mojo</groupId>
35                <artifactId>exec-maven-plugin</artifactId>
36                <version>3.5.0</version>
37                <configuration>
38                    <mainClass>${classe.principal}</mainClass>
39                </configuration>
40            </plugin>
41        </plugins>
42    </build>
43 </project>
```

ARQUIVO: ABRIR_PAINEL.bat

raiz do projeto · 10 linhas · Script do Windows que abre o painel com o Java portátil (_sistema/java) e o diarios.jar.

```
1 @echo off
2 chcp 65001 >nul
3 cd /d "%~dp0"
4 if not exist "_sistema\java\bin\java.exe" (
5     echo Extraia o zip INTEIRO antes de abrir.
6     pause
7     exit /b 1
8 )
9 "_sistema\java\bin\java.exe" -Dfile.encoding=UTF-8 -Dstdout.encoding=UTF-8 -jar "_sistema\diarios.jar" %*
10 pause
```

ARQUIVO: LEIA-ME.txt

raiz do projeto · 22 linhas · Instruções curtas de uso para quem recebe o zip (abrir o painel, o que tem em cada pasta).

```
1 PROJETO DIARIOS OFICIAIS
2 =====
3
4 PARA USAR
5   Dois cliques em  ABRIR_PAINEL.bat
6   O navegador abre em http://localhost:8080
7   Para parar, feche a janela preta.
8
9   (Extraia o zip INTEIRO antes. Nao precisa instalar nada.)
10
11 O QUE TEM AQUI
12 ABRIR_PAINEL.bat   abre o programa
13 codigo\           o codigo-fonte para ler, mais os guias de estudo (.md)
14 dados\            os diarios coletados
15 _sistema\         Java e programa compilado. NAO MEXER.
16                   (dentro: RODAR_TESTES.bat, para ver os 85 testes)
17
18 SE O WINDOWS AVISAR "protegeu o computador":
19   Mais informacoes -> Executar assim mesmo.
20
21 Nesta versao nao da para editar e recompilar (o Java portatil nao tem o
22 compilador). Para editar, use o IntelliJ abrindo codigo\pom.xml em outro PC.
```

ARQUIVO: diarios/modelo/Fato.java

pacote diarios.modelo · 55 linhas · Um dado achado no texto (CNPJ, data, valor...) + a POSIÇÃO exata onde foi achado.

```
1 package diarios.modelo;
2
3 public class Fato {
4
5     public static final String DATA = "DATA";
6     public static final String VALOR = "VALOR";
7     public static final String CNPJ = "CNPJ";
8     public static final String PROCESSO = "PROCESSO";
9     public static final String REFERENCIA = "REFERENCIA";
10
11     private final String tipo;
12     private final String valor;
13     private final int inicio;
14     private final int fim;
15
16     public Fato(String tipo, String valor, int inicio, int fim) {
17         this.tipo = tipo;
18         this.valor = valor;
19         this.inicio = inicio;
20         this.fim = fim;
21     }
22
23     public String getTipo() { return tipo; }
24     public String getValor() { return valor; }
25     public int getInicio() { return inicio; }
26     public int getFim() { return fim; }
27
28     public String provar(String textoOriginal, int margem) {
29         int de = Math.max(0, inicio - margem);
30         int ate = Math.min(textoOriginal.length(), fim + margem);
31         return textoOriginal.substring(de, ate).replace('\n', ' ').trim();
32     }
33
34     @Override
35     public boolean equals(Object outro) {
36         if (this == outro) {
37             return true;
38         }
39         if (!(outro instanceof Fato)) {
40             return false;
41         }
42         Fato f = (Fato) outro;
43         return tipo.equals(f.tipo) && valor.equals(f.valor);
44     }
45
46     @Override
47     public int hashCode() {
48         return (tipo + "|" + valor).hashCode();
49     }
50
51     @Override
52     public String toString() {
53         return String.format("%-10s %-30s [%d..%d]", tipo, valor, inicio, fim);
54     }
55 }
```

ARQUIVO: diarios/modelo/Ato.java

pacote diarios.modelo · 99 linhas · Unidade de trabalho: um decreto/portaria/extrato do diário, com seus fatos e a decisão do portão.

```
1 package diarios.modelo;
2
3 import java.util.ArrayList;
4
5 public class Ato {
6
7     public static final String ACEITO = "ACEITO";
8     public static final String DUVIDOSO = "DUVIDOSO";
9     public static final String REJEITADO = "REJEITADO";
10
11     private final String municipio;
12     private final String dataDiario;
13     private final String especie;
14     private final String numero;
15     private final String titulo;
16     private final String texto;
17     private final int posicaoNoDiario;
18
19     private final ArrayList<Fato> fatos = new ArrayList<>();
20
21     private String decisao = DUVIDOSO;
22     private String motivo = "ainda nao avaliado";
23     private String desambiguador = "";
24
25     public Ato(String municipio, String dataDiario, String especie, String numero,
26         String titulo, String texto, int posicaoNoDiario) {
27         this.municipio = municipio;
28         this.dataDiario = dataDiario;
29         this.especie = especie;
30         this.numero = numero;
31         this.titulo = titulo;
32         this.texto = texto;
33         this.posicaoNoDiario = posicaoNoDiario;
34     }
35
36     public String getIdentidade() {
37         String num = numero.isEmpty() ? "s-num@" + posicaoNoDiario : numero;
38         return municipio + ":" + especie.toLowerCase() + ":" + num + ":" + dataDiario
39             + desambiguador;
40     }
41
42     public void desambiguar(int ordem) {
43         this.desambiguador = "#" + ordem;
44     }
45
46     public void adicionarFato(Fato f) {
47         fatos.add(f);
48     }
49
50     public ArrayList<Fato> getFatos() {
51         return fatos;
52     }
53
54     public ArrayList<Fato> getFatosDoTipo(String tipo) {
55         ArrayList<Fato> achados = new ArrayList<>();
56         for (Fato f : fatos) {
57             if (f.getTipo().equals(tipo)) {
58                 achados.add(f);
59             }
60         }
61         return achados;
62     }
63
64     public void decidir(String decisao, String motivo) {
65         this.decisao = decisao;
66         this.motivo = motivo;
67     }
68
69     public String getMunicipio() { return municipio; }
70     public String getDataDiario() { return dataDiario; }
71     public String getEspecie() { return especie; }
72     public String getNumero() { return numero; }
73     public String getTitulo() { return titulo; }
74     public String getTexto() { return texto; }
75     public String getDecisao() { return decisao; }
76     public String getMotivo() { return motivo; }
77     public int getPosicao() { return posicaoNoDiario; }
78
79     @Override
80     public boolean equals(Object outro) {
81         if (this == outro) {
82             return true;
83         }
84         if (!(outro instanceof Ato)) {
85             return false;
86         }
87     }
88 }
```

```
87         return getIdentidade().equals(((Ato) outro).getIdentidade());
88     }
89
90     @Override
91     public int hashCode() {
92         return getIdentidade().hashCode();
93     }
94
95     @Override
96     public String toString() {
97         return String.format("%-9s %-46s %d fatos", decisao, getIdentidade(), fatos.size());
98     }
99 }
```

ARQUIVO: diarios/modelo/Diario.java

pacote diarios.modelo · 80 linhas · O arquivo de um dia inteiro: texto bruto e texto limpo lado a lado, mais os atos.

```
1 package diarios.modelo;
2
3 import java.util.ArrayList;
4
5 public class Diario {
6
7     private final String arquivo;
8     private final String municipio;
9     private final String data;
10    private final String textoBruto;
11    private final int linhasOriginais;
12
13    private String textoLimpo = "";
14    private int linhasRemovidas = 0;
15    private int titulosCandidatos = 0;
16    private int titulosDescartados = 0;
17
18    private final ArrayList<Ato> atos = new ArrayList<>();
19
20    private boolean utilizavel = true;
21    private String motivoRecusa = "";
22
23    public Diario(String arquivo, String municipio, String data, String textoBruto) {
24        this.arquivo = arquivo;
25        this.municipio = municipio;
26        this.data = data;
27        this.textoBruto = textoBruto;
28        this.linhasOriginais = textoBruto.split("\n").length;
29    }
30
31    public void registrarLimpeza(String textoLimpo, int linhasRemovidas) {
32        this.textoLimpo = textoLimpo;
33        this.linhasRemovidas = linhasRemovidas;
34    }
35
36    public void registrarFragmentacao(int candidatos, int descartados) {
37        this.titulosCandidatos = candidatos;
38        this.titulosDescartados = descartados;
39    }
40
41    public void recusar(String motivo) {
42        this.utilizavel = false;
43        this.motivoRecusa = motivo;
44    }
45
46    public void adicionarAto(Ato a) {
47        atos.add(a);
48    }
49
50    public int contar(String decisao) {
51        int n = 0;
52        for (Ato a : atos) {
53            if (a.getDecisao().equals(decisao)) {
54                n++;
55            }
56        }
57        return n;
58    }
59
60    public String getArquivo() { return arquivo; }
61    public String getMunicipio() { return municipio; }
62    public String getData() { return data; }
63    public String getTextoBruto() { return textoBruto; }
64    public String getTextoLimpo() { return textoLimpo; }
65    public int getLinhasRemovidas() { return linhasRemovidas; }
66    public int getLinhasOriginais() { return linhasOriginais; }
67    public int getTitulosCandidatos() { return titulosCandidatos; }
68    public int getTitulosDescartados() { return titulosDescartados; }
69    public ArrayList<Ato> getAtos() { return atos; }
70    public boolean isUtilizavel() { return utilizavel; }
71    public String getMotivoRecusa() { return motivoRecusa; }
72
73    @Override
74    public String toString() {
75        if (!utilizavel) {
76            return municipio + " " + data + " RECUSADO: " + motivoRecusa;
77        }
78        return municipio + " " + data + " " + atos.size() + " atos";
79    }
80 }
```

ARQUIVO: diarios.modelo/Contrato.java

pacote diarios.modelo · 59 linhas · Uma contratação (quem, quanto, processo) e o formatador de reais.

```
1 package diarios.modelo;
2
3 public class Contrato {
4
5     private final String municipio;
6     private final String data;
7     private final String cnpj;
8     private final String contratada;
9     private final String papel;
10    private final double valor;
11    private final String processo;
12    private final String identidadeAto;
13
14    public Contrato(String municipio, String data, String cnpj, String contratada,
15                    String papel, double valor, String processo, String identidadeAto) {
16        this.municipio = municipio;
17        this.data = data;
18        this.cnpj = cnpj;
19        this.contratada = contratada;
20        this.papel = papel;
21        this.valor = valor;
22        this.processo = processo;
23        this.identidadeAto = identidadeAto;
24    }
25
26    public String getPapel() { return papel; }
27
28    public String getMunicipio() { return municipio; }
29    public String getData() { return data; }
30    public String getCnpj() { return cnpj; }
31    public String getContratada() { return contratada; }
32    public double getValor() { return valor; }
33    public String getProcesso() { return processo; }
34    public String getIdentidadeAto() { return identidadeAto; }
35
36    public static String formatarReal(double v) {
37        String s = String.format(java.util.Locale.ROOT, "%.2f", v);
38        int ponto = s.indexOf('.');
39        String inteiro = s.substring(0, ponto);
40        String centavos = s.substring(ponto + 1);
41
42        StringBuilder comPonto = new StringBuilder();
43        int digitos = 0;
44        for (int i = inteiro.length() - 1; i >= 0; i--) {
45            comPonto.insert(0, inteiro.charAt(i));
46            digitos++;
47            if (digitos % 3 == 0 && i > 0) {
48                comPonto.insert(0, '.');
49            }
50        }
51        return "R$ " + comPonto + "," + centavos;
52    }
53
54    @Override
55    public String toString() {
56        return String.format("%-20s %-38s %18s %5s",
57                              cnpj, contratada, formatarReal(valor), processo);
58    }
59 }
```

ARQUIVO: diarios.modelo/Relacao.java

pacote diarios.modelo · 48 linhas · Aresta do grafo: ato --(ALTERA | REVOGA | PRORROGA | CITA)--> norma.

```
1 package diarios.modelo;
2
3 public class Relacao {
4
5     public static final String ALTERA = "ALTERA";
6     public static final String REVOGA = "REVOGA";
7     public static final String PRORROGA = "PRORROGA";
8     public static final String CITA = "CITA";
9
10    private final String origem;
11    private final String acao;
12    private final String alvo;
13    private final int posicao;
14
15    public Relacao(String origem, String acao, String alvo, int posicao) {
16        this.origem = origem;
17        this.acao = acao;
18        this.alvo = alvo;
19        this.posicao = posicao;
20    }
21
22    public String getOrigem() { return origem; }
23    public String getAcao() { return acao; }
24    public String getAlvo() { return alvo; }
25    public int getPosicao() { return posicao; }
26
27    @Override
28    public boolean equals(Object outro) {
29        if (this == outro) {
30            return true;
31        }
32        if (!(outro instanceof Relacao)) {
33            return false;
34        }
35        Relacao r = (Relacao) outro;
36        return origem.equals(r.origem) && acao.equals(r.acao) && alvo.equals(r.alvo);
37    }
38
39    @Override
40    public int hashCode() {
41        return (origem + acao + alvo).hashCode();
42    }
43
44    @Override
45    public String toString() {
46        return String.format("%-42s --%-8s--> %s", origem, acao, alvo);
47    }
48 }
```

ARQUIVO: diarios/modelo/Achado.java

pacote diarios.modelo · 31 linhas · Uma frase da interpretação, com nível, título e evidência.

```
1 package diarios.modelo;
2
3 public class Achado {
4
5     public static final String LEITURA = "LEITURA";
6     public static final String ATENCAO = "ATENCAO";
7     public static final String LIMITE = "LIMITE";
8
9     private final String nivel;
10    private final String titulo;
11    private final String frase;
12    private final String evidencia;
13
14    public Achado(String nivel, String titulo, String frase, String evidencia) {
15        this.nivel = nivel;
16        this.titulo = titulo;
17        this.frase = frase;
18        this.evidencia = evidencia;
19    }
20
21    public String getNivel()    { return nivel; }
22    public String getTitulo()   { return titulo; }
23    public String getFrase()    { return frase; }
24    public String getEvidencia() { return evidencia; }
25
26    @Override
27    public String toString() {
28        return "[" + nivel + "] " + titulo + "\n      " + frase
29            + "\n      evidencia: " + evidencia;
30    }
31 }
```

ARQUIVO: [diarios/erros/DiarioIlegivelException.java](#)

pacote diarios.erros · 16 linhas · Exceção própria: diário sem texto aproveitável (carrega motivo e evidência).

```
1 package diarios.erros;
2
3 public class DiarioIlegivelException extends Exception {
4
5     private final String arquivo;
6     private final String evidencia;
7
8     public DiarioIlegivelException(String arquivo, String motivo, String evidencia) {
9         super(motivo);
10        this.arquivo = arquivo;
11        this.evidencia = evidencia;
12    }
13
14    public String getArquivo() { return arquivo; }
15    public String getEvidencia() { return evidencia; }
16 }
```

ARQUIVO: diarios/erros/ColetaException.java

pacote diarios.erros · 23 linhas · Exceção própria: falha ao falar com a API (diz se vale tentar de novo).

```
1 package diarios.erros;
2
3 public class ColetaException extends Exception {
4
5     private final String url;
6     private final boolean valeTentarDeNovo;
7     private final long esperaSugeridaMs;
8
9     public ColetaException(String url, String motivo) {
10         this(url, motivo, false, 0);
11     }
12
13     public ColetaException(String url, String motivo, boolean valeTentarDeNovo, long esperaSugeridaMs) {
14         super(motivo);
15         this.url = url;
16         this.valeTentarDeNovo = valeTentarDeNovo;
17         this.esperaSugeridaMs = esperaSugeridaMs;
18     }
19
20     public String getUrl() { return url; }
21     public boolean podeTentarDeNovo() { return valeTentarDeNovo; }
22     public long getEsperaSugeridaMs() { return esperaSugeridaMs; }
23 }
```

ARQUIVO: diarios/extratores/Extrator.java

pacote diarios.extratores · 56 linhas · Classe ABSTRATA: contrato "extrair" + utilitários (casaMolde, semAcento...).

```
1 package diarios.extratores;
2
3 import diarios.modelo.Fato;
4 import java.util.ArrayList;
5
6 public abstract class Extrator {
7
8     private final String tipo;
9
10    protected Extrator(String tipo) {
11        this.tipo = tipo;
12    }
13
14    public String getTipo() {
15        return tipo;
16    }
17
18    public abstract ArrayList<Fato> extrair(String texto);
19
20    protected Fato criar(String valor, int inicio, int fim) {
21        return new Fato(tipo, valor, inicio, fim);
22    }
23
24    protected static boolean ehDigito(char c) {
25        return c >= '0' && c <= '9';
26    }
27
28    protected static int contarDigitos(String texto, int posicao) {
29        int n = 0;
30        while (posicao + n < texto.length() && ehDigito(texto.charAt(posicao + n))) {
31            n++;
32        }
33        return n;
34    }
35
36    protected static boolean casaMolde(String texto, int posicao, String molde) {
37        if (posicao + molde.length() > texto.length()) {
38            return false;
39        }
40        for (int i = 0; i < molde.length(); i++) {
41            char esperado = molde.charAt(i);
42            char achado = texto.charAt(posicao + i);
43            if (esperado == '9' ? !ehDigito(achado) : achado != esperado) {
44                return false;
45            }
46        }
47        return true;
48    }
49
50    public static String semAcento(String s) {
51        return s.replace('ç', 'c').replace('ã', 'a').replace('á', 'a')
52            .replace('â', 'a').replace('à', 'a').replace('é', 'e')
53            .replace('ê', 'e').replace('í', 'i').replace('ó', 'o')
54            .replace('ô', 'o').replace('õ', 'o').replace('ú', 'u');
55    }
56 }
```

ARQUIVO: [diarios/extratores/ExtratorCnpj.java](#)pacote `diarios.extratores` · 25 linhas · Filha: `acha CNPJ pelo molde 99.999.999/9999-99`.

```
1 package diarios.extratores;
2
3 import diarios.modelo.Fato;
4 import java.util.ArrayList;
5
6 public class ExtratorCnpj extends Extrator {
7
8     private static final String MOLDE = "99.999.999/9999-99";
9
10    public ExtratorCnpj() {
11        super(Fato.CNPJ);
12    }
13
14    @Override
15    public ArrayList<Fato> extrair(String texto) {
16        ArrayList<Fato> achados = new ArrayList<>();
17        for (int i = 0; i + MOLDE.length() <= texto.length(); i++) {
18            if (casaMolde(texto, i, MOLDE)) {
19                achados.add(criar(texto.substring(i, i + MOLDE.length()), i, i + MOLDE.length()));
20                i += MOLDE.length() - 1;
21            }
22        }
23        return achados;
24    }
25 }
```

ARQUIVO: diarios/extratores/ExtratorData.java

pacote diarios.extratores · 82 linhas · Filha: acha datas numéricas (dd/mm/aaaa) e por extenso ("24 de março de 2026").

```
1 package diarios.extratores;
2
3 import diarios.modelo.Fato;
4 import java.util.ArrayList;
5
6 public class ExtratorData extends Extrator {
7
8     private static final String[] MESES = {
9         "janeiro", "fevereiro", "marco", "abril", "maio", "junho",
10        "julho", "agosto", "setembro", "outubro", "novembro", "dezembro"
11    };
12
13    public ExtratorData() {
14        super(Fato.DATA);
15    }
16
17    @Override
18    public ArrayList<Fato> extrair(String texto) {
19        ArrayList<Fato> achados = new ArrayList<>();
20        acharNumericas(texto, achados);
21        acharPorExtenso(texto, achados);
22        return achados;
23    }
24
25    private void acharNumericas(String texto, ArrayList<Fato> achados) {
26        for (int i = 0; i < texto.length(); i++) {
27            if (casaMolde(texto, i, "99/99/9999")) {
28                achados.add(criar(texto.substring(i, i + 10), i, i + 10));
29                i += 9;
30            } else if (casaMolde(texto, i, "99/99/99")) {
31                achados.add(criar(texto.substring(i, i + 8), i, i + 8));
32                i += 7;
33            }
34        }
35    }
36
37    private void acharPorExtenso(String texto, ArrayList<Fato> achados) {
38        String base = semAcento(texto.toLowerCase());
39        for (String mes : MESES) {
40            int p = base.indexOf(mes);
41            while (p >= 0) {
42                int inicio = recuarAteDia(base, p);
43                int fim = avancarAteAno(base, p + mes.length());
44                if (inicio >= 0 && fim > 0) {
45                    achados.add(criar(texto.substring(inicio, fim), inicio, fim));
46                }
47                p = base.indexOf(mes, p + mes.length());
48            }
49        }
50    }
51
52    private int recuarAteDia(String texto, int posMes) {
53        int p = posMes - 1;
54        if (p < 4 || texto.charAt(p) != ' ') {
55            return -1;
56        }
57        if (!texto.startsWith("de", p - 2)) {
58            return -1;
59        }
60        p -= 3;
61        if (p < 0 || texto.charAt(p) != ' ') {
62            return -1;
63        }
64        int fimDia = p;
65        while (p > 0 && (ehDigito(texto.charAt(p - 1)) || texto.charAt(p - 1) == '0')) {
66            p--;
67        }
68        return p < fimDia ? p : -1;
69    }
70
71    private int avancarAteAno(String texto, int depoisDoMes) {
72        int p = depoisDoMes;
73        if (p + 4 > texto.length() || texto.charAt(p) != ' ') {
74            return -1;
75        }
76        if (!texto.startsWith("de ", p + 1)) {
77            return -1;
78        }
79        p += 4;
80        return contarDigitos(texto, p) == 4 ? p + 4 : -1;
81    }
82 }
```

ARQUIVO: diarios/extratores/ExtratorValor.java

pacote diarios.extratores · 64 linhas · Filha: acha valores em reais ("R\$ 1.234,56") e converte para número.

Nota: a(s) linha(s) 30 deste arquivo contém(êm) um **espaço não separável** (caractere U+00A0), que aparece aqui escrito como `\u00a0` para ficar visível; no arquivo original é um único caractere.

```
1 package diarios.extratores;
2
3 import diarios.modelo.Fato;
4 import java.util.ArrayList;
5
6 public class ExtratorValor extends Extrator {
7
8     public ExtratorValor() {
9         super(Fato.VALOR);
10    }
11
12    @Override
13    public ArrayList<Fato> extrair(String texto) {
14        ArrayList<Fato> achados = new ArrayList<>();
15        int i = texto.indexOf("R$");
16        while (i >= 0) {
17            int fim = medir(texto, i);
18            if (fim > 0) {
19                achados.add(criar(texto.substring(i, fim), i, fim));
20                i = texto.indexOf("R$", fim);
21            } else {
22                i = texto.indexOf("R$", i + 2);
23            }
24        }
25        return achados;
26    }
27
28    private int medir(String texto, int inicioRS) {
29        int p = inicioRS + 2;
30        while (p < texto.length() && (texto.charAt(p) == ' ' || texto.charAt(p) == '\u00a0')) {
31            p++;
32        }
33        if (contarDigitos(texto, p) == 0) {
34            return -1;
35        }
36        while (p < texto.length() && (ehDigito(texto.charAt(p)) || texto.charAt(p) == '.')) {
37            p++;
38        }
39        boolean temCentavos = p + 2 < texto.length() && texto.charAt(p) == ','
40            && ehDigito(texto.charAt(p + 1)) && ehDigito(texto.charAt(p + 2));
41        if (!temCentavos) {
42            return -1;
43        }
44        boolean digitoExtra = p + 3 < texto.length() && ehDigito(texto.charAt(p + 3));
45        return digitoExtra ? -1 : p + 3;
46    }
47
48    public static double paraNumero(String valorTexto) {
49        StringBuilder limpo = new StringBuilder();
50        for (int i = 0; i < valorTexto.length(); i++) {
51            char c = valorTexto.charAt(i);
52            if (c >= '0' && c <= '9') {
53                limpo.append(c);
54            } else if (c == ',') {
55                limpo.append('.');
56            }
57        }
58        try {
59            return Double.parseDouble(limpo.toString());
60        } catch (NumberFormatException e) {
61            return 0.0;
62        }
63    }
64 }
```

ARQUIVO: diarios/extratores/ExtratorProcesso.java

pacote diarios.extratores · 62 linhas · Filha: acha o número do processo depois da palavra "processo".

```
1 package diarios.extratores;
2
3 import diarios.modelo.Fato;
4 import java.util.ArrayList;
5
6 public class ExtratorProcesso extends Extrator {
7
8     private static final String MARCA = "rocesso";
9     private static final int TAMANHO_MINIMO = 6;
10
11     public ExtratorProcesso() {
12         super(Fato.PROCESSO);
13     }
14
15     @Override
16     public ArrayList<Fato> extrair(String texto) {
17         ArrayList<Fato> achados = new ArrayList<>();
18         int p = texto.indexOf(MARCA);
19         while (p >= 0) {
20             int inicio = pularAteNumero(texto, p + MARCA.length());
21             if (inicio > 0) {
22                 int fim = medirNumero(texto, inicio);
23                 if (fim - inicio >= TAMANHO_MINIMO) {
24                     achados.add(criar(texto.substring(inicio, fim), inicio, fim));
25                 }
26             }
27             p = texto.indexOf(MARCA, p + MARCA.length());
28         }
29         return achados;
30     }
31
32     private int pularAteNumero(String texto, int posicao) {
33         int limite = Math.min(texto.length(), posicao + 12);
34         for (int i = posicao; i < limite; i++) {
35             char c = texto.charAt(i);
36             if (ehDigito(c)) {
37                 return i;
38             }
39             boolean aceitavel = c == ':' || c == ' ' || c == 'n' || c == 'N'
40                 || c == 'o' || c == 'o' || c == '.';
41             if (!aceitavel) {
42                 return -1;
43             }
44         }
45         return -1;
46     }
47
48     private int medirNumero(String texto, int inicio) {
49         int p = inicio;
50         while (p < texto.length()) {
51             char c = texto.charAt(p);
52             if (!ehDigito(c) && c != '.' && c != '-' && c != '/') {
53                 break;
54             }
55             p++;
56         }
57         while (p > inicio && !ehDigito(texto.charAt(p - 1))) {
58             p--;
59         }
60         return p;
61     }
62 }
```

ARQUIVO: diarios/extratores/ExtratorReferencia.java

pacote diarios.extratores · 106 linhas · Filha: acha citações de normas (lei, decreto, portaria...) e as padroniza ("lei:11065").

```
1 package diarios.extratores;
2
3 import diarios.modelo.Fato;
4 import java.util.ArrayList;
5
6 public class ExtratorReferencia extends Extrator {
7
8     private static final String[] ESPECIES = {
9         "lei complementar", "lei", "decreto", "portaria", "resolucao", "instrucao normativa"
10    };
11
12    private static final String[] ESFERAS = {"municipal", "federal", "estadual", "conjunta"};
13
14    public ExtratorReferencia() {
15        super(Fato.REFERENCIA);
16    }
17
18    @Override
19    public ArrayList<Fato> extrair(String texto) {
20        ArrayList<Fato> achados = new ArrayList<>();
21        String base = semAcento(texto.toLowerCase());
22
23        for (String especie : ESPECIES) {
24            int p = base.indexOf(especie);
25            while (p >= 0) {
26                boolean ehPrefixoDeOutra = especie.equals("lei")
27                    && base.startsWith("lei complementar", p);
28                if (!ehPrefixoDeOutra) {
29                    int inicioNumero = pularAteNumero(base, texto, p + especie.length());
30                    if (inicioNumero > 0) {
31                        int fimNumero = medirNumero(base, inicioNumero);
32                        if (fimNumero > inicioNumero) {
33                            achados.add(criar(
34                                canonica(especie, texto.substring(inicioNumero, fimNumero)),
35                                p, fimNumero));
36                        }
37                    }
38                }
39                p = base.indexOf(especie, p + especie.length());
40            }
41        }
42        return achados;
43    }
44
45    public static String canonica(String especie, String numero) {
46        StringBuilder digitos = new StringBuilder();
47        for (int i = 0; i < numero.length(); i++) {
48            char c = numero.charAt(i);
49            if (c == '/') {
50                break;
51            }
52            if (c >= '0' && c <= '9') {
53                digitos.append(c);
54            }
55        }
56        return especie.replace(' ', '-') + ":" + digitos;
57    }
58
59    private int pularAteNumero(String base, String original, int posicao) {
60        int limite = Math.min(base.length(), posicao + 28);
61        for (int i = posicao; i < limite; i++) {
62            char c = base.charAt(i);
63            if (ehDigito(c)) {
64                return i;
65            }
66            if (" .:~/_°noum".indexOf(c) >= 0) {
67                continue;
68            }
69            int pulo = tamanhoDaEsfera(base, i);
70            if (pulo > 0) {
71                i += pulo - 1;
72                continue;
73            }
74            boolean siglaDeOrgao = Character.isLetter(c) && i < original.length()
75                && Character.isUpperCase(original.charAt(i));
76            if (!siglaDeOrgao) {
77                return -1;
78            }
79        }
80        return -1;
81    }
82
83    private int tamanhoDaEsfera(String base, int i) {
84        for (String esfera : ESFERAS) {
85            if (base.startsWith(esfera, i)) {
86                return esfera.length();
87            }
88        }
89        return -1;
90    }
91}
```

```
87         }
88     }
89     return 0;
90 }
91
92 private int medirNumero(String texto, int inicio) {
93     int p = inicio;
94     while (p < texto.length()) {
95         char c = texto.charAt(p);
96         if (!ehDigito(c) && c != '.' && c != '/') {
97             break;
98         }
99         p++;
100     }
101     while (p > inicio && !ehDigito(texto.charAt(p - 1))) {
102         p--;
103     }
104     return p;
105 }
106 }
```

ARQUIVO: diarios/etapas/Ingestao.java

pacote diarios.etapas · 60 linhas · Etapa 1: lista a pasta e cria um Diario por arquivo .txt.

```
1 package diarios.etapas;
2
3 import diarios.modelo.Diario;
4 import java.io.IOException;
5 import java.nio.charset.StandardCharsets;
6 import java.nio.file.DirectoryStream;
7 import java.nio.file.Files;
8 import java.nio.file.Path;
9 import java.nio.file.Paths;
10 import java.util.ArrayList;
11 import java.util.Collections;
12
13 public class Ingestao {
14
15     private final String pasta;
16
17     public Ingestao(String pasta) {
18         this.pasta = pasta;
19     }
20
21     public ArrayList<Diario> lerTodos(ArrayList<String> falhas) {
22         ArrayList<Diario> lidos = new ArrayList<>();
23         ArrayList<Path> arquivos = new ArrayList<>();
24
25         try (DirectoryStream<Path> fluxo = Files.newDirectoryStream(Paths.get(pasta), "*.txt")) {
26             for (Path p : fluxo) {
27                 arquivos.add(p);
28             }
29         } catch (IOException e) {
30             falhas.add("nao foi possivel abrir a pasta '" + pasta + "': " + e.getMessage());
31             return lidos;
32         }
33
34         Collections.sort(arquivos);
35
36         for (Path p : arquivos) {
37             String nome = p.getFileName().toString();
38             try {
39                 lidos.add(new Diario(nome, municipioDe(nome), dataDe(nome),
40                                     Files.readString(p, StandardCharsets.UTF_8)));
41             } catch (IOException e) {
42                 falhas.add(nome + ": falha de leitura - " + e.getMessage());
43             }
44         }
45         return lidos;
46     }
47
48     public static String municipioDe(String nomeArquivo) {
49         int sep = nomeArquivo.indexOf('_');
50         return sep > 0 ? nomeArquivo.substring(0, sep) : "desconhecido";
51     }
52
53     public static String dataDe(String nomeArquivo) {
54         int sep = nomeArquivo.indexOf('_');
55         if (sep < 0 || nomeArquivo.length() < sep + 11) {
56             return "0000-00-00";
57         }
58         return nomeArquivo.substring(sep + 1, sep + 11);
59     }
60 }
```

ARQUIVO: diarios/etapas/Limpeza.java

pacote diarios.etapas · 61 linhas · Etapa 2: remove lixo de PDF e RECUSA o diário sem texto aproveitável.

```
1 package diarios.etapas;
2
3 import diarios.erros.DiarioIlegivelException;
4 import diarios.modelo.Diario;
5
6 public class Limpeza {
7
8     private static final int MINIMO_DE_CARACTERES = 3;
9     private static final int MINIMO_DE_LINHAS_LONGAS = 10;
10    private static final int LINHA_LONGA = 80;
11
12    private static final String[] MOBILIA = {
13        "CHAVE DE INÍCIO DA MATÉRIA",
14        "CHAVE DE ACESSO DA MATÉRIA"
15    };
16
17    public void limpar(Diario diario) throws DiarioIlegivelException {
18        String[] linhas = diario.getTextoBruto().split("\n");
19        StringBuilder limpo = new StringBuilder();
20        int removidas = 0;
21        int longas = 0;
22
23        for (String linha : linhas) {
24            String s = linha.trim();
25            if (s.isEmpty()) {
26                continue;
27            }
28            if (s.length() <= MINIMO_DE_CARACTERES || ehMobiliaDePagina(s)) {
29                removidas++;
30                continue;
31            }
32            if (s.length() > LINHA_LONGA) {
33                longas++;
34            }
35            limpo.append(s).append('\n');
36        }
37
38        diario.registrarLimpeza(limpo.toString(), removidas);
39
40        if (longas < MINIMO_DE_LINHAS_LONGAS) {
41            String motivo = "PDF sem texto extraível (provavelmente escaneado)";
42            diario.recusar(motivo);
43            throw new DiarioIlegivelException(diario.getArquivo(), motivo,
44                longas + " linha(s) de corpo em " + linhas.length
45                + " linhas do arquivo; o mínimo esperado é " + MINIMO_DE_LINHAS_LONGAS);
46        }
47    }
48
49    private boolean ehMobiliaDePagina(String linha) {
50        String s = linha.toUpperCase();
51        for (String marca : MOBILIA) {
52            if (s.contains(marca)) {
53                return true;
54            }
55        }
56        if (s.contains("DIÁRIO OFICIAL") && s.contains("EDIÇÃO N")) {
57            return true;
58        }
59        return s.startsWith("HTTPS://") || s.startsWith("WWW.");
60    }
61 }
```

ARQUIVO: diarios/etapas/Fragmentacao.java

pacote diarios.etapas · 125 linhas · Etapa 3: corta o diário em atos; só corta com evidência de corpo de texto.

```
1 package diarios.etapas;
2
3 import diarios.extratores.Extrator;
4 import diarios.modelo.Ato;
5 import diarios.modelo.Diario;
6 import java.util.ArrayList;
7
8 public class Fragmentacao {
9
10     private static final String[] ESPECIES = {
11         "DECRETO", "LEI COMPLEMENTAR", "LEI", "PORTARIA", "RESOLUCAO", "EDITAL",
12         "INSTRUCAO NORMATIVA", "EXTRATO", "TERMO", "AVISO", "ATO", "ORDEM DE SERVICO",
13         "DELIBERACAO", "COMUNICADO", "RETIFICACAO"
14     };
15
16     private static final int CORPO_MINIMO = 200;
17     private static final int TITULO_MAXIMO = 120;
18     private static final int PARAGRAFO_MINIMO = 60;
19
20     private int candidatos = 0;
21     private int descartados = 0;
22
23     public void fragmentar(Diario diario) {
24         String texto = diario.getTextoLimpo();
25         ArrayList<Integer> inicios = new ArrayList<>();
26         ArrayList<String> titulos = new ArrayList<>();
27         ArrayList<String> especies = new ArrayList<>();
28
29         int posicao = 0;
30         for (String linha : texto.split("\n")) {
31             String s = linha.trim();
32             String especie = especieDaLinha(s);
33             if (especie != null && s.length() <= TITULO_MAXIMO) {
34                 inicios.add(posicao);
35                 titulos.add(s);
36                 especies.add(especie);
37             }
38             posicao += linha.length() + 1;
39         }
40
41         candidatos = inicios.size();
42         descartados = 0;
43
44         for (int i = 0; i < inicios.size(); i++) {
45             int comeco = inicios.get(i);
46             int fim = (i + 1 < inicios.size()) ? inicios.get(i + 1) : texto.length();
47             String bloco = texto.substring(comeco, fim);
48
49             if (!temCorpo(bloco, titulos.get(i))) {
50                 descartados++;
51                 continue;
52             }
53             Ato novo = new Ato(diario.getMunicipio(), diario.getData(),
54                             especies.get(i), numeroDoTitulo(titulos.get(i)),
55                             titulos.get(i), bloco, comeco);
56             int repeticoes = contarIdentidade(diario, novo.getIdentidade());
57             if (repeticoes > 0) {
58                 novo.desambiguar(repeticoes + 1);
59             }
60             diario.adicionarAto(novo);
61         }
62         diario.registrarFragmentacao(candidatos, descartados);
63     }
64
65     private int contarIdentidade(Diario diario, String identidade) {
66         int n = 0;
67         for (Ato a : diario.getAtos()) {
68             if (a.getIdentidade().startsWith(identidade)) {
69                 n++;
70             }
71         }
72         return n;
73     }
74
75     public int getCandidatos() { return candidatos; }
76     public int getDescartados() { return descartados; }
77
78     private boolean temCorpo(String bloco, String titulo) {
79         if (bloco.length() <= titulo.length()) {
80             return false;
81         }
82         String semTitulo = bloco.substring(titulo.length());
83         if (semTitulo.trim().length() < CORPO_MINIMO) {
84             return false;
85         }
86         for (String l : semTitulo.split("\n")) {
```

```
87         if (l.trim().length() > PARAGRAFO_MINIMO) {
88             return true;
89         }
90     }
91     return false;
92 }
93
94 private String especieDaLinha(String linha) {
95     String s = Extrator.semAcento(linha.toUpperCase());
96     for (String e : ESPECIES) {
97         if (!s.startsWith(e)) {
98             continue;
99         }
100         if (e.length() >= s.length()) {
101             return e;
102         }
103         char depois = s.charAt(e.length());
104         if (" Nª:-.".indexOf(depois) >= 0) {
105             return e;
106         }
107     }
108     return null;
109 }
110
111 private String numeroDoTitulo(String titulo) {
112     StringBuilder num = new StringBuilder();
113     boolean camecou = false;
114     for (int i = 0; i < titulo.length(); i++) {
115         char c = titulo.charAt(i);
116         if (c >= '0' && c <= '9') {
117             num.append(c);
118             camecou = true;
119         } else if (camecou && c != '.') {
120             break;
121         }
122     }
123     return num.toString();
124 }
125 }
```

ARQUIVO: diarios/etapas/Extracao.java

pacote diarios.etapas · 37 linhas · Etapa 4: roda todos os extratores em cada ato (polimorfismo).

```
1 package diarios.etapas;
2
3 import diarios.extratores.Extrator;
4 import diarios.extratores.ExtratorCnpj;
5 import diarios.extratores.ExtratorData;
6 import diarios.extratores.ExtratorProcesso;
7 import diarios.extratores.ExtratorReferencia;
8 import diarios.extratores.ExtratorValor;
9 import diarios.modelo.Ato;
10 import diarios.modelo.Diario;
11 import diarios.modelo.Fato;
12 import java.util.ArrayList;
13
14 public class Extracao {
15
16     private final ArrayList<Extrator> extratores = new ArrayList<>();
17
18     public Extracao() {
19         extratores.add(new ExtratorData());
20         extratores.add(new ExtratorValor());
21         extratores.add(new ExtratorCnpj());
22         extratores.add(new ExtratorProcesso());
23         extratores.add(new ExtratorReferencia());
24     }
25
26     public void extrair(Diario diario) {
27         for (Ato ato : diario.getAtoes()) {
28             for (Extrator e : extratores) {
29                 for (Fato f : e.extrair(ato.getTexto())) {
30                     ato.adicionarFato(new Fato(f.getTipo(), f.getValor(),
31                                             f.getInicio() + ato.getPosicao(),
32                                             f.getFim() + ato.getPosicao()));
33                 }
34             }
35         }
36     }
37 }
```

ARQUIVO: diarios/etapas/Portao.java

pacote diarios.etapas · 63 linhas · Etapa 5: dá a nota a cada ato — ACEITO, DUVIDOSO ou REJEITADO — com motivo.

```
1 package diarios.etapas;
2
3 import diarios.modelo.Ato;
4 import diarios.modelo.Diario;
5 import diarios.modelo.Fato;
6
7 public class Portao {
8
9     private static final int TEXTO_MINIMO = 200;
10    private static final int FATOS_MINIMOS = 2;
11
12    public void avaliar(Diario diario) {
13        for (Ato ato : diario.getAto()) {
14            avaliarAto(ato);
15        }
16    }
17
18    private void avaliarAto(Ato ato) {
19        int tamanho = ato.getTexto().trim().length();
20        if (tamanho < TEXTO_MINIMO) {
21            ato.decidir(Ato.REJEITADO, "texto curto demais (" + tamanho
22                + " caracteres, minimo " + TEXTO_MINIMO + ")");
23            return;
24        }
25        if (ato.getEspecie().isEmpty()) {
26            ato.decidir(Ato.REJEITADO, "sem especie identificada");
27            return;
28        }
29
30        int datas = ato.getFatosDoTipo(Fato.DATA).size();
31        int total = ato.getFatos().size();
32        boolean temNumero = !ato.getNumero().isEmpty();
33        boolean temDinheiro = !ato.getFatosDoTipo(Fato.VALOR).isEmpty()
34            || !ato.getFatosDoTipo(Fato.CNPJ).isEmpty();
35        boolean temNorma = !ato.getFatosDoTipo(Fato.REFERENCIA).isEmpty();
36
37        if (total < FATOS_MINIMOS) {
38            ato.decidir(Ato.DUVIDOSO, "poucos fatos extraidos (" + total + ")");
39            return;
40        }
41        if (!temNumero && !temDinheiro && !temNorma) {
42            ato.decidir(Ato.DUVIDOSO, "sem numero, sem valor/CNPJ e sem norma citada");
43            return;
44        }
45        if (datas == 0) {
46            ato.decidir(Ato.DUVIDOSO, "nenhuma data encontrada no ato");
47            return;
48        }
49
50        StringBuilder porque = new StringBuilder("tem ");
51        if (temNumero) {
52            porque.append("numero, ");
53        }
54        if (temDinheiro) {
55            porque.append("valor/CNPJ, ");
56        }
57        if (temNorma) {
58            porque.append("norma citada, ");
59        }
60        porque.append(datas).append(" data(s), ").append(total).append(" fatos");
61        ato.decidir(Ato.ACEITO, porque.toString());
62    }
63 }
```

ARQUIVO: diarios/etapas/Privacidade.java

pacote diarios.etapas · 49 linhas · Mascara CPFs (***.***.***-20) sem mudar o tamanho do texto.

```
1 package diarios.etapas;
2
3 public class Privacidade {
4
5     private static final String MOLDE_CPF = "999.999.999-99";
6
7     public static String mascararCpf(String texto) {
8         if (texto == null) {
9             return "";
10        }
11        StringBuilder saida = new StringBuilder(texto);
12        int i = 0;
13        while (i + MOLDE_CPF.length() <= saida.length()) {
14            if (casa(saida, i) && !digitoAntes(saida, i)
15                && !digitoDepois(saida, i + MOLDE_CPF.length())) {
16                for (int k = 0; k < 11; k++) {
17                    char c = saida.charAt(i + k);
18                    if (c >= '0' && c <= '9') {
19                        saida.setCharAt(i + k, '*');
20                    }
21                }
22                i += MOLDE_CPF.length();
23            } else {
24                i++;
25            }
26        }
27        return saida.toString();
28    }
29
30    private static boolean casa(StringBuilder s, int p) {
31        for (int k = 0; k < MOLDE_CPF.length(); k++) {
32            char esperado = MOLDE_CPF.charAt(k);
33            char achado = s.charAt(p + k);
34            boolean ok = esperado == '9' ? (achado >= '0' && achado <= '9') : achado == esperado;
35            if (!ok) {
36                return false;
37            }
38        }
39        return true;
40    }
41
42    private static boolean digitoAntes(StringBuilder s, int p) {
43        return p > 0 && Character.isDigit(s.charAt(p - 1));
44    }
45
46    private static boolean digitoDepois(StringBuilder s, int p) {
47        return p < s.length() && Character.isDigit(s.charAt(p));
48    }
49 }
```

ARQUIVO: diarios/etapas/Relatorio.java

pacote diarios.etapas · 33 linhas · Grava arquivos de saída (BufferedWriter + try-with-resources).

```
1 package diarios.etapas;
2
3 import java.io.BufferedWriter;
4 import java.io.IOException;
5 import java.nio.charset.StandardCharsets;
6 import java.nio.file.Files;
7 import java.nio.file.Path;
8 import java.nio.file.Paths;
9 import java.util.ArrayList;
10
11 public class Relatorio {
12     private final String pastaSaida;
13
14     public Relatorio(String pastaSaida) {
15         this.pastaSaida = pastaSaida;
16     }
17
18     public String gravar(String nomeArquivo, ArrayList<String> linhas) throws IOException {
19         Path pasta = Paths.get(pastaSaida);
20         if (!Files.exists(pasta)) {
21             Files.createDirectories(pasta);
22         }
23         Path destino = pasta.resolve(nomeArquivo);
24         try (BufferedWriter escritor = Files.newBufferedWriter(destino, StandardCharsets.UTF_8)) {
25             for (String l : linhas) {
26                 escritor.write(l);
27                 escritor.newLine();
28             }
29         }
30         return destino.toString();
31     }
32 }
33 }
```

ARQUIVO: diarios/etapas/Visualizador.java

pacote diarios.etapas · 236 linhas · Navegador de arquivos do menu de terminal (paginação, busca).

```
1 package diarios.etapas;
2
3 import diarios.extratores.Extrator;
4 import diarios.modelo.Ato;
5 import diarios.modelo.Diario;
6 import java.io.IOException;
7 import java.nio.charset.StandardCharsets;
8 import java.nio.file.DirectoryStream;
9 import java.nio.file.Files;
10 import java.nio.file.Path;
11 import java.nio.file.Paths;
12 import java.util.ArrayList;
13 import java.util.Collections;
14 import java.util.Scanner;
15
16 public class Visualizador {
17
18     private static final int LINHAS_POR_PAGINA = 30;
19
20     private final Scanner teclado;
21
22     public Visualizador(Scanner teclado) {
23         this.teclado = teclado;
24     }
25
26     public void abrir(String pasta, ArrayList<Diario> processados) {
27         while (true) {
28             ArrayList<Path> arquivos = listar(pasta);
29             if (arquivos.isEmpty()) {
30                 System.out.println(" pasta '' + pasta + '' vazia.");
31                 return;
32             }
33
34             System.out.println();
35             System.out.println(" ARQUIVOS EM " + pasta + "/");
36             for (int i = 0; i < arquivos.size(); i++) {
37                 System.out.printf(" %2d - %-42s %s%n", i,
38                     arquivos.get(i).getFileName(), tamanho(arquivos.get(i)));
39             }
40             System.out.print(" numero do arquivo (enter volta): ");
41
42             String entrada = ler();
43             if (entrada.isEmpty()) {
44                 return;
45             }
46             int indice = numero(entrada, arquivos.size());
47             if (indice < 0) {
48                 System.out.println(" fora da lista.");
49                 continue;
50             }
51             mostrarArquivo(arquivos.get(indice), processados);
52         }
53     }
54
55     private void mostrarArquivo(Path caminho, ArrayList<Diario> processados) {
56         String nome = caminho.getFileName().toString();
57         Diario processado = acharProcessado(nome, processados);
58
59         while (true) {
60             System.out.println();
61             System.out.println(" " + nome);
62             System.out.println(" 1 - Ver o texto como veio (bruto)");
63             if (processado != null && processado.isUtilizavel()) {
64                 System.out.println(" 2 - Ver o texto depois da limpeza");
65                 System.out.println(" 3 - Ver um ato separado, inteiro");
66                 System.out.println(" 4 - Procurar uma palavra");
67             } else if (processado != null) {
68                 System.out.println(" (recusado: " + processado.getMotivoRecusa() + ")");
69             }
70             System.out.print(" escolha (enter volta): ");
71
72             String escolha = ler();
73             if (escolha.isEmpty()) {
74                 return;
75             }
76             if (escolha.equals("1")) {
77                 paginar(lerLinhas(caminho), "texto bruto");
78             } else if (escolha.equals("2") && processado != null) {
79                 paginar(quebrar(processado.getTextoLimpo()), "texto limpo");
80             } else if (escolha.equals("3") && processado != null) {
81                 verAto(processado);
82             } else if (escolha.equals("4") && processado != null) {
83                 procurar(processado);
84             } else {
85                 System.out.println(" opcao invalida.");
86             }
87         }
88     }
89 }
```

```

87     }
88 }
89
90 private void verAto(Diario diario) {
91     ArrayList<Ato> atos = diario.getAtos();
92     if (atos.isEmpty()) {
93         System.out.println("  nenhum ato neste diario.");
94         return;
95     }
96     System.out.println();
97     int mostrar = Math.min(25, atos.size());
98     for (int i = 0; i < mostrar; i++) {
99         System.out.printf("    %2d - %-9s %-46s%n", i,
100             atos.get(i).getDecisao(), corta(atos.get(i).getTitulo(), 46));
101     }
102     if (atos.size() > mostrar) {
103         System.out.println("    ... e mais " + (atos.size() - mostrar));
104     }
105     System.out.print("  numero do ato: ");
106
107     int indice = numero(ler(), atos.size());
108     if (indice < 0) {
109         System.out.println("  fora da lista.");
110         return;
111     }
112     Ato ato = atos.get(indice);
113
114     System.out.println();
115     System.out.println("  identidade : " + ato.getIdentidade());
116     System.out.println("  decisao    : " + ato.getDecisao() + " - " + ato.getMotivo());
117     System.out.println("  fatos      : " + ato.getFatos().size());
118     System.out.println("  posicao     : caractere " + ato.getPosicao() + " do diario");
119     paginar(quebrar(ato.getTexto()), "texto do ato");
120 }
121
122 private void procurar(Diario diario) {
123     System.out.print("  palavra: ");
124     String palavra = ler();
125     if (palavra.isEmpty()) {
126         return;
127     }
128     String alvo = Extrator.semAcento(palavra.toLowerCase());
129     ArrayList<String> achados = new ArrayList<>();
130     int numeroDaLinha = 0;
131
132     for (String linha : quebrar(diario.getTextoLimpado())) {
133         numeroDaLinha++;
134         if (Extrator.semAcento(linha.toLowerCase()).contains(alvo)) {
135             achados.add(String.format("%5d| %s", numeroDaLinha, linha));
136         }
137     }
138     if (achados.isEmpty()) {
139         System.out.println("  nao encontrei '" + palavra + "'.");
140         return;
141     }
142     System.out.println("  " + achados.size() + " linha(s) com '" + palavra + "'");
143     paginar(achados, "ocorrencias");
144 }
145
146 private void paginar(ArrayList<String> linhas, String titulo) {
147     System.out.println();
148     System.out.println("  --- " + titulo + " (" + linhas.size() + " linhas) ---");
149
150     int posicao = 0;
151     while (posicao < linhas.size()) {
152         int fim = Math.min(posicao + LINHAS_POR_PAGINA, linhas.size());
153         for (int i = posicao; i < fim; i++) {
154             System.out.println("    " + linhas.get(i));
155         }
156         posicao = fim;
157         if (posicao >= linhas.size()) {
158             break;
159         }
160         System.out.print("  -- " + posicao + "/" + linhas.size()
161             + " -- enter continua, 's' sai: ");
162         if (ler().equalsIgnoreCase("s")) {
163             return;
164         }
165     }
166     System.out.println("  --- fim ---");
167 }
168
169 private ArrayList<Path> listar(String pasta) {
170     ArrayList<Path> arquivos = new ArrayList<>();
171     try (DirectoryStream<Path> fluxo = Files.newDirectoryStream(Paths.get(pasta))) {
172         for (Path p : fluxo) {
173             if (Files.isRegularFile(p)) {
174                 arquivos.add(p);
175             }
176         }
177     }

```

```
177     } catch (IOException e) {
178         System.out.println(" nao consegui abrir '" + pasta + "': " + e.getMessage());
179     }
180     Collections.sort(arquivos);
181     return arquivos;
182 }
183
184 private ArrayList<String> lerLinhas(Path caminho) {
185     ArrayList<String> linhas = new ArrayList<>();
186     try {
187         linhas.addAll(Files.readAllLines(caminho, StandardCharsets.UTF_8));
188     } catch (IOException e) {
189         linhas.add("nao consegui ler o arquivo: " + e.getMessage() + " ");
190     }
191     return linhas;
192 }
193
194 private ArrayList<String> quebrar(String texto) {
195     ArrayList<String> linhas = new ArrayList<>();
196     for (String l : texto.split("\n")) {
197         linhas.add(l);
198     }
199     return linhas;
200 }
201
202 private Diario acharProcessado(String nome, ArrayList<Diario> processados) {
203     for (Diario d : processados) {
204         if (d.getArquivo().equals(nome)) {
205             return d;
206         }
207     }
208     return null;
209 }
210
211 private String tamanho(Path caminho) {
212     try {
213         long bytes = Files.size(caminho);
214         return bytes < 1024 ? bytes + " B" : (bytes / 1024) + " KB";
215     } catch (IOException e) {
216         return "?";
217     }
218 }
219
220 private int numero(String entrada, int limite) {
221     try {
222         int n = Integer.parseInt(entrada.trim());
223         return (n >= 0 && n < limite) ? n : -1;
224     } catch (NumberFormatException e) {
225         return -1;
226     }
227 }
228
229 private String ler() {
230     return teclado.hasNextLine() ? teclado.nextLine().trim() : "";
231 }
232
233 private static String corta(String s, int max) {
234     return s.length() <= max ? s : s.substring(0, max - 1) + ".";
235 }
236 }
```

ARQUIVO: diarios/etapas/PaginaHtml.java

pacote diarios.etapas · 508 linhas · Monta as seções HTML do painel e o arquivo painel.html (concatenação de String).

```

1 package diarios.etapas;
2
3 import diarios.analise.Cobertura;
4 import diarios.analise.GrafoNormas;
5 import diarios.analise.Interpretacao;
6 import diarios.analise.PainelContratos;
7 import diarios.modelo.Achado;
8 import diarios.modelo.Ato;
9 import diarios.modelo.Contrato;
10 import diarios.modelo.Diario;
11 import diarios.modelo.Relacao;
12 import java.io.IOException;
13 import java.util.ArrayList;
14
15 public class PaginaHtml {
16
17     private final Relatorio relatorio;
18
19     public PaginaHtml() {
20         this.relatorio = null;
21     }
22
23     public PaginaHtml(Relatorio relatorio) {
24         this.relatorio = relatorio;
25     }
26
27     public String gerar(ArrayList<Diario> diarios, Cobertura cobertura,
28                         PaineContratos painel, GrafoNormas grafo,
29                         Interpretacao interpretacao) throws IOException {
30         ArrayList<String> h = new ArrayList<>();
31
32         h.add("<!DOCTYPE html>");
33         h.add("<html lang=\"pt-BR\">");
34         h.add("<head>");
35         h.add("<meta charset=\"UTF-8\">");
36         h.add("<meta name=\"viewport\" content=\"width=device-width, initial-scale=1\">");
37         h.add("<title>Diarios Oficiais - resultado</title>");
38         h.add("<style>" + estilo() + "</style>");
39         h.add("</head>");
40         h.add("<body>");
41         h.add("<div class=\"viz-root\">");
42
43         cabecalho(h, diarios, cobertura, painel, grafo);
44         secasInterpretacao(h, interpretacao);
45         secasCobertura(h, diarios, cobertura);
46         secasFunil(h, diarios, cobertura);
47         secasPortao(h, cobertura);
48         secasFornecedores(h, painel);
49         secasGrafo(h, grafo);
50         secasTabela(h, painel);
51         rodape(h);
52
53         h.add("</div>");
54         h.add("</body>");
55         h.add("</html>");
56
57         return relatorio.gravar("painel.html", h);
58     }
59
60     public void cabecalho(ArrayList<String> h, ArrayList<Diario> diarios, Cobertura c,
61                           PaineContratos painel, GrafoNormas grafo) {
62         h.add("<header>");
63         h.add("<h1>Diarios Oficiais municipais</h1>");
64         h.add("<p class=\"sub\">Leitura, separacao em atos e analise &mdash; "
65             + c.getLidos() + " diario(s) processado(s)</p>");
66         h.add("</header>");
67         h.add("<section class=\"tiles\">");
68         tile(h, String.valueOf(c.getTotalAtos()), "atos separados",
69             c.getUtilizaveis() + " diario(s) utilizavel(is)");
70         tile(h, String.valueOf(painel.getContratos().size()), "contratacoes",
71             Contrato.formatarReal(painel.total()) + " no total");
72         tile(h, String.valueOf(grafo.getArestas().size()), "relacoes entre normas",
73             grafo.doTipo(Relacao.ALTERA).size() + " alteracoes reais");
74         tile(h, String.valueOf(c.getRecusados()), "diario(s) recusado(s)",
75             "sem texto aproveitavel");
76         h.add("</section>");
77     }
78
79     private void tile(ArrayList<String> h, String numero, String rotulo, String apoio) {
80         h.add("<div class=\"tile\">");
81         h.add("<div class=\"tile-num\">" + escapar(numero) + "</div>");
82         h.add("<div class=\"tile-lab\">" + escapar(rotulo) + "</div>");
83         h.add("<div class=\"tile-sub\">" + escapar(apoio) + "</div>");
84         h.add("</div>");
85     }
86     public void secasInterpretacao(ArrayList<String> h, Interpretacao i) {

```

```

87         if (i.getAchados().isEmpty() && i.getResumo().isEmpty()) {
88             return;
89         }
90         h.add("<section class=\"leitura\">");
91         h.add("<h2>0 que esses dados dizem</h2>");
92         h.add("<p class=\"resumo\">" + escapar(i.getResumo()) + "</p>");
93
94         blocoDeAchados(h, "Leitura do periodo", i.doNivel(Achado.LEITURA), "ok");
95         blocoDeAchados(h, "Onde vale a pena olhar", i.doNivel(Achado.ATENCAO), "duv");
96         blocoDeAchados(h, "Limites desta leitura", i.doNivel(Achado.LIMITE), "rej");
97
98         h.add("<p class=\"nota\">Cada frase acima sai de uma regra no codigo, sobre "
99             + "numero medido no texto dos diarios. Nenhuma foi escrita a mao.</p>");
100        h.add("</section>");
101    }
102
103    private void blocoDeAchados(ArrayList<String> h, String titulo,
104        ArrayList<Achado> lista, String classe) {
105        if (lista.isEmpty()) {
106            return;
107        }
108        h.add("<h3>" + escapar(titulo) + "</h3>");
109        for (Achado a : lista) {
110            h.add("<div class=\"achado achado-\" + classe + \"\">");
111            h.add("<strong>" + escapar(a.getTitulo()) + "</strong>");
112            h.add("<p>" + escapar(a.getFrase()) + "</p>");
113            h.add("<p class=\"evidencia\">" + escapar(a.getEvidencia()) + "</p>");
114            h.add("</div>");
115        }
116    }
117
118    public void secaoCobertura(ArrayList<String> h, ArrayList<Diario> diarios, Cobertura c) {
119        h.add("<section>");
120        h.add("<h2>Cobertura</h2>");
121        h.add("<p class=\"nota\">0 que o programa conseguiu ler &mdash; e o que nao "
122            + "conseguiu, com o motivo. &quot;Zero atos&quot; e &quot;nao consegui "
123            + "ler&quot; sao coisas diferentes.</p>");
124
125        for (Diario d : diarios) {
126            boolean ok = d.isUtilizavel();
127            h.add("<div class=\"card \" + (ok ? \"card-ok\" : \"card-no\") + \"\">");
128            h.add("<div class=\"card-top\">");
129            h.add("<span class=\"sinal\">" + (ok ? "&#10003;" : "&#10007;") + "</span>");
130            h.add("<strong>" + escapar(d.getArquivo()) + "</strong>");
131            h.add("<span class=\"pill \" + (ok ? \"pill-ok\" : \"pill-no\") + \"\">"
132                + (ok ? "PROCESSADO" : "RECUSADO") + "</span>");
133            h.add("</div>");
134            if (ok) {
135                h.add("<div class=\"card-corpo\">" + d.getAtos().size() + " atos &middot; "
136                    + d.getLinhasRemovidas() + " linhas de lixo removidas de "
137                    + d.getLinhasOriginais() + "</div>");
138            } else {
139                h.add("<div class=\"card-corpo\">" + escapar(d.getMotivoRecusa()) + "</div>");
140            }
141            h.add("</div>");
142        }
143        h.add("</section>");
144    }
145
146    public void secaoFunil(ArrayList<String> h, ArrayList<Diario> diarios, Cobertura c) {
147        Diario d = null;
148        for (Diario cand : diarios) {
149            if (cand.isUtilizavel()) {
150                d = cand;
151                break;
152            }
153        }
154        if (d == null) {
155            return;
156        }
157
158        h.add("<section>");
159        h.add("<h2>0 funil, etapa a etapa</h2>");
160        h.add("<p class=\"nota\">" + escapar(d.getArquivo()) + "</p>");
161
162        int linhas = d.getLinhasOriginais();
163        int limpas = linhas - d.getLinhasRemovidas();
164        int atos = d.getAtos().size();
165
166        h.add("<div class=\"funil\">");
167        etapaFunil(h, "1. Limpeza", limpas, linhas,
168            d.getLinhasRemovidas() + " linhas descartadas (marca d'agua da conversao de PDF)");
169        etapaFunil(h, "2. Fragmentacao", atos, d.getTitulosCandidatos(),
170            d.getTitulosCandidatos() + " titulos candidatos, " + d.getTitulosDescartados()
171            + " descartados por nao terem corpo de texto");
172        etapaFunil(h, "3. Portao", d.contar(Ato.ACEITO), atos,
173            d.contar(Ato.ACEITO) + " aceitos de " + atos + " atos");
174        h.add("</div>");
175        h.add("</section>");
176    }

```

```

177 private void etapaFunil(ArrayList<String> h, String nome, int resta, int total, String nota) {
178     int pct = total > 0 ? (resta * 100 / total) : 0;
179     h.add("<div class=\"etapa\">");
180     h.add("<div class=\"etapa-cab\"><span>\" + escapar(nome) + \"</span>\"
181         + \"<span class=\"etapa-num\">\" + resta + \" / \" + total + \"</span></div>\"");
182     h.add("<div class=\"trilho\" title=\"\" + escapar(nota) + \">\"");
183     h.add("<div class=\"barra\" style=\"width:\" + pct + \"%\"></div>\"");
184     h.add("</div>\"");
185     h.add("<div class=\"etapa-nota\">\" + escapar(nota) + \"</div>\"");
186     h.add("</div>\"");
187 }
188 public void secarPortao(ArrayList<String> h, Cobertura c) {
189     int total = c.getTotalAtos();
190     if (total == 0) {
191         return;
192     }
193     h.add("<section>\"");
194     h.add("<h2>0 portao: tres saidas</h2>\"");
195     h.add("<p class=\"nota\">Com duas saidas, o caso duvidoso e empurrado para um \"
196         + \"dos lados e some. Com a terceira, ele fica contavel.</p>\"");
197     h.add("<div class=\"portao\">\"");
198     faixaPortao(h, "&#10003;\", \"ACEITO\", c.getAceitos(), total, \"ok\",
199         \"teve sinal positivo: numero, valor/CNPJ ou norma citada, mais data\");
200     faixaPortao(h, "&#9888;\", \"DUVIDOSO\", c.getDuvidosos(), total, \"duv\",
201         \"passou, mas faltou evidencia - e a lista do que o programa ainda nao trata\");
202     faixaPortao(h, "&#10007;\", \"REJEITADO\", c.getRejeitados(), total, \"rej\",
203         \"reprovou em checagem definitiva, com o motivo gravado\");
204     h.add("</div>\"");
205     h.add("</section>\"");
206 }
207
208 private void faixaPortao(ArrayList<String> h, String icone, String nome, int n,
209     int total, String classe, String explicacao) {
210     int pct = total > 0 ? (n * 100 / total) : 0;
211     h.add("<div class=\"faixa\">\"");
212     h.add("<div class=\"faixa-cab\">\"");
213     h.add("<span class=\"ic ic-\" + classe + \">\" + icone + \"</span>\"");
214     h.add("<strong>\" + nome + \"</strong>\"");
215     h.add("<span class=\"faixa-num\">\" + n + \" <span class=\"pct\">\" + pct + \"%\"</span></span>\"");
216     h.add("</div>\"");
217     h.add("<div class=\"trilho\"><div class=\"barra b-\" + classe
218         + \" style=\"width:\" + pct + \"%\"></div></div>\"");
219     h.add("<div class=\"etapa-nota\">\" + escapar(explicacao) + \"</div>\"");
220     h.add("</div>\"");
221 }
222 public void secarFornecedores(ArrayList<String> h, PainelContratos painel) {
223     ArrayList<String[]> ranking = painel.ranking();
224     if (ranking.isEmpty()) {
225         return;
226     }
227     h.add("<section>\"");
228     h.add("<h2>Maiores fornecedores</h2>\"");
229     h.add("<p class=\"nota\">Somado por CNPJ &mdash; o nome da empresa aparece \"
230         + \"escrito de varias formas no diario, o CNPJ nao.</p>\"");
231
232     double maior = valorDe(ranking.get(0)[2]);
233     int mostrar = Math.min(10, ranking.size());
234
235     h.add("<div class=\"grafico\">\"");
236     for (int i = 0; i < mostrar; i++) {
237         String[] r = ranking.get(i);
238         double v = valorDe(r[2]);
239         int pct = maior > 0 ? (int) Math.round(v * 100 / maior) : 0;
240         String dica = r[1] + \" - \" + r[2] + \" em \" + r[3] + \" contrato(s)\";
241
242         h.add("<div class=\"linha\">\"");
243         h.add("<div class=\"rotulo\" title=\"\" + escapar(r[0]) + \">\"
244             + escapar(cortar(r[1], 34)) + \"</div>\"");
245         h.add("<div class=\"trilho\" data-dica=\"\" + escapar(dica) + \">\"");
246         h.add("<div class=\"barra\" style=\"width:\" + Math.max(pct, 1) + \"%\"></div>\"");
247         h.add("</div>\"");
248         h.add("<div class=\"valor\">\" + escapar(r[2]) + \"</div>\"");
249         h.add("</div>\"");
250     }
251     h.add("</div>\"");
252
253     ArrayList<String[]> repetidos = painel.repeticoes(3);
254     if (!repetidos.isEmpty()) {
255         h.add("<div class=\"aviso\">\"");
256         h.add("<span class=\"ic ic-duv\">&#9888;</span>\"");
257         h.add("<div><strong>Aparecem 3 vezes ou mais no periodo</strong>\"");
258         h.add("<p class=\"nota\">E uma contagem, nao uma acusacao: repeticao tem \"
259             + \"explicacao legitima (ata de registro de precos, fornecedor unico). \"
260             + \"Serve para saber onde olhar.</p>\"");
261         h.add("<cul>\"");
262         for (String[] r : repetidos) {
263             h.add("<li><strong>\" + escapar(r[3]) + \"x</strong> \"
264                 + escapar(r[1]) + \" <span class=\"mono\">\" + escapar(r[0])
265                 + \"</span> &mdash; \" + escapar(r[2]) + \"</li>\"");
266         }
267     }

```

```

267         h.add("</ul></div></div>");
268     }
269     h.add("</section>");
270 }
271
272 public void secarGrafo(ArrayList<String> h, GrafoNormas grafo) {
273     if (grafo.getArestas().isEmpty()) {
274         return;
275     }
276     h.add("<section>");
277     h.add("<h2>Grafo de normas</h2>");
278     h.add("<p class='nota'>Nem toda mencao a uma norma e uma acao sobre ela. "
279         + "A maior parte das vezes em que aparece &quot;altera&quot; ou "
280         + "&quot;prorroga&quot; num diario municipal e clausula de contrato "
281         + "(&quot;alteracao dos precos&quot;, &quot;prorrogacao da vigencia da "
282         + "Ata&quot;), nao norma. Por isso a classificacao comeca eliminando: "
283         + "citacao de fundamento legal e forma nao-dispositiva viram apenas "
284         + "CITA.</p>");
285
286     h.add("<div class='contagem'>");
287     contar(h, "ALTERA", grafo.doTipo(Relacao.ALTERA).size());
288     contar(h, "REVOGA", grafo.doTipo(Relacao.REVOGA).size());
289     contar(h, "PRORROGA", grafo.doTipo(Relacao.PRORROGA).size());
290     contar(h, "CITA", grafo.doTipo(Relacao.CITA).size());
291     h.add("</div>");
292
293     ArrayList<String[]> mexidas = grafo.maisAlteradas();
294     if (!mexidas.isEmpty()) {
295         h.add("<h3>Normas mais mexidas</h3>");
296         h.add("<table><thead><tr><th>norma</th><th class='num'>vezes</th>"
297             + "</tr></thead><tbody>");
298         int mostrar = Math.min(8, mexidas.size());
299         for (int i = 0; i < mostrar; i++) {
300             h.add("<tr><td class='mono'>" + escapar(mexidas.get(i)[0]) + "</td>"
301                 + "<td class='num'>" + escapar(mexidas.get(i)[1]) + "</td></tr>");
302         }
303         h.add("</tbody></table>");
304     }
305
306     h.add("<h3>Relacoes que operam sobre a norma</h3>");
307     h.add("<table><thead><tr><th>ato</th><th>acao</th><th>norma</th>"
308         + "</tr></thead><tbody>");
309     int mostradas = 0;
310     for (Relacao r : grafo.getArestas()) {
311         if (r.getAcao().equals(Relacao.CITA) || mostradas >= 10) {
312             continue;
313         }
314         h.add("<tr><td class='mono'>" + escapar(r.getOrigem()) + "</td>"
315             + "<td><span class='tag'>" + escapar(r.getAcao()) + "</span></td>"
316             + "<td class='mono'>" + escapar(r.getAlvo()) + "</td></tr>");
317         mostradas++;
318     }
319     if (mostradas == 0) {
320         h.add("<tr><td colspan='3'>nenhuma relacao de acao neste periodo "
321             + "&mdash; so citacoes de fundamento legal</td></tr>");
322     }
323     h.add("</tbody></table>");
324     h.add("</section>");
325 }
326
327 private void contar(ArrayList<String> h, String nome, int n) {
328     h.add("<div class='cont'><div class='cont-num'>" + n + "</div>"
329         + "<div class='cont-lab'>" + nome + "</div></div>");
330 }
331
332 public void secarTabela(ArrayList<String> h, PaineisContratos painel) {
333     ArrayList<Contrato> contratos = painel.getContratos();
334     if (contratos.isEmpty()) {
335         return;
336     }
337     h.add("<section>");
338     h.add("<h2>Todas as contratacoes</h2>");
339     h.add("<p class='nota'>" + contratos.size() + " registros. Os mesmos dados "
340         + "das barras acima, em texto.</p>");
341     h.add("<table><thead><tr><th>CNPJ</th><th>contratada</th>"
342         + "<th class='num'>valor</th><th>processo</th></tr></thead><tbody>");
343     for (Contrato c : contratos) {
344         h.add("<tr><td class='mono'>" + escapar(c.getCnpj()) + "</td>"
345             + "<td>" + escapar(c.getContratada()) + "</td>"
346             + "<td class='num'>" + escapar(Contrato.formatarReal(c.getValor())) + "</td>"
347             + "<td class='mono'>" + escapar(c.getProcesso()) + "</td></tr>");
348     }
349     h.add("</tbody></table>");
350     h.add("</section>");
351 }
352
353 private void rodape(ArrayList<String> h) {
354     h.add("<footer>");
355     h.add("<p>Pagina gerada pelo proprio programa Java &mdash; a classe "
356         + "<span class='mono'>PaginaHtml</span> monta este arquivo com "
357         + "concatenacao de String e o grava com BufferedWriter. "

```

```

357         + "Quem desenha e o navegador.</p>");
358     h.add("<p>Dados: <span class=\"mono\">Querido Diario</span>, "
359         + "Open Knowledge Brasil.</p>");
360     h.add("</footer>");
361 }
362 private double valorDe(String formatado) {
363     StringBuilder n = new StringBuilder();
364     for (int i = 0; i < formatado.length(); i++) {
365         char c = formatado.charAt(i);
366         if (c >= '0' && c <= '9') {
367             n.append(c);
368         } else if (c == ',') {
369             n.append('.');
370         }
371     }
372     try {
373         return Double.parseDouble(n.toString());
374     } catch (NumberFormatException e) {
375         return 0.0;
376     }
377 }
378
379 public static String cortar(String s, int max) {
380     if (s == null) {
381         return "";
382     }
383     return s.length() <= max ? s : s.substring(0, max - 1) + "...";
384 }
385 public static String escapar(String s) {
386     if (s == null) {
387         return "";
388     }
389     return s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")
390         .replace("\"", "&quot;");
391 }
392 public String estilo() {
393     return String.join("\n",
394         ":root{",
395         "  --surface:#fcfcfb; --plano:#f9f9f7;",
396         "  --ink:#0b0b0b; --ink2:#52514e; --ink3:#898781;",
397         "  --grade:#e1e0d9; --eixo:#c3c2b7;",
398         "  --serie:#2a78d6;",
399         "  --ok:#0ca30c; --duv:#fab219; --rej:#d03b3b;",
400         "  color-scheme:light;",
401         "}",
402         "@media (prefers-color-scheme:dark) { :root:not([data-theme=\"light\"]){",
403         "  --surface:#1a1a19; --plano:#0d0d0d;",
404         "  --ink:#ffffff; --ink2:#c3c2b7; --ink3:#898781;",
405         "  --grade:#2c2c2a; --eixo:#383835;",
406         "  --serie:#3987e5;",
407         "  color-scheme:dark;",
408         "}}",
409         ":root[data-theme=\"dark\"]{",
410         "  --surface:#1a1a19; --plano:#0d0d0d;",
411         "  --ink:#ffffff; --ink2:#c3c2b7; --ink3:#898781;",
412         "  --grade:#2c2c2a; --eixo:#383835;",
413         "  --serie:#3987e5;",
414         "  color-scheme:dark;",
415         "}",
416         "/*{box-sizing:border-box}",
417         "body{margin:0;background:var(--plano);color:var(--ink);",
418         "  font-family:-apple-system,BlinkMacSystemFont,'Segoe UI',Roboto,sans-serif;",
419         "  font-size:15px;line-height:1.55}",
420         ".viz-root{max-width:940px;margin:0 auto;padding:32px 16px 64px}",
421         "header{padding:8px 0 24px;border-bottom:1px solid var(--grade);margin-bottom:28px}",
422         "h1{font-size:26px;margin:0 0 4px;letter-spacing:-.01em}",
423         "h2{font-size:19px;margin:0 0 6px;letter-spacing:-.01em}",
424         "h3{font-size:15px;margin:22px 0 8px;color:var(--ink2)}",
425         ".sub{margin:0;color:var(--ink2)}",
426         ".nota{margin:0 0 16px;color:var(--ink2);font-size:13.5px;max-width:70ch}",
427         "section{background:var(--surface);border:1px solid var(--grade);",
428         "  border-radius:10px;padding:22px;margin-bottom:18px}",
429         ".tiles{display:grid;grid-template-columns:repeat(auto-fit,minmax(170px,1fr))}",
430         "  gap:12px;background:none;border:none;padding:0}",
431         ".tile{background:var(--surface);border:1px solid var(--grade);",
432         "  border-radius:10px;padding:16px}",
433         ".tile-num{font-size:30px;font-weight:650;letter-spacing:-.02em;line-height:1.1}",
434         ".tile-lab{font-size:13.5px;color:var(--ink);margin-top:2px}",
435         ".tile-sub{font-size:12.5px;color:var(--ink3);margin-top:2px}",
436         ".card{border:1px solid var(--grade);border-radius:8px;padding:12px 14px;",
437         "  margin-bottom:10px;border-left-width:3px}",
438         ".card-ok{border-left-color:var(--ok)} .card-no{border-left-color:var(--rej)}",
439         ".card-top{display:flex;align-items:center;gap:8px;flex-wrap:wrap}",
440         ".card-corpo{color:var(--ink2);font-size:13.5px;margin-top:4px}",
441         ".sinal{font-weight:700}",
442         ".card-ok .sinal{color:var(--ok)} .card-no .sinal{color:var(--rej)}",
443         ".pill{font-size:11px;letter-spacing:.06em;padding:2px 8px;border-radius:99px;",
444         "  border:1px solid var(--eixo);color:var(--ink2)}",
445         ".trilho{position:relative;height:10px;background:var(--grade);",
446         "  border-radius:5px;overflow:hidden}",

```

```

447 ".barra{height:100%;background:var(--serie);border-radius:0 4px 4px 0}",
448 ".b-ok{background:var(--ok)} .b-duv{background:var(--duv)} .b-rej{background:var(--rej)}",
449 ".etapa{margin-bottom:16px}",
450 ".etapa-cab{display:flex;justify-content:space-between;font-size:13.5px;",
451 "  margin-bottom:5px}",
452 ".etapa-num{color:var(--ink2);font-variant-numeric:tabular-nums}",
453 ".etapa-nota{font-size:12.5px;color:var(--ink3);margin-top:5px}",
454 ".faixa{margin-bottom:16px}",
455 ".faixa-cab{display:flex;align-items:center;gap:8px;margin-bottom:5px;font-size:14px}",
456 ".faixa-num{margin-left:auto;font-variant-numeric:tabular-nums}",
457 ".pct{color:var(--ink3)}",
458 ".ic{font-weight:700}",
459 ".ic-ok{color:var(--ok)} .ic-duv{color:var(--duv)} .ic-rej{color:var(--rej)}",
460 ".grafico{display:flex;flex-direction:column;gap:9px}",
461 ".linha{display:grid;grid-template-columns:200px 1fr 120px;gap:12px;",
462 "  align-items:center}",
463 ".rotulo{font-size:13px;color:var(--ink2);overflow:hidden;",
464 "  text-overflow:ellipsis;white-space:nowrap}",
465 ".valor{font-size:13px;text-align:right;font-variant-numeric:tabular-nums}",
466 ".linha .trilha{height:14px;border-radius:4px}",
467 ".trilha[data-dica]:hover::after{content:attr(data-dica);position:absolute;",
468 "  left:0;top:-30px;background:var(--ink);color:var(--surface);",
469 "  padding:4px 8px;border-radius:5px;font-size:12px;white-space:nowrap;z-index:5}",
470 ".aviso{display:flex;gap:10px;margin-top:20px;padding:14px;",
471 "  border:1px solid var(--grade);border-left:3px solid var(--duv);",
472 "  border-radius:8px}",
473 ".aviso ul{margin:8px 0 0;padding-left:18px}",
474 ".aviso li{font-size:13.5px;margin-bottom:3px}",
475 ".contagem{display:grid;grid-template-columns:repeat(auto-fit,minmax(110px,1fr));",
476 "  gap:10px;margin-bottom:6px}",
477 ".cont{border:1px solid var(--grade);border-radius:8px;padding:12px;",
478 "  text-align:center}",
479 ".cont-num{font-size:22px;font-weight:650}",
480 ".cont-lab{font-size:11.5px;color:var(--ink3);letter-spacing:.06em}",
481 ".table{width:100%;border-collapse:collapse;font-size:13px;margin-top:6px}",
482 ".th{text-align:left;color:var(--ink3);font-weight:500;font-size:11.5px;",
483 "  letter-spacing:.06em;text-transform:uppercase;padding:6px 8px;",
484 "  border-bottom:1px solid var(--eixo)}",
485 ".td{padding:6px 8px;border-bottom:1px solid var(--grade)}",
486 ".th.num,td.num{text-align:right;font-variant-numeric:tabular-nums}",
487 ".mono{font-family:ui-monospace,SFMono-Regular,Menlo,monospace;font-size:12px;",
488 "  color:var(--ink2)}",
489 ".tag{font-size:11px;letter-spacing:.05em;padding:2px 7px;border-radius:4px;",
490 "  border:1px solid var(--eixo);color:var(--ink2)}",
491 ".resumo{font-size:16.5px;line-height:1.5;margin:0 0 18px;color:var(--ink)}",
492 ".achado{border:1px solid var(--grade);border-left-width:3px;border-radius:8px;",
493 "  padding:12px 14px;margin-bottom:10px}",
494 ".achado-ok{border-left-color:var(--ok)}",
495 ".achado-duv{border-left-color:var(--duv)}",
496 ".achado-rej{border-left-color:var(--rej)}",
497 ".achado p{margin:5px 0 0;font-size:13.5px;color:var(--ink2)}",
498 ".achado .evidencia{font-family:ui-monospace,SFMono-Regular,Menlo,monospace;",
499 "  font-size:12px;color:var(--ink3);margin-top:6px}",
500 ".footer{color:var(--ink3);font-size:12.5px;margin-top:24px}",
501 ".footer p{margin:4px 0}",
502 "@media (max-width:640px){",
503 "  .linha{grid-template-columns:1fr;gap:4px}",
504 "  .valor{text-align:left}",
505 "  .viz-root{padding:20px 16px 48px}",
506 "  }",
507 }
508 }
```

ARQUIVO: diarios/analise/Assunto.java

pacote diarios.analise · 71 linhas · Classifica o assunto de um ato (contratação, pessoal, norma...).

```
1 package diarios.analise;
2
3 import diarios.extratores.Extrator;
4 import diarios.modelo.Ato;
5 import diarios.modelo.Fato;
6
7 public class Assunto {
8
9     public static final String CONTRATACAO = "CONTRATACAO";
10    public static final String PESSOAL = "PESSOAL";
11    public static final String NORMA = "NORMA";
12    public static final String ORCAMENTO = "ORCAMENTO";
13    public static final String LICITACAO = "LICITACAO";
14    public static final String OUTRO = "OUTRO";
15
16    private static final String[] MARCAS_PESSOAL = {
17        "nomear", "nomeia", "exonerar", "exonera", "designar", "designa",
18        "conceder licenca", "aposentadoria", "cargo em comissao", "servidor"
19    };
20
21    private static final String[] MARCAS_ORCAMENTO = {
22        "credito adicional", "credito suplementar", "credito especial",
23        "abre credito", "dotacao orcamentaria", "suplementacao"
24    };
25
26    private static final String[] MARCAS_LICITACAO = {
27        "homologa", "adjudica", "pregao", "licitacao", "dispensa de licitacao",
28        "inexigibilidade", "concorrencia", "chamamento publico"
29    };
30
31    private static final String[] MARCAS_NORMA = {
32        "fica alterad", "ficam alterad", "passa a vigorar", "passam a vigorar",
33        "fica revogad", "ficam revogad", "nova redacao", "decreta:", "resolve:"
34    };
35
36    public static String de(Ato ato) {
37        String texto = Extrator.semAcento(ato.getTexto().toLowerCase());
38        String especie = ato.getEspecie();
39
40        boolean temDinheiro = !ato.getFatosDoTipo(Fato.VALOR).isEmpty();
41        boolean temEmpresa = !ato.getFatosDoTipo(Fato.CNPJ).isEmpty();
42
43        if (temEmpresa && temDinheiro) {
44            return CONTRATACAO;
45        }
46        if (contem(texto, MARCAS_ORCAMENTO)) {
47            return ORCAMENTO;
48        }
49        if (contem(texto, MARCAS_PESSOAL)) {
50            return PESSOAL;
51        }
52        if (contem(texto, MARCAS_LICITACAO) || especie.equals("EDITAL")) {
53            return LICITACAO;
54        }
55        if (contem(texto, MARCAS_NORMA)
56            && (especie.equals("DECRETO") || especie.equals("LEI")
57                || especie.equals("LEI COMPLEMENTAR") || especie.equals("PORTARIA"))) {
58            return NORMA;
59        }
60        return OUTRO;
61    }
62
63    private static boolean contem(String texto, String[] marcas) {
64        for (String m : marcas) {
65            if (texto.contains(m)) {
66                return true;
67            }
68        }
69        return false;
70    }
71 }
```

ARQUIVO: diarios/analise/PainelContratos.java

pacote diarios.analise · 328 linhas · Monta a lista de contratações, separa multas, faz o ranking por CNPJ.

```
1 package diarios.analise;
2
3 import diarios.extratores.ExtratorValor;
4 import diarios.modelo.Ato;
5 import diarios.modelo.Contrato;
6 import diarios.modelo.Diario;
7 import diarios.modelo.Fato;
8 import java.util.ArrayList;
9
10 public class PainelContratos {
11
12     private static final String[] SUFIXOS = {
13         " LTDA", " S/A", " S.A", " EIRELI", " ME", " EPP", " COOPERATIVA"
14     };
15
16     private static final String[] RUIDOS = {
17         "denominada ", "denominado ", "empresa ", "contratada ", "contratado ",
18         "favor de ", "a ", "o "
19     };
20
21     private static final int JANELA_NOME = 220;
22     private static final int NOME_MINIMO = 4;
23     private static final int NOME_MAXIMO = 70;
24     private static final int MAIUSCULAS_MINIMAS = 70;
25
26     public static final String LADO_MUNICIPIO = "LADO_MUNICIPIO";
27     public static final String NOTIFICADO = "NOTIFICADO";
28     public static final String FORNECEDOR = "FORNECEDOR";
29     public static final String INDEFINIDO = "INDEFINIDO";
30
31     private static final String[][] PAPEIS = {
32         {"NOTIFICANTE", LADO_MUNICIPIO},
33         {"CONTRATANTE", LADO_MUNICIPIO},
34         {"NOTIFICADO", NOTIFICADO},
35         {"AUTUADO", NOTIFICADO},
36         {"CONTRATADA", FORNECEDOR},
37         {"CONTRATADO", FORNECEDOR},
38         {"DETENTORA", FORNECEDOR},
39         {"FAVORECIDO", FORNECEDOR},
40         {"VENCEDORA", FORNECEDOR},
41         {"ADJUDICAT", FORNECEDOR},
42         {"FORNECEDOR", FORNECEDOR},
43         {"EMPRESA", FORNECEDOR}
44     };
45
46     private static final int JANELA_PAPEL = 320;
47
48     private final ArrayList<Contrato> contratos = new ArrayList<>();
49     private final ArrayList<Contrato> descartados = new ArrayList<>();
50
51     public void limpar() {
52         contratos.clear();
53         descartados.clear();
54     }
55
56     public void processar(ArrayList<Diario> diarios) {
57         for (Diario d : diarios) {
58             if (!d.isUtilizavel()) {
59                 continue;
60             }
61             for (Ato ato : d.getAto()) {
62                 ArrayList<Fato> cnpjjs = ato.getFatosDoTipo(Fato.CNPJ);
63                 ArrayList<Fato> valores = ato.getFatosDoTipo(Fato.VALOR);
64                 if (cnpjjs.isEmpty() || valores.isEmpty()) {
65                     continue;
66                 }
67                 ArrayList<Fato> processos = ato.getFatosDoTipo(Fato.PROCESSO);
68                 String processo = processos.isEmpty() ? "-" : processos.get(0).getValor();
69
70                 for (Fato cnpj : cnpjjs) {
71                     Fato valor = maisProximo(valores, cnpj.getInicio());
72                     String[] papelENome = papelENomeAntesDe(d.getTextoLimpo(), cnpj.getInicio());
73
74                     Contrato c = new Contrato(d.getMunicipio(), d.getData(),
75                         cnpj.getValor(), papelENome[1], papelENome[0],
76                         ExtratorValor.paraNumero(valor.getValor()),
77                         processo, ato.getIdentidade());
78
79                     if (papelENome[0].equals(FORNECEDOR) || papelENome[0].equals(INDEFINIDO)) {
80                         contratos.add(c);
81                     } else {
82                         descartados.add(c);
83                     }
84                 }
85             }
86         }
87     }
```

```

87     }
88
89     private Fato maisProximo(ArrayList<Fato> valores, int posicao) {
90         Fato melhor = valores.get(0);
91         int menorDistancia = Integer.MAX_VALUE;
92         for (Fato v : valores) {
93             int d = Math.abs(v.getInicio() - posicao);
94             if (d < menorDistancia) {
95                 menorDistancia = d;
96                 melhor = v;
97             }
98         }
99         return melhor;
100     }
101
102     private String[] papelENomeAntesDe(String texto, int posicaoCnpj) {
103         int de = Math.max(0, posicaoCnpj - JANELA_PAPEL);
104         String janela = texto.substring(de, posicaoCnpj).replace('\n', ' ');
105         String maiusculo = janela.toUpperCase();
106
107         int ultimaMarca = -1;
108         String papel = INDEFINIDO;
109         for (String[] par : PAPEIS) {
110             int p = maiusculo.lastIndexOf(par[0]);
111             if (p > ultimaMarca) {
112                 ultimaMarca = p;
113                 papel = par[1];
114             }
115         }
116         if (ultimaMarca < 0) {
117             return new String[] {INDEFINIDO, nomeAntesDe(texto, posicaoCnpj)};
118         }
119         return new String[] {papel, nomeDepoisDaMarca(janela, ultimaMarca)};
120     }
121
122     private String nomeDepoisDaMarca(String janela, int marca) {
123         int i = marca;
124         while (i < janela.length() && janela.charAt(i) != ':' && janela.charAt(i) != ' ') {
125             i++;
126         }
127         while (i < janela.length() && (janela.charAt(i) == ':' || janela.charAt(i) == ' ')) {
128             i++;
129         }
130         String resto = janela.substring(Math.min(i, janela.length()));
131
132         int corte = resto.length();
133         String[] paradas = {"", ";", ":", " sito", " SITO", " inscrit", " INSCRIT",
134             "CNPJ", "cnpj", " - ", " - "};
135         for (String parada : paradas) {
136             int p = resto.indexOf(parada);
137             if (p > 0 && p < corte) {
138                 corte = p;
139             }
140         }
141         String nome = limparBordas(resto.substring(0, corte));
142         return nome.length() >= NOME_MINIMO && nome.length() <= NOME_MAXIMO
143             ? nome : "(nao identificada)";
144     }
145
146     private String limparBordas(String s) {
147         String nome = s.trim();
148         while (!nome.isEmpty() && !Character.isLetterOrDigit(nome.charAt(0))) {
149             nome = nome.substring(1).trim();
150         }
151         while (!nome.isEmpty() && ".,;:--".indexOf(nome.charAt(nome.length() - 1)) >= 0) {
152             nome = nome.substring(0, nome.length() - 1).trim();
153         }
154         return tirarRestoDePalavra(nome);
155     }
156
157     private String tirarRestoDePalavra(String nome) {
158         int espaco = nome.indexOf(' ');
159         if (espaco < 0 || espaco > 3) {
160             return nome;
161         }
162         String primeira = nome.substring(0, espaco);
163         boolean itemDeLista = primeira.endsWith("(") || primeira.endsWith(":")
164             || primeira.endsWith(".");
165         boolean pedacoDePalavra = primeira.length() <= 2
166             && primeira.equals(primeira.toLowerCase());
167         return (itemDeLista || pedacoDePalavra) ? nome.substring(espaco + 1).trim() : nome;
168     }
169
170     private String nomeAntesDe(String texto, int posicaoCnpj) {
171         int de = Math.max(0, posicaoCnpj - JANELA_NOME);
172         String antes = texto.substring(de, posicaoCnpj).replace('\n', ' ');
173         String maiusculo = antes.toUpperCase();
174
175         int fimDoNome = -1;
176         for (String sufixo : SUFIXOS) {

```

```

177         int p = maiusculo.lastIndexOf(sufixo);
178         int fim = p + sufixo.length();
179         if (p >= 0 && fim > fimDoNome && terminaAqui(maiusculo, fim)) {
180             fimDoNome = fim;
181         }
182     }
183     if (fimDoNome > 0) {
184         String nome = limparBordas(recuarNome(antes, fimDoNome));
185         if (nome.length() >= NOME_MINIMO) {
186             return nome;
187         }
188     }
189     return nomePorSeparador(antes, maiusculo);
190 }
191
192 private String nomePorSeparador(String antes, String maiusculo) {
193     int marcaCnpj = maiusculo.lastIndexOf("CNPJ");
194     String ateMarca = marcaCnpj > 0 ? antes.substring(0, marcaCnpj) : antes;
195
196     int corte = -1;
197     for (char sep : new char[] {';', ':', '.'}) {
198         corte = Math.max(corte, ateMarca.lastIndexOf(sep));
199     }
200     String nome = (corte >= 0 ? ateMarca.substring(corte + 1) : ateMarca).trim();
201     while (!nome.isEmpty() && "-".indexOf(nome.charAt(nome.length() - 1)) >= 0) {
202         nome = nome.substring(0, nome.length() - 1).trim();
203     }
204     nome = limparBordas(nome);
205     boolean aceitavel = nome.length() >= NOME_MINIMO && nome.length() <= NOME_MAXIMO
206         && pareceRazaoSocial(nome);
207     return aceitavel ? nome : "(nao identificada)";
208 }
209
210 private boolean terminaAqui(String texto, int posicao) {
211     return posicao >= texto.length() || !Character.isLetterOrDigit(texto.charAt(posicao));
212 }
213
214 private String recuarNome(String texto, int fimDoNome) {
215     int p = fimDoNome;
216     int limite = Math.max(0, fimDoNome - NOME_MAXIMO);
217     int inicio = fimDoNome;
218     while (p > limite) {
219         char c = texto.charAt(p - 1);
220         boolean parteDoNome = Character.isLetterOrDigit(c) || "-./".indexOf(c) >= 0;
221         if (!parteDoNome) {
222             break;
223         }
224         p--;
225         if (c != ' ') {
226             inicio = p;
227         }
228     }
229     String nome = texto.substring(inicio, fimDoNome).trim();
230     for (String ruido : RUIDOS) {
231         if (nome.toLowerCase().startsWith(ruido)) {
232             nome = nome.substring(ruido.length()).trim();
233         }
234     }
235     return nome;
236 }
237
238 private boolean pareceRazaoSocial(String s) {
239     int maiusculas = 0;
240     int letras = 0;
241     for (int i = 0; i < s.length(); i++) {
242         char c = s.charAt(i);
243         if (Character.isLetter(c)) {
244             letras++;
245             if (Character.isUpperCase(c)) {
246                 maiusculas++;
247             }
248         }
249     }
250     return letras > 0 && (maiusculas * 100 / letras) >= MAIUSCULAS_MINIMAS;
251 }
252
253 public ArrayList<Contrato> getContratos() {
254     return contratos;
255 }
256
257 public ArrayList<Contrato> getDescartados() {
258     return descartados;
259 }
260
261 public int contarDescartadosDoPapel(String papel) {
262     int n = 0;
263     for (Contrato c : descartados) {
264         if (c.getPapel().equals(papel)) {
265             n++;
266         }
267     }

```

```
267     }
268     return n;
269 }
270
271 public double total() {
272     double soma = 0;
273     for (Contrato c : contratos) {
274         soma += c.getValor();
275     }
276     return soma;
277 }
278
279 public ArrayList<String[]> ranking() {
280     ArrayList<String> cnpjjs = new ArrayList<>();
281     ArrayList<String> nomes = new ArrayList<>();
282     ArrayList<Double> somas = new ArrayList<>();
283     ArrayList<Integer> vezes = new ArrayList<>();
284
285     for (Contrato c : contratos) {
286         int i = cnpjjs.indexOf(c.getCnpj());
287         if (i < 0) {
288             cnpjjs.add(c.getCnpj());
289             nomes.add(c.getContratada());
290             somas.add(c.getValor());
291             vezes.add(1);
292         } else {
293             somas.set(i, somas.get(i) + c.getValor());
294             vezes.set(i, vezes.get(i) + 1);
295             if (nomes.get(i).equals("(nao identificada)")) {
296                 nomes.set(i, c.getContratada());
297             }
298         }
299     }
300
301     ArrayList<String[]> linhas = new ArrayList<>();
302     boolean[] usado = new boolean[cnpjjs.size()];
303     for (int n = 0; n < cnpjjs.size(); n++) {
304         int melhor = -1;
305         for (int i = 0; i < cnpjjs.size(); i++) {
306             if (!usado[i] && (melhor < 0 || somas.get(i) > somas.get(melhor))) {
307                 melhor = i;
308             }
309         }
310         usado[melhor] = true;
311         linhas.add(new String[] {
312             cnpjjs.get(melhor), nomes.get(melhor),
313             Contrato.formatarReal(somas.get(melhor)), String.valueOf(vezes.get(melhor))
314         });
315     }
316     return linhas;
317 }
318
319 public ArrayList<String[]> repeticoes(int minimo) {
320     ArrayList<String[]> alertas = new ArrayList<>();
321     for (String[] linha : ranking()) {
322         if (Integer.parseInt(linha[3]) >= minimo) {
323             alertas.add(linha);
324         }
325     }
326     return alertas;
327 }
328 }
```

ARQUIVO: diarios/analise/GrafoNormas.java

pacote diarios.analise · 176 linhas · Grafo de normas: decide ALTERA / REVOGA / PRORROGA / CITA; percorre vizinhança.

```
1 package diarios.analise;
2
3 import diarios.extratores.Extrator;
4 import diarios.modelo.Ato;
5 import diarios.modelo.Diario;
6 import diarios.modelo.Fato;
7 import diarios.modelo.Relacao;
8 import java.util.ArrayList;
9
10 public class GrafoNormas {
11
12     private static final int JANELA_ANTES = 160;
13     private static final int JANELA_DEPOIS = 110;
14     private static final int TRECHO_FUNDAMENTO = 45;
15     private static final int TRECHO_VERBO = 70;
16     private static final int TRECHO_DEPOIS = 90;
17
18     private static final String[] FUNDAMENTO = {
19         "nos termos d", "conforme", "com base n", "previsto n", "prevista n",
20         "de acordo com", "na forma d", "a que se refere", "disposto n", "art."
21     };
22
23     private final ArrayList<Relacao> arestas = new ArrayList<>();
24
25     public void limpar() {
26         arestas.clear();
27     }
28
29     public void processar(ArrayList<Diario> diarios) {
30         for (Diario d : diarios) {
31             if (!d.isUtilizavel()) {
32                 continue;
33             }
34             for (Ato ato : d.getAto()) {
35                 for (Fato ref : ato.getFatosDoTipo(Fato.REFERENCIA)) {
36                     Relacao nova = new Relacao(ato.getIdentidade(),
37                         classificar(d.getTextoLimpo(), ref.getInicio(), ref.getFim(),
38                             ref.getValor(), ref.getInicio()));
39                     if (!arestas.contains(nova)) {
40                         arestas.add(nova);
41                     }
42                 }
43             }
44         }
45     }
46
47     private String classificar(String texto, int inicioCitacao, int fimCitacao) {
48         String antes = trecho(texto, Math.max(0, inicioCitacao - JANELA_ANTES), inicioCitacao);
49         String depois = trecho(texto, fimCitacao,
50             Math.min(texto.length(), fimCitacao + JANELA_DEPOIS));
51
52         String comecoDepois = primeiros(depois, TRECHO_DEPOIS);
53         if (comecoDepois.contains("fica revogad") || comecoDepois.contains("ficam revogad")) {
54             return Relacao.REVOGA;
55         }
56         if (comecoDepois.contains("passa a vigorar") || comecoDepois.contains("passam a vigorar")) {
57             return Relacao.ALTERA;
58         }
59
60         if (ehFundamento(antes)) {
61             return Relacao.CITA;
62         }
63
64         String fim = ultimos(antes, TRECHO_VERBO);
65         if (operou(fim, "revogad", "revoga")) {
66             return Relacao.REVOGA;
67         }
68         if (operou(fim, "prorrogad", "prorroga")) {
69             return Relacao.PRORROGA;
70         }
71         if (operou(fim, "alterad", "altera") || fim.contains("nova redacao")) {
72             return Relacao.ALTERA;
73         }
74         return Relacao.CITA;
75     }
76
77     private boolean operou(String trecho, String participio, String verbo) {
78         return trecho.contains("fica " + participio) || trecho.contains("ficam " + participio)
79             || trecho.contains(verbo + " o ") || trecho.contains(verbo + " a ");
80     }
81
82     private boolean ehFundamento(String antes) {
83         String fim = ultimos(antes, TRECHO_FUNDAMENTO);
84         for (String marca : FUNDAMENTO) {
85             if (fim.contains(marca)) {
86                 return true;
87             }
88         }
89         return false;
90     }
91
92     private String primeiros(String texto, int tamanho) {
93         return texto.substring(0, tamanho);
94     }
95
96     private String ultimos(String texto, int tamanho) {
97         return texto.substring(texto.length() - tamanho, texto.length());
98     }
99
100     private String trecho(String texto, int inicio, int fim) {
101         return texto.substring(inicio, fim);
102     }
103 }
```

```

87         }
88     }
89     return false;
90 }
91
92 private String trecho(String texto, int de, int ate) {
93     return Extrator.semAcento(texto.substring(de, ate).toLowerCase().replace('\n', ' '));
94 }
95
96 private String primeiros(String s, int n) {
97     return s.length() > n ? s.substring(0, n) : s;
98 }
99
100 private String ultimos(String s, int n) {
101     return s.length() > n ? s.substring(s.length() - n) : s;
102 }
103
104 public ArrayList<Relacao> getArestas() {
105     return arestas;
106 }
107
108 public ArrayList<Relacao> doTipo(String acao) {
109     ArrayList<Relacao> lista = new ArrayList<>();
110     for (Relacao r : arestas) {
111         if (r.getAcao().equals(acao)) {
112             lista.add(r);
113         }
114     }
115     return lista;
116 }
117
118 public ArrayList<String[]> maisAlteradas() {
119     ArrayList<String> alvos = new ArrayList<>();
120     ArrayList<Integer> contagem = new ArrayList<>();
121
122     for (Relacao r : arestas) {
123         if (r.getAcao().equals(Relacao.CITA)) {
124             continue;
125         }
126         int i = alvos.indexOf(r.getAlvo());
127         if (i < 0) {
128             alvos.add(r.getAlvo());
129             contagem.add(1);
130         } else {
131             contagem.set(i, contagem.get(i) + 1);
132         }
133     }
134
135     ArrayList<String[]> linhas = new ArrayList<>();
136     boolean[] usado = new boolean[alvos.size()];
137     for (int n = 0; n < alvos.size(); n++) {
138         int melhor = -1;
139         for (int i = 0; i < alvos.size(); i++) {
140             if (!usado[i] && (melhor < 0 || contagem.get(i) > contagem.get(melhor))) {
141                 melhor = i;
142             }
143         }
144         usado[melhor] = true;
145         linhas.add(new String[] {alvos.get(melhor), String.valueOf(contagem.get(melhor))});
146     }
147     return linhas;
148 }
149
150 public ArrayList<String> vizinhanca(String norma, int niveis) {
151     ArrayList<String> visitados = new ArrayList<>();
152     ArrayList<String> fronteira = new ArrayList<>();
153     visitados.add(norma);
154     fronteira.add(norma);
155
156     for (int nivel = 0; nivel < niveis && !fronteira.isEmpty(); nivel++) {
157         ArrayList<String> proxima = new ArrayList<>();
158         for (String atual : fronteira) {
159             for (Relacao r : arestas) {
160                 String outro = null;
161                 if (r.getAlvo().equals(atual)) {
162                     outro = r.getOrigem();
163                 } else if (r.getOrigem().equals(atual)) {
164                     outro = r.getAlvo();
165                 }
166                 if (outro != null && !visitados.contains(outro)) {
167                     visitados.add(outro);
168                     proxima.add(outro);
169                 }
170             }
171         }
172         fronteira = proxima;
173     }
174     return visitados;
175 }
176 }

```

ARQUIVO: diarios/analise/LeituraDoAto.java

pacote diarios.analise · 156 linhas · Escreve, em português, o que cada ato faz.

```
1 package diarios.analise;
2
3 import diarios.extratores.Extrator;
4 import diarios.modelo.Ato;
5 import diarios.modelo.Contrato;
6 import diarios.modelo.Fato;
7 import diarios.modelo.Relacao;
8 import java.util.ArrayList;
9
10 public class LeituraDoAto {
11
12     private static final String[][] PESSOAL = {
13         {"exonera", "exoneracao"}, {"nomea", "nomeacao"}, {"nomeia", "nomeacao"},
14         {"designa", "designacao"}, {"aposentadoria", "aposentadoria"},
15         {"licenca", "concessao de licenca"}, {"ferias", "ferias"}
16     };
17
18     private static final String[][] LICITACAO = {
19         {"homologa", "homologacao"}, {"adjudica", "adjudicacao"},
20         {"pregao", "pregao"}, {"dispensa de licitacao", "dispensa de licitacao"},
21         {"inexigibilidade", "inexigibilidade"}, {"chamamento publico", "chamamento publico"},
22         {"concorrenca", "concorrenca"}
23     };
24
25     private final PaineisContratos painel;
26     private final GrafoNormas grafo;
27
28     public LeituraDoAto(PaineisContratos painel, GrafoNormas grafo) {
29         this.painel = painel;
30         this.grafo = grafo;
31     }
32
33     public String ler(Ato ato, String dataDoDiario) {
34         String texto = Extrator.semAcento(ato.getTexto().toLowerCase());
35         String assunto = Assunto.de(ato);
36         StringBuilder s = new StringBuilder();
37
38         if (assunto.equals(Assunto.CONTRATACAO)) {
39             s.append(lerContratacao(ato));
40         } else if (assunto.equals(Assunto.NORMA)) {
41             s.append(lerNorma(ato));
42         } else if (assunto.equals(Assunto.PESSOAL)) {
43             s.append("Ato de pessoal: ").append(primeiraMarca(texto, PESSOAL, "movimentacao de servidor"))
44                 .append(".");
45         } else if (assunto.equals(Assunto.LICITACAO)) {
46             s.append("Ato de licitacao: ").append(primeiraMarca(texto, LICITACAO, "procedimento licitatorio"))
47                 .append(".");
48         } else if (assunto.equals(Assunto.ORCAMENTO)) {
49             s.append(lerOrcamento(ato));
50         } else {
51             s.append("Ato sem categoria reconhecida: ").append(ato.getFatos().size())
52                 .append(" fato(s) extraido(s).");
53         }
54
55         if (texto.contains("entra em vigor na data de sua publicacao")
56             || texto.contains("entra em vigor na data da sua publicacao")) {
57             s.append(" Vigora desde a publicacao ").append(dataDoDiario).append(".");
58         }
59         if (ato.getDecisao().equals(Ato.DUVIDOSO)) {
60             s.append(" Leitura com ressalva: ").append(ato.getMotivo()).append(".");
61         }
62         return s.toString();
63     }
64
65     private String lerContratacao(Ato ato) {
66         ArrayList<Contrato> doAto = contratosDoAto(painel.getContratos(), ato);
67         if (doAto.isEmpty()) {
68             ArrayList<Contrato> multas = contratosDoAto(painel.getDescartados(), ato);
69             if (!multas.isEmpty()) {
70                 return "Notificacao ou multa envolvendo " + multas.size()
71                     + " CNPJ(s); nao conta como contratacao.";
72             }
73             return "Menciona empresa e valor, mas o papel de cada parte nao ficou claro.";
74         }
75         double total = 0;
76         for (Contrato c : doAto) {
77             total += c.getValor();
78         }
79         Contrato principal = doAto.get(0);
80         for (Contrato c : doAto) {
81             if (c.getValor() > principal.getValor()) {
82                 principal = c;
83             }
84         }
85         StringBuilder s = new StringBuilder();
86         if (doAto.size() == 1) {
```

```

87         s.append("Contrata ").append(quem(principal)).append(" por ")
88         .append(Contrato.formatarReal(total)).append(".");
89     } else {
90         s.append(doAto.size()).append(" contratacoes somando ")
91         .append(Contrato.formatarReal(total)).append("; a maior com ")
92         .append(quem(principal)).append(".");
93     }
94     if (!principal.getProcesso().equals("-")) {
95         s.append(" Processo ").append(principal.getProcesso()).append(".");
96     }
97     return s.toString();
98 }
99
100 private String quem(Contrato c) {
101     if (c.getContratada().startsWith("(")) {
102         return "a empresa de CNPJ " + c.getCnpj() + " (nome nao lido)";
103     }
104     return c.getContratada() + " (" + c.getCnpj() + ")";
105 }
106
107 private String lerNorma(Ato ato) {
108     ArrayList<String> acoes = new ArrayList<>();
109     int citacoes = 0;
110     for (Relacao r : grafo.getArestas()) {
111         if (!r.getOrigem().equals(ato.getIdentidade())) {
112             continue;
113         }
114         if (r.getAcao().equals(Relacao.CITA)) {
115             citacoes++;
116         } else {
117             String frase = r.getAcao().toLowerCase() + " " + r.getAlvo();
118             if (!acoes.contains(frase)) {
119                 acoes.add(frase);
120             }
121         }
122     }
123     if (acoes.isEmpty()) {
124         return "Ato normativo que cria regra propria; cita " + citacoes
125             + " norma(s) como fundamento.";
126     }
127     return "Ato normativo que " + String.join(" ", acoes) + ".";
128 }
129
130 private String lerOrcamento(Ato ato) {
131     ArrayList<Fato> valores = ato.getFatosDoTipo(Fato.VALOR);
132     if (valores.isEmpty()) {
133         return "Ato orcamentario (credito ou dotacao), sem valor identificado.";
134     }
135     return "Ato orcamentario movimentando " + valores.get(0).getValor() + ".";
136 }
137
138 private ArrayList<Contrato> contratosDoAto(ArrayList<Contrato> lista, Ato ato) {
139     ArrayList<Contrato> achados = new ArrayList<>();
140     for (Contrato c : lista) {
141         if (c.getIdentidadeAto().equals(ato.getIdentidade())) {
142             achados.add(c);
143         }
144     }
145     return achados;
146 }
147
148 private String primeiraMarca(String texto, String[][] marcas, String padrao) {
149     for (String[] par : marcas) {
150         if (texto.contains(par[0])) {
151             return par[1];
152         }
153     }
154     return padrao;
155 }
156 }

```

ARQUIVO: diarios/analise/Interpretacao.java

pacote diarios.analise · 413 linhas · Gera as frases do "o que esses dados dizem" a partir de regras e limites.

```
1 package diarios.analise;
2
3 import diarios.modelo.Achado;
4 import diarios.modelo.Ato;
5 import diarios.modelo.Contrato;
6 import diarios.modelo.Diario;
7 import diarios.modelo.Fato;
8 import diarios.modelo.Relacao;
9 import java.util.ArrayList;
10
11 public class Interpretacao {
12
13     private static final int CONCENTRACAO_ALTA = 60;
14     private static final int UM_SO_CONTRATO_ALTO = 30;
15     private static final int REPETICAO_MINIMA = 3;
16     private static final int SEM_PROCESSO_ALTO = 30;
17     private static final int DUVIDA_ALTA = 25;
18
19     private final ArrayList<Achado> achados = new ArrayList<>();
20     private String resumo = "";
21
22     public void limpar() {
23         achados.clear();
24         resumo = "";
25     }
26
27     public void interpretar(ArrayList<Diario> diarios, Cobertura cobertura,
28                             PaineisContratos painel, GrafoNormas grafo) {
29         limpar();
30         montarResumo(diarios, painel, grafo);
31         perfilDoDiario(diarios);
32         concentracaoDeGasto(painel);
33         contratoDominante(painel);
34         fornecedorRepetido(painel);
35         rastreabilidade(painel);
36         papeisSeparados(painel);
37         normasDesatualizadas(grafo);
38         buracosDeCobertura(cobertura);
39         oQueNaoFoiEntendido(diarios, cobertura);
40     }
41
42     private void montarResumo(ArrayList<Diario> diarios, PaineisContratos painel,
43                               GrafoNormas grafo) {
44         int atos = 0;
45         String periodo = periodoDe(diarios);
46         ArrayList<String> municipios = new ArrayList<>();
47
48         for (Diario d : diarios) {
49             if (!d.isUtilizavel()) {
50                 continue;
51             }
52             atos += d.getAtos().size();
53             if (!municipios.contains(d.getMunicipio())) {
54                 municipios.add(d.getMunicipio());
55             }
56         }
57         if (atos == 0) {
58             resumo = "Nenhum ato pode ser lido no periodo.";
59             return;
60         }
61
62         int contratacoes = painel.getContratos().size();
63         int atosDeContrato = contarAtosDoAssunto(diarios, Assunto.CONTRATACAO);
64         int operacoes = grafo.doTipo(Relacao.ALTERA).size()
65             + grafo.doTipo(Relacao.REVOGA).size()
66             + grafo.doTipo(Relacao.PRORROGA).size();
67         int normasAtingidas = grafo.maisAlteradas().size();
68
69         StringBuilder s = new StringBuilder();
70         s.append("Em ").append(periodo).append(", ");
71         s.append(municipios.size() == 1 ? municipios.get(0) : municipios.size() + " municipios");
72         s.append(" publicou ").append(atos).append(" atos. ");
73
74         if (contratacoes > 0) {
75             s.append("Em ").append(atosDeContrato)
76                 .append(atosDeContrato == 1 ? " deles ha " : " deles ha ")
77                 .append(contratacoes).append(contratacoes == 1 ? " contratacao" : " contratacoes")
78                 .append(", somando ").append(Contrato.formatarReal(painel.total()).append(". ");
79         }
80         if (operacoes > 0) {
81             s.append(operacoes).append(operacoes == 1 ? " alteracao atinge " : " alteracoes atingem ")
82                 .append(normasAtingidas)
83                 .append(normasAtingidas == 1 ? " norma anterior. " : " normas anteriores. ");
84         } else {
85             s.append("Nenhuma norma anterior foi tocada. ");
86         }
87     }
88 }
```

```

87         resumo = s.toString().trim();
88     }
89
90     private void perfilDoDiario(ArrayList<Diario> diarios) {
91         int total = 0;
92         int contratacao = 0;
93         int pessoal = 0;
94         int norma = 0;
95         int licitacao = 0;
96         int orcamento = 0;
97
98         for (Diario d : diarios) {
99             if (!d.isUtilizavel()) {
100                 continue;
101             }
102             for (Ato a : d.getAtos()) {
103                 total++;
104                 String assunto = Assunto.de(a);
105                 if (assunto.equals(Assunto.CONTRATACAO)) {
106                     contratacao++;
107                 } else if (assunto.equals(Assunto.PESSOAL)) {
108                     pessoal++;
109                 } else if (assunto.equals(Assunto.NORMA)) {
110                     norma++;
111                 } else if (assunto.equals(Assunto.LICITACAO)) {
112                     licitacao++;
113                 } else if (assunto.equals(Assunto.ORDENAMENTO)) {
114                     orcamento++;
115                 }
116             }
117         }
118         if (total == 0) {
119             return;
120         }
121
122         int gastoPct = (contratacao + licitacao) * 100 / total;
123         int normaPct = norma * 100 / total;
124         String tipo = gastoPct > normaPct * 3
125             ? "um diario de EXECUCAO, nao de legislacao"
126             : "um diario com peso legislativo relevante";
127
128         achados.add(new Achado(Achado.LEITURA,
129             "O que este diario e",
130             "E " + tipo + ": " + gastoPct + "% dos atos tratam de gastar dinheiro "
131             + "(contrato ou licitacao) e " + normaPct + "% mexem em norma. "
132             + "Quem acompanha a prefeitura por aqui esta vendo, sobretudo, "
133             + "para onde o dinheiro vai.",
134             "de " + total + " atos: " + contratacao + " contratacoes, " + licitacao
135             + " licitacoes, " + pessoal + " pessoal, " + norma + " normas, "
136             + orcamento + " orcamento"));
137     }
138
139     private void concentracaoDeGasto(PainelContratos painel) {
140         ArrayList<String[]> ranking = painel.ranking();
141         double total = painel.total();
142         if (ranking.size() < 3 || total <= 0) {
143             return;
144         }
145
146         double top3 = 0;
147         StringBuilder nomes = new StringBuilder();
148         for (int i = 0; i < 3; i++) {
149             top3 += valorDe(ranking.get(i)[2]);
150             nomes.append(i > 0 ? ", " : "").append(ranking.get(i)[1]);
151         }
152         int pct = (int) Math.round(top3 * 100 / total);
153
154         String leitura = pct >= CONCENTRACAO_ALTA
155             ? "O gasto do periodo esta CONCENTRADO: tres empresas ficam com a "
156             + "maior parte do valor, enquanto as outras " + (ranking.size() - 3)
157             + " dividem o resto."
158             : "O gasto do periodo esta DISTRIBUIDO entre os " + ranking.size()
159             + " fornecedores: nenhum trio domina o valor.";
160
161         achados.add(new Achado(pct >= CONCENTRACAO_ALTA ? Achado.ATENCAO : Achado.LEITURA,
162             "Concentracao do gasto",
163             leitura + " Os tres maiores somam " + pct + "% do total.",
164             nomes + " = " + Contrato.formatarReal(top3) + " de "
165             + Contrato.formatarReal(total));
166     }
167
168     private void contratoDominante(PainelContratos painel) {
169         ArrayList<Contrato> contratos = painel.getContratos();
170         double total = painel.total();
171         if (contratos.isEmpty() || total <= 0) {
172             return;
173         }
174
175         Contrato maior = contratos.get(0);
176         for (Contrato c : contratos) {

```

```

177         if (c.getValor() > maior.getValor()) {
178             maior = c;
179         }
180     }
181     int pct = (int) Math.round(maior.getValor() * 100 / total);
182     if (pct < UM_SO_CONTRATO_ALTO) {
183         return;
184     }
185
186     achados.add(new Achado(Achado.ATENCAO,
187         "Um contrato pesa mais que todos os outros",
188         "Um unico contrato responde por " + pct + "% de tudo que foi contratado "
189         + "no periodo. Os outros " + (contratos.size() - 1)
190         + " contratos somados valem menos que ele.",
191         maior.getContratada() + " (" + maior.getCnpj() + ") = "
192         + Contrato.formatarReal(maior.getValor()) + ", processo "
193         + maior.getProcesso()));
194 }
195
196 private void fornecedorRepetido(PainelContratos painel) {
197     ArrayList<String[]> repetidos = painel.repeticoes(REPETICAO_MINIMA);
198     if (repetidos.isEmpty()) {
199         return;
200     }
201
202     StringBuilder lista = new StringBuilder();
203     for (String[] r : repetidos) {
204         lista.append(lista.length() > 0 ? "; " : "").append(r[1]).append(" ")
205             .append(r[3]).append("x");
206     }
207
208     achados.add(new Achado(Achado.ATENCAO,
209         "Fornecedor que se repete",
210         repetidos.size() + (repetidos.size() == 1 ? " fornecedor aparece" : " fornecedores aparecem")
211         + " em tres ou mais contratos do mesmo periodo. Isso tem explicacao "
212         + "legitima (ata de registro de precos, fornecedor unico de um insumo) "
213         + "e nao e acusacao de nada - e o lugar onde vale a pena olhar o "
214         + "processo original.",
215         lista.toString()));
216 }
217
218 private void rastreabilidade(PainelContratos painel) {
219     ArrayList<Contrato> contratos = painel.getContratos();
220     if (contratos.isEmpty()) {
221         return;
222     }
223     int semProcesso = 0;
224     for (Contrato c : contratos) {
225         if (c.getProcesso().equals("-")) {
226             semProcesso++;
227         }
228     }
229     int pct = semProcesso * 100 / contratos.size();
230     if (pct < SEM_PROCESSO_ALTO) {
231         return;
232     }
233
234     achados.add(new Achado(Achado.LIMITE,
235         "Contrato sem processo identificado",
236         pct + "% das contratacoes nao trazem numero de processo no texto "
237         + "publicado. Sem ele nao da para ligar o contrato a licitacao que o "
238         + "originou - nem pelo programa, nem por uma pessoa lendo o diario.",
239         semProcesso + " de " + contratos.size() + " contratacoes"));
240 }
241
242 private void papeisSeparados(PainelContratos painel) {
243     int prefeitura = painel.contarDescartadosDoPapel(PainelContratos.LADO_MUNICIPIO);
244     int notificados = painel.contarDescartadosDoPapel(PainelContratos.NOTIFICADO);
245     if (prefeitura + notificados == 0) {
246         return;
247     }
248
249     achados.add(new Achado(Achado.LIMITE,
250         "Nem todo CNPJ com valor ao lado e um contrato",
251         (prefeitura + notificados) + " CNPJs apareciam junto de um valor mas nao "
252         + "entram no total contratado: " + prefeitura + " sao o proprio orgao "
253         + "municipal aplicando multa, e " + notificados + " sao empresas sendo "
254         + "multadas. Somar tudo daria um numero maior e errado - multa nao e "
255         + "compra, e o orgao nao e fornecedor de si mesmo.",
256         "separados pelo papel escrito antes do CNPJ: NOTIFICANTE, NOTIFICADO, "
257         + "CONTRATADA, DETENTORA"));
258 }
259
260 private void normasDesatualizadas(GrafoNormas grafo) {
261     ArrayList<String[]> mexidas = grafo.maisAlteradas();
262     if (mexidas.isEmpty()) {
263         return;
264     }
265
266     StringBuilder lista = new StringBuilder();

```

```

267     int mostrar = Math.min(5, mexidas.size());
268     for (int i = 0; i < mostrar; i++) {
269         lista.append(lista.length() > 0 ? " : " : "")
270             .append(mexidas.get(i)[0]).append(" ").append(mexidas.get(i)[1]).append("x");
271     }
272
273     achados.add(new Achado(Achado.LEITURA,
274         "Normas que mudaram de texto",
275         mexidas.size() + (mexidas.size() == 1
276             ? " norma anterior foi alterada, revogada ou prorrogada"
277             : " normas anteriores foram alteradas, revogadas ou prorrogadas")
278         + " neste periodo. Quem estiver "
279         + "trabalhando com a versao antiga do texto esta desatualizado, e nao "
280         + "tem como saber disso lendo so a norma original - a mudanca esta no "
281         + "diario, nao nela.",
282         lista.toString()));
283 }
284
285 private void buracosDeCobertura(Cobertura cobertura) {
286     if (cobertura.getRecusados() == 0) {
287         return;
288     }
289     int pct = cobertura.getRecusados() * 100 / cobertura.getLidos();
290
291     achados.add(new Achado(Achado.LIMITE,
292         "0 que nao deu para ler",
293         pct + "% dos diarios do periodo nao puderam ser lidos - sao PDFs "
294         + "escaneados, sem texto por baixo. Tudo que este relatorio diz vale "
295         + "para os outros " + (100 - pct) + "%". Nao e que nao houve publicacao "
296         + "nesses dias: e que o programa nao consegue enxerga-la.",
297         cobertura.getRecusados() + " de " + cobertura.getLidos() + " diarios"));
298 }
299
300 private void oQueNaoFoiEntendido(ArrayList<Diario> diarios, Cobertura cobertura) {
301     if (cobertura.getTotalAtos() == 0) {
302         return;
303     }
304     int pct = cobertura.getDuvidosos() * 100 / cobertura.getTotalAtos();
305     if (pct < DUVIDA_ALTA) {
306         return;
307     }
308
309     ArrayList<String> motivos = new ArrayList<>();
310     ArrayList<Integer> vezes = new ArrayList<>();
311     for (Diario d : diarios) {
312         for (Ato a : d.getAtos()) {
313             if (!a.getDecisao().equals(Ato.DUVIDOSO)) {
314                 continue;
315             }
316             String motivo = resumirMotivo(a.getMotivo());
317             int i = motivos.indexOf(motivo);
318             if (i < 0) {
319                 motivos.add(motivo);
320                 vezes.add(1);
321             } else {
322                 vezes.set(i, vezes.get(i) + 1);
323             }
324         }
325     }
326
327     StringBuilder lista = new StringBuilder();
328     for (int i = 0; i < motivos.size(); i++) {
329         lista.append(lista.length() > 0 ? " : " : "")
330             .append(vezes.get(i)).append("x ").append(motivos.get(i));
331     }
332
333     achados.add(new Achado(Achado.LIMITE,
334         "0 que o programa ainda nao trata",
335         pct + "% dos atos ficaram em duvida. Eles nao foram descartados nem "
336         + "contados como certos: estao separados, com o motivo de cada um. "
337         + "Esta e a lista do que falta melhorar, e ela so existe porque o "
338         + "portao tem tres saidas em vez de duas.",
339         lista.toString()));
340 }
341
342 private int contarAtosDoAssunto(ArrayList<Diario> diarios, String assunto) {
343     int n = 0;
344     for (Diario d : diarios) {
345         if (!d.isUtilizavel()) {
346             continue;
347         }
348         for (Ato a : d.getAtos()) {
349             if (Assunto.de(a).equals(assunto)) {
350                 n++;
351             }
352         }
353     }
354     return n;
355 }
356

```

```
357     private String resumirMotivo(String motivo) {
358         int parenteses = motivo.indexOf('(');
359         return parenteses > 0 ? motivo.substring(0, parenteses).trim() : motivo;
360     }
361
362     private String periodoDe(ArrayList<Diario> diarios) {
363         String menor = null;
364         String maior = null;
365         for (Diario d : diarios) {
366             if (menor == null || d.getData().compareTo(menor) < 0) {
367                 menor = d.getData();
368             }
369             if (maior == null || d.getData().compareTo(maior) > 0) {
370                 maior = d.getData();
371             }
372         }
373         if (menor == null) {
374             return "periodo vazio";
375         }
376         return menor.equals(maior) ? menor : menor + " a " + maior;
377     }
378
379     private double valorDe(String formatado) {
380         StringBuilder n = new StringBuilder();
381         for (int i = 0; i < formatado.length(); i++) {
382             char c = formatado.charAt(i);
383             if (c >= '0' && c <= '9') {
384                 n.append(c);
385             } else if (c == ',') {
386                 n.append('.');
387             }
388         }
389         try {
390             return Double.parseDouble(n.toString());
391         } catch (NumberFormatException e) {
392             return 0.0;
393         }
394     }
395
396     public String getResumo() {
397         return resumo;
398     }
399
400     public ArrayList<Achado> getAchados() {
401         return achados;
402     }
403
404     public ArrayList<Achado> doNivel(String nivel) {
405         ArrayList<Achado> lista = new ArrayList<>();
406         for (Achado a : achados) {
407             if (a.getNivel().equals(nivel)) {
408                 lista.add(a);
409             }
410         }
411         return lista;
412     }
413 }
```

ARQUIVO: diarios/analise/Cobertura.java

pacote diarios.analise · 78 linhas · Relatório do que foi lido, do que foi recusado e dos totais do portão.

```
1 package diarios.analise;
2
3 import diarios.modelo.Ato;
4 import diarios.modelo.Diario;
5 import java.util.ArrayList;
6
7 public class Cobertura {
8
9     private final ArrayList<String> linhas = new ArrayList<>();
10
11     private int lidos = 0;
12     private int utilizaveis = 0;
13     private int recusados = 0;
14     private int totalAtos = 0;
15     private int aceitos = 0;
16     private int duvidosos = 0;
17     private int rejeitados = 0;
18
19     public void limpar() {
20         linhas.clear();
21         lidos = utilizaveis = recusados = 0;
22         totalAtos = aceitos = duvidosos = rejeitados = 0;
23     }
24
25     public void registrarRecusa(String arquivo, String motivo, String evidencia) {
26         lidos++;
27         recusados++;
28         linhas.add(" " + arquivo);
29         linhas.add("      RECUSADO: " + motivo);
30         linhas.add("      evidencia: " + evidencia);
31         linhas.add("");
32     }
33
34     public void registrarProcessado(Diario d) {
35         lidos++;
36         utilizaveis++;
37         totalAtos += d.getAtos().size();
38         aceitos += d.contar(Ato.ACEITO);
39         duvidosos += d.contar(Ato.DUVIDOSO);
40         rejeitados += d.contar(Ato.REJEITADO);
41
42         linhas.add(" " + d.getArquivo());
43         linhas.add(String.format("      limpeza      : %d linhas -> %d removidas como lixo",
44             d.getLinhasOriginais(), d.getLinhasRemovidas()));
45         linhas.add(String.format("      fragmentacao : %d titulos candidatos, %d descartados "
46             + "(sem corpo), %d atos",
47             d.getTitulosCandidatos(), d.getTitulosDescartados(), d.getAtos().size()));
48         linhas.add(String.format("      portao      : ACEITO %d - DUVIDOSO %d - REJEITADO %d",
49             d.contar(Ato.ACEITO), d.contar(Ato.DUVIDOSO), d.contar(Ato.REJEITADO)));
50         linhas.add("");
51     }
52
53     public ArrayList<String> montar() {
54         ArrayList<String> saida = new ArrayList<>();
55         saida.add("RELATORIO DE COBERTURA");
56         saida.add("=====");
57         saida.add("");
58         saida.add(String.format(" diarios lidos : %d", lidos));
59         saida.add(String.format(" utilizaveis   : %d", utilizaveis));
60         saida.add(String.format(" recusados     : %d", recusados));
61         saida.add("");
62         saida.addAll(linhas);
63         saida.add("TOTAIS");
64         saida.add(String.format(" atos        : %d", totalAtos));
65         saida.add(String.format(" ACEITO      : %d", aceitos));
66         saida.add(String.format(" DUVIDOSO    : %d", duvidosos));
67         saida.add(String.format(" REJEITADO   : %d", rejeitados));
68         return saida;
69     }
70
71     public int getLidos() { return lidos; }
72     public int getUtilizaveis() { return utilizaveis; }
73     public int getRecusados() { return recusados; }
74     public int getTotalAtos() { return totalAtos; }
75     public int getAceitos() { return aceitos; }
76     public int getDuvidosos() { return duvidosos; }
77     public int getRejeitados() { return rejeitados; }
78 }
```

ARQUIVO: diarios/coleta/Coletor.java

pacote diarios.coleta · 216 linhas · Fala com a API do Querido Diário (HttpClient), baixa os textos, tenta de novo.

```
1 package diarios.coleta;
2
3 import diarios.erros.ColetaException;
4 import java.io.IOException;
5 import java.net.ProxySelector;
6 import java.net.URI;
7 import java.net.http.HttpClient;
8 import java.net.http.HttpRequest;
9 import java.net.http.HttpResponse;
10 import java.nio.charset.StandardCharsets;
11 import java.nio.file.Files;
12 import java.nio.file.Path;
13 import java.nio.file.Paths;
14 import java.time.Duration;
15 import java.util.ArrayList;
16
17 public class Coletor {
18
19     private static final String API = "https://api.queridodiario.org.br/gazettes";
20     private static final String AGENTE = "ProjetoFaculdade/1.0 (trabalho academico)";
21     private static final int LIMITE = 50;
22     private static final int PAGINA_DESCOBERTA = 100;
23     private static final int PAGINAS_DESCOBERTA = 3;
24     private static final long PAUSA_MS = 1500;
25     private static final long[] ESPERAS_ENTRE_TENTATIVAS_MS = {5000, 15000, 45000};
26     private static final long ESPERA_MAXIMA_MS = 60000;
27
28     private final String pastaDestino;
29     private final HttpClient cliente;
30
31     public Coletor(String pastaDestino) {
32         this.pastaDestino = pastaDestino;
33         if (System.getProperty("https.proxyHost") == null) {
34             System.setProperty("java.net.useSystemProxies", "true");
35         }
36         this.cliente = HttpClient.newBuilder()
37             .proxy(ProxySelector.getDefault())
38             .connectTimeout(Duration.ofSeconds(20))
39             .followRedirects(HttpClient.Redirect.NORMAL)
40             .build();
41     }
42
43     public ResultadoColeta coletar(Municipio municipio, String desde, String ate) {
44         ResultadoColeta r = new ResultadoColeta(municipio);
45         String url = "?territory_ids=" + municipio.getCodigo()
46             + "&published_since=" + desde + "&published_until=" + ate
47             + "&size=" + LIMITE;
48
49         String resposta;
50         try {
51             resposta = consultarApi(url, "consulta");
52         } catch (ColetaException e) {
53             r.marcarFalhaDeRede(e.getMessage());
54             r.anotar(municipio.getNome() + ": " + e.getMessage());
55             return r;
56         }
57
58         ArrayList<String> gazetas = Json.objetos(resposta, "gazettes");
59         if (gazetas.isEmpty()) {
60             r.anotar(municipio.getNome() + ": nenhuma edicao de " + desde + " a " + ate);
61             return r;
62         }
63
64         for (String gazeta : gazetas) {
65             r.contarEdicao();
66             baixarEdicao(municipio, gazeta, r);
67         }
68         return r;
69     }
70
71     public ArrayList<Municipio> descobrir(String desde, String ate) throws ColetaException {
72         ArrayList<Municipio> achados = new ArrayList<>();
73         ArrayList<String> codigos = new ArrayList<>();
74
75         for (int pagina = 0; pagina < PAGINAS_DESCOBERTA; pagina++) {
76             String url = "?published_since=" + desde + "&published_until=" + ate
77                 + "&size=" + PAGINA_DESCOBERTA + "&offset=" + (pagina * PAGINA_DESCOBERTA);
78             ArrayList<String> gazetas = Json.objetos(consultarApi(url, "descoberta"), "gazettes");
79
80             for (String g : gazetas) {
81                 String codigo = Json.campo(g, "territory_id");
82                 if (!codigo.isEmpty() && !Json.campo(g, "txt_url").isEmpty()
83                     && !codigos.contains(codigo)) {
84                     codigos.add(codigo);
85                     achados.add(new Municipio(codigo, Json.campo(g, "territory_name"),
86                         Json.campo(g, "state_code")));
87                 }
88             }
89             try {
90                 Thread.sleep(PAUSA_MS);
91             } catch (InterruptedException e) {
92                 Thread.currentThread().interrupt();
93             }
94             if (esperaEntreTentativas()) {
95                 continue;
96             }
97         }
98         return achados;
99     }
100
101     private boolean esperaEntreTentativas() {
102         for (long espera : ESPERAS_ENTRE_TENTATIVAS_MS) {
103             try {
104                 Thread.sleep(espera);
105             } catch (InterruptedException e) {
106                 Thread.currentThread().interrupt();
107             }
108         }
109         return true;
110     }
111
112     private void baixarEdicao(Municipio municipio, String gazeta, ResultadoColeta r) {
113         String url = API + "/" + municipio.getCodigo() + "/" + gazeta;
114         HttpRequest request = HttpRequest.newBuilder()
115             .uri(URI.create(url))
116             .header("User-Agent", AGENTE)
117             .build();
118         HttpResponse<String> response = cliente.send(request, HttpResponse.BodyHandlers.ofString());
119         if (response.statusCode() != 200) {
120             r.marcarFalhaDeRede("Status: " + response.statusCode());
121             r.anotar(municipio.getNome() + ": falha ao baixar edicao " + gazeta);
122             return;
123         }
124         String texto = response.body();
125         Path path = Paths.get(pastaDestino, municipio.getNome(), gazeta + ".txt");
126         try {
127             Files.write(path, texto.getBytes(StandardCharsets.UTF_8));
128         } catch (IOException e) {
129             r.marcarFalhaDeRede("Erro ao salvar edicao " + gazeta);
130             r.anotar(municipio.getNome() + ": falha ao salvar edicao " + gazeta);
131         }
132     }
133
134     private String consultarApi(String url, String tipo) {
135         String resposta;
136         try {
137             resposta = cliente.send(HttpRequest.newBuilder()
138                 .uri(URI.create(url))
139                 .header("User-Agent", AGENTE)
140                 .build(), HttpResponse.BodyHandlers.ofString());
141         } catch (IOException e) {
142             throw new ColetaException("Erro ao consultar API: " + e.getMessage());
143         }
144         return resposta;
145     }
146 }
```

```

87         }
88     }
89     if (gazetas.size() < PAGINA_DESCOBERTA) {
90         break;
91     }
92     esperar();
93 }
94 return achados;
95 }
96
97 private void baixarEdicao(Municipio municipio, String gazeta, ResultadoColeta r) {
98     String data = Json.campo(gazeta, "date");
99     String urlTexto = Json.campo(gazeta, "txt_url");
100    String nome = municipio.getNome() + " " + data;
101
102    if (urlTexto.isEmpty()) {
103        r.contarSemTexto();
104        r.anoar(nome + ": so PDF, sem texto");
105        return;
106    }
107
108    Path destino = Paths.get(pastaDestino, municipio.getApelido() + "_" + data
109        + sufixoDeEdicao(gazeta) + ".txt");
110    if (Files.exists(destino)) {
111        r.contarJaExistia();
112        r.anoar(nome + ": ja estava na pasta");
113        return;
114    }
115
116    try {
117        String texto = buscarTexto(urlTexto, "download de " + data);
118        Files.createDirectories(destino.getParent());
119        Files.writeString(destino, texto, StandardCharsets.UTF_8);
120        r.contarBaixado();
121        r.anoar(nome + ": " + (texto.length() / 1024) + " KB -> " + destino.getFileName());
122    } catch (IOException e) {
123        r.anoar(nome + ": falha ao gravar - " + e.getMessage());
124    } catch (ColetaException e) {
125        r.anoar(nome + ": " + e.getMessage());
126    }
127    esperar();
128 }
129
130 private String sufixoDeEdicao(String gazeta) {
131     boolean extra = Json.campo(gazeta, "is_extra_edition").equals("true");
132     String edicao = Json.campo(gazeta, "edition").replaceAll("[^0-9A-Za-z]", "");
133     if (!extra) {
134         return "";
135     }
136     return "-extra" + (edicao.isEmpty() ? "" : edicao);
137 }
138
139 private String consultarApi(String parametros, String oQue) throws ColetaException {
140     String url = API + parametros;
141     for (int tentativa = 0; ; tentativa++) {
142         try {
143             String resposta = buscarTexto(url, oQue);
144             esperar();
145             return resposta;
146         } catch (ColetaException e) {
147             boolean acabaram = tentativa >= ESPERAS_ENTRE_TENTATIVAS_MS.length;
148             if (!e.podeTentarDeNovo() || acabaram) {
149                 throw new ColetaException(url, e.getMessage()
150                     + (tentativa > 0 ? " (depois de " + (tentativa + 1) + " tentativas)" : ""),
151                     e.podeTentarDeNovo(), e.getEsperaSugeridaMs());
152             }
153             long espera = Math.max(ESPERAS_ENTRE_TENTATIVAS_MS[tentativa], e.getEsperaSugeridaMs());
154             esperar(Math.min(espera, ESPERA_MAXIMA_MS));
155         }
156     }
157 }
158
159 private String buscarTexto(String url, String oQue) throws ColetaException {
160     HttpRequest pedido = HttpRequest.newBuilder()
161         .uri(URI.create(url))
162         .header("User-Agent", AGENTE)
163         .header("Accept", "application/json, text/plain, */*")
164         .timeout(Duration.ofSeconds(90))
165         .GET()
166         .build();
167
168     try {
169         HttpResponse<String> resposta =
170             cliente.send(pedido, HttpResponse.BodyHandlers.ofString(StandardCharsets.UTF_8));
171         int status = resposta.statusCode();
172         if (status != 200) {
173             boolean passageiro = status == 429 || status >= 500;
174             long sugerida = resposta.headers().firstValue("Retry-After")
175                 .map(this::segundosParaMs).orElse(0L);
176             throw new ColetaException(url, oQue + " respondeu HTTP " + status
177                 + (status == 429 ? " (limite de pedidos da API)" : ""));
178         }
179     }

```

```
177         + " [" + trechoDoCorpo(resposta.body()) + "]", passageiro, sugerida);
178     }
179     return resposta.body();
180 } catch (IOException e) {
181     throw new ColetaException(url, oQue + " falhou: " + e.getClass().getSimpleName()
182         + " " + e.getMessage(), true, 0);
183 } catch (InterruptedException e) {
184     Thread.currentThread().interrupt();
185     throw new ColetaException(url, oQue + " foi interrompido");
186 }
187 }
188
189 private long segundosParaMs(String valor) {
190     try {
191         return Long.parseLong(valor.trim()) * 1000;
192     } catch (NumberFormatException e) {
193         return 0;
194     }
195 }
196
197 private String trechoDoCorpo(String corpo) {
198     if (corpo == null) {
199         return "";
200     }
201     String limpo = corpo.replaceAll("<[^>*>", " ").replaceAll("\\s+", " ").trim();
202     return limpo.length() > 160 ? limpo.substring(0, 160) + "..." : limpo;
203 }
204
205 private void esperar() {
206     esperar(PAUSA_MS);
207 }
208
209 private void esperar(long ms) {
210     try {
211         Thread.sleep(ms);
212     } catch (InterruptedException e) {
213         Thread.currentThread().interrupt();
214     }
215 }
216 }
```

ARQUIVO: diarios/coleta/Json.java

pacote diarios.coleta · 109 linhas · Leitor de JSON feito à mão (indexOf / substring).

```
1 package diarios.coleta;
2
3 import java.util.ArrayList;
4
5 public class Json {
6
7     public static ArrayList<String> objetos(String resposta, String lista) {
8         ArrayList<String> partes = new ArrayList<>();
9         String marca = "\"" + lista + "\"";
10        int p = resposta.indexOf(marca);
11        if (p < 0) {
12            return partes;
13        }
14        int abre = resposta.indexOf('[', p);
15        if (abre < 0) {
16            return partes;
17        }
18
19        int profundidade = 0;
20        int inicio = -1;
21        boolean dentroDeTexto = false;
22
23        for (int i = abre; i < resposta.length(); i++) {
24            char c = resposta.charAt(i);
25
26            if (dentroDeTexto) {
27                if (c == '\\') {
28                    i++;
29                } else if (c == '"') {
30                    dentroDeTexto = false;
31                }
32                continue;
33            }
34            if (c == '"') {
35                dentroDeTexto = true;
36            } else if (c == '{') {
37                if (profundidade == 0) {
38                    inicio = i;
39                }
40                profundidade++;
41            } else if (c == '}') {
42                profundidade--;
43                if (profundidade == 0 && inicio >= 0) {
44                    partes.add(resposta.substring(inicio, i + 1));
45                    inicio = -1;
46                }
47            } else if (c == ']' && profundidade == 0) {
48                break;
49            }
50        }
51        return partes;
52    }
53
54    public static String campo(String objeto, String chave) {
55        String marca = "\"" + chave + "\"";
56        int p = objeto.indexOf(marca);
57        if (p < 0) {
58            return "";
59        }
60        int i = objeto.indexOf(':', p + marca.length());
61        if (i < 0) {
62            return "";
63        }
64        i++;
65        while (i < objeto.length() && objeto.charAt(i) == ' ') {
66            i++;
67        }
68        if (i >= objeto.length()) {
69            return "";
70        }
71        if (objeto.charAt(i) != '"') {
72            return valorSemAspas(objeto, i);
73        }
74        i++;
75
76        StringBuilder valor = new StringBuilder();
77        while (i < objeto.length()) {
78            char c = objeto.charAt(i);
79            if (c == '\\' && i + 1 < objeto.length()) {
80                char seguinte = objeto.charAt(i + 1);
81                if (seguinte == 'n') {
82                    valor.append('\n');
83                } else if (seguinte == 'u' && i + 5 < objeto.length()) {
84                    valor.append((char) Integer.parseInt(objeto.substring(i + 2, i + 6), 16));
85                    i += 4;
86                } else {
```

```
87         valor.append(seguinte);
88     }
89     i += 2;
90     continue;
91 }
92 if (c == '"') {
93     break;
94 }
95 valor.append(c);
96 i++;
97 }
98 return valor.toString();
99 }
100
101 private static String valorSemAspas(String objeto, int inicio) {
102     int fim = inicio;
103     while (fim < objeto.length() && ",,}".indexOf(objeto.charAt(fim)) < 0) {
104         fim++;
105     }
106     String valor = objeto.substring(inicio, fim).trim();
107     return valor.equals("null") ? "" : valor;
108 }
109 }
```

ARQUIVO: diarios/coleta/Municipio.java

pacote diarios.coleta · 44 linhas · Código IBGE, nome e UF; gera o "apelido" usado nos nomes de arquivo.

```
1 package diarios.coleta;
2
3 public class Municipio {
4
5     private final String codigo;
6     private final String nome;
7     private final String uf;
8
9     public Municipio(String codigo, String nome, String uf) {
10         this.codigo = codigo;
11         this.nome = nome;
12         this.uf = uf;
13     }
14
15     public String getCodigo() { return codigo; }
16     public String getNome() { return nome; }
17     public String getUf() { return uf; }
18
19     public String getApelido() {
20         StringBuilder s = new StringBuilder();
21         String base = nome.toLowerCase();
22         for (int i = 0; i < base.length(); i++) {
23             char c = base.charAt(i);
24             if (c == ' ') {
25                 s.append('-');
26             } else if (Character.isLetterOrDigit(c)) {
27                 s.append(semAcento(c));
28             }
29         }
30         return s.toString();
31     }
32
33     private char semAcento(char c) {
34         String comAcento = "àáâãäåêëíîóôõúç";
35         String sem = "aaaaeeiooouc";
36         int i = comAcento.indexOf(c);
37         return i >= 0 ? sem.charAt(i) : c;
38     }
39
40     @Override
41     public String toString() {
42         return nome + " (" + uf + ")";
43     }
44 }
```

ARQUIVO: diarios/coleta/Fonte.java

pacote diarios.coleta · 72 linhas · Catálogo das 27 capitais e do conjunto padrão de 5 fontes.

```
1 package diarios.coleta;
2
3 import java.util.ArrayList;
4
5 public class Fonte {
6
7     private static final String[][] CAPITAIS = {
8         {"1200401", "Rio Branco", "AC"},
9         {"2704302", "Maceió", "AL"},
10        {"1600303", "Macapá", "AP"},
11        {"1302603", "Manaus", "AM"},
12        {"2927408", "Salvador", "BA"},
13        {"2304400", "Fortaleza", "CE"},
14        {"5300108", "Brasília", "DF"},
15        {"3205309", "Vitória", "ES"},
16        {"5208707", "Goiânia", "GO"},
17        {"2111300", "São Luís", "MA"},
18        {"5103403", "Cuiabá", "MT"},
19        {"5002704", "Campo Grande", "MS"},
20        {"3106200", "Belo Horizonte", "MG"},
21        {"1501402", "Belém", "PA"},
22        {"2507507", "João Pessoa", "PB"},
23        {"4106902", "Curitiba", "PR"},
24        {"2611606", "Recife", "PE"},
25        {"2211001", "Teresina", "PI"},
26        {"3304557", "Rio de Janeiro", "RJ"},
27        {"2408102", "Natal", "RN"},
28        {"4314902", "Porto Alegre", "RS"},
29        {"1100205", "Porto Velho", "RO"},
30        {"1400100", "Boa Vista", "RR"},
31        {"4205407", "Florianópolis", "SC"},
32        {"3550308", "São Paulo", "SP"},
33        {"2800308", "Aracaju", "SE"},
34        {"1721000", "Palmas", "TO"}
35    };
36
37    private final ArrayList<Municipio> municipios = new ArrayList<>();
38
39    public Fonte() {
40        for (String[] linha : CAPITAIS) {
41            municipios.add(new Municipio(linha[0], linha[1], linha[2]));
42        }
43    }
44
45    public ArrayList<Municipio> todos() {
46        return municipios;
47    }
48
49    public Municipio porNome(String busca) {
50        String alvo = busca.trim().toLowerCase();
51        for (Municipio m : municipios) {
52            if (m.getNome().toLowerCase().startsWith(alvo)
53                || m.getApelido().startsWith(alvo)
54                || m.getCodigo().equals(alvo)) {
55                return m;
56            }
57        }
58        return null;
59    }
60
61    public ArrayList<Municipio> padrao() {
62        ArrayList<Municipio> lista = new ArrayList<>();
63        String[] codigos = {"3106200", "2611606", "2304400", "4106902", "2800308"};
64        for (String c : codigos) {
65            Municipio m = porNome(c);
66            if (m != null) {
67                lista.add(m);
68            }
69        }
70        return lista;
71    }
72 }
```

ARQUIVO: diarios/coleta/EstadoFonte.java

pacote diarios.coleta · 86 linhas · Estado de uma fonte: ligada?, status da última tentativa, período coberto.

```
1 package diarios.coleta;
2
3 public class EstadoFonte {
4
5     public static final String NUNCA_TENTADA = "NUNCA_TENTADA";
6     public static final String COM_DADO = "COM_DADO";
7     public static final String SEM_EDICAO = "SEM_EDICAO";
8     public static final String SO_PDF = "SO_PDF";
9     public static final String FALHOU = "FALHOU";
10
11     private final Municipio municipio;
12     private final boolean descoberta;
13     private boolean ligada;
14     private String status = NUNCA_TENTADA;
15     private String ultimaTentativa = "-";
16     private String motivo = "";
17     private int diariosNoAcervo = 0;
18     private String coberturaDesde = "";
19     private String coberturaAte = "";
20
21     public EstadoFonte(Municipio municipio, boolean ligada, boolean descoberta) {
22         this.municipio = municipio;
23         this.ligada = ligada;
24         this.descoberta = descoberta;
25     }
26
27     public void registrarResultado(String quando, ResultadoColeta r, String desde, String ate) {
28         ultimaTentativa = quando;
29         motivo = "";
30         coberturaDesde = "";
31         coberturaAte = "";
32         if (r.isFalhouRede()) {
33             status = FALHOU;
34             motivo = r.getMotivoFalha();
35             return;
36         }
37         coberturaDesde = desde;
38         coberturaAte = ate;
39         if (r.getBaixados() > 0 || r.getJaExistiam() > 0) {
40             status = COM_DADO;
41         } else if (r.getSemTexto() > 0) {
42             status = SO_PDF;
43         } else {
44             status = SEM_EDICAO;
45         }
46     }
47
48     public boolean jaCobre(String desde, String ate) {
49         boolean resolvida = status.equals(COM_DADO) || status.equals(SEM_EDICAO)
50             || status.equals(SO_PDF);
51         return resolvida && !coberturaDesde.isEmpty()
52             && coberturaDesde.compareTo(desde) <= 0 && coberturaAte.compareTo(ate) >= 0;
53     }
54
55     public void restaurarCobertura(String desde, String ate) {
56         this.coberturaDesde = desde;
57         this.coberturaAte = ate;
58     }
59
60     public void restaurar(String status, String ultimaTentativa, String motivo) {
61         this.status = status;
62         this.ultimaTentativa = ultimaTentativa;
63         this.motivo = motivo;
64     }
65
66     public void marcarComoVistaNaDescoberta(String quando) {
67         if (status.equals(NUNCA_TENTADA)) {
68             status = COM_DADO;
69             ultimaTentativa = quando;
70             motivo = "vista na descoberta";
71         }
72     }
73
74     public Municipio getMunicipio() { return municipio; }
75     public boolean isLigada() { return ligada; }
76     public boolean isDescoberta() { return descoberta; }
77     public String getStatus() { return status; }
78     public String getUltimaTentativa() { return ultimaTentativa; }
79     public String getMotivo() { return motivo; }
80     public int getDiariosNoAcervo() { return diariosNoAcervo; }
81     public String getCoberturaDesde() { return coberturaDesde; }
82     public String getCoberturaAte() { return coberturaAte; }
83
84     public void setLigada(boolean ligada) { this.ligada = ligada; }
85     public void setDiariosNoAcervo(int n) { this.diariosNoAcervo = n; }
86 }
```

ARQUIVO: diarios/coleta/Fontes.java

pacote diarios.coleta · 156 linhas · Lista de fontes + persistência em dados/estado/fontes.txt.

```
1 package diarios.coleta;
2
3 import java.io.IOException;
4 import java.nio.charset.StandardCharsets;
5 import java.nio.file.DirectoryStream;
6 import java.nio.file.Files;
7 import java.nio.file.Path;
8 import java.nio.file.Paths;
9 import java.util.ArrayList;
10
11 public class Fontes {
12
13     private static final String ARQUIVO = "fontes.txt";
14
15     private final Path arquivo;
16     private final ArrayList<EstadoFonte> estados = new ArrayList<>();
17
18     public Fontes(String pastaEstado) {
19         this.arquivo = Paths.get(pastaEstado, ARQUIVO);
20         Fonte catalogo = new Fonte();
21         boolean temArquivo = Files.exists(arquivo);
22         ArrayList<Municipio> padrao = catalogo.padrao();
23
24         for (Municipio m : catalogo.todos()) {
25             estados.add(new EstadoFonte(m, !temArquivo && contem(padrao, m.getCodigo()), false));
26         }
27         if (temArquivo) {
28             carregar();
29         }
30     }
31
32     public synchronized ArrayList<EstadoFonte> todas() {
33         return new ArrayList<>(estados);
34     }
35
36     public synchronized ArrayList<EstadoFonte> ligadas() {
37         ArrayList<EstadoFonte> lista = new ArrayList<>();
38         for (EstadoFonte e : estados) {
39             if (e.isLigada()) {
40                 lista.add(e);
41             }
42         }
43         return lista;
44     }
45
46     public synchronized EstadoFonte doCodigo(String codigo) {
47         for (EstadoFonte e : estados) {
48             if (e.getMunicipio().getCodigo().equals(codigo)) {
49                 return e;
50             }
51         }
52         return null;
53     }
54
55     public synchronized void alternar(String codigo) {
56         EstadoFonte e = doCodigo(codigo);
57         if (e != null) {
58             e.setLigada(!e.isLigada());
59             salvar();
60         }
61     }
62
63     public synchronized void ligarTodas(boolean ligar) {
64         for (EstadoFonte e : estados) {
65             e.setLigada(ligar);
66         }
67         salvar();
68     }
69
70     public synchronized int adicionarDescobertas(ArrayList<Municipio> achados, String quando) {
71         int novas = 0;
72         for (Municipio m : achados) {
73             EstadoFonte existente = doCodigo(m.getCodigo());
74             if (existente == null) {
75                 existente = new EstadoFonte(m, false, true);
76                 estados.add(existente);
77                 novas++;
78             }
79             existente.marcarComoVistaNaDescoberta(quando);
80         }
81         salvar();
82         return novas;
83     }
84
85     public synchronized void atualizarAcervo(String pastaDiarios) {
86         for (EstadoFonte e : estados) {
```

```

87         e.setDiariosNoAcervo(contarArquivos(pastaDiarios, e.getMunicipio().getApelido()));
88     }
89 }
90
91 public synchronized void salvar() {
92     ArrayList<String> linhas = new ArrayList<>();
93     for (EstadoFonte e : estados) {
94         Municipio m = e.getMunicipio();
95         linhas.add(String.join(";", m.getCodigo(), limpo(m.getNome()), m.getUf(),
96             String.valueOf(e.isLigada()), e.getStatus(), limpo(e.getUltimaTentativa()),
97             limpo(e.getMotivo()), String.valueOf(e.isDescoberta()),
98             e.getCoberturaDesde(), e.getCoberturaAte()));
99     }
100     try {
101         Files.createDirectories(arquivo.getParent());
102         Files.write(arquivo, linhas, StandardCharsets.UTF_8);
103     } catch (IOException ex) {
104         System.out.println("aviso: nao consegui salvar " + arquivo + ": " + ex.getMessage());
105     }
106 }
107
108 private void carregar() {
109     try {
110         for (String linha : Files.readAllLines(arquivo, StandardCharsets.UTF_8)) {
111             String[] c = linha.split(";", -1);
112             if (c.length < 8) {
113                 continue;
114             }
115             EstadoFonte e = doCodigo(c[0]);
116             if (e == null) {
117                 e = new EstadoFonte(new Municipio(c[0], c[1], c[2]), false, true);
118                 estados.add(e);
119             }
120             e.setLigada(Boolean.parseBoolean(c[3]));
121             e.restaurar(c[4], c[5], c[6]);
122             if (c.length >= 10) {
123                 e.restaurarCobertura(c[8], c[9]);
124             }
125         }
126     } catch (IOException ex) {
127         System.out.println("aviso: nao consegui ler " + arquivo + ": " + ex.getMessage());
128     }
129 }
130
131 private int contarArquivos(String pasta, String apelido) {
132     int n = 0;
133     try (DirectoryStream<Path> fluxo = Files.newDirectoryStream(Paths.get(pasta),
134         apelido + "/*.txt")) {
135         for (Path p : fluxo) {
136             n++;
137         }
138     } catch (IOException ex) {
139         return 0;
140     }
141     return n;
142 }
143
144 private boolean contem(ArrayList<Municipio> lista, String codigo) {
145     for (Municipio m : lista) {
146         if (m.getCodigo().equals(codigo)) {
147             return true;
148         }
149     }
150     return false;
151 }
152
153 private String limpo(String s) {
154     return s == null ? "" : s.replace(';', ',').replace('\n', ' ');
155 }
156 }

```

ARQUIVO: diarios/coleta/ResultadoColeta.java

pacote diarios.coleta · 42 linhas · Contadores e registro do que aconteceu com UMA fonte numa rodada.

```
1 package diarios.coleta;
2
3 import java.util.ArrayList;
4
5 public class ResultadoColeta {
6
7     private final Municipio municipio;
8     private final ArrayList<String> registro = new ArrayList<>();
9     private int baixados = 0;
10    private int semTexto = 0;
11    private int jaExistiam = 0;
12    private int edicoes = 0;
13    private boolean falhouRede = false;
14    private boolean pulada = false;
15    private String motivoFalha = "";
16
17    public ResultadoColeta(Municipio municipio) {
18        this.municipio = municipio;
19    }
20
21    public void anotar(String linha)    { registro.add(linha); }
22    public void contarEdicao()           { edicoes++; }
23    public void contarBaixado()         { baixados++; }
24    public void contarSemTexto()        { semTexto++; }
25    public void contarJaExistia()       { jaExistiam++; }
26
27    public void marcarFalhaDeRede(String motivo) {
28        falhouRede = true;
29        motivoFalha = motivo;
30    }
31
32    public Municipio getMunicipio()      { return municipio; }
33    public ArrayList<String> getRegistro() { return registro; }
34    public int getBaixados()             { return baixados; }
35    public int getSemTexto()             { return semTexto; }
36    public int getJaExistiam()           { return jaExistiam; }
37    public int getEdicoes()              { return edicoes; }
38    public boolean isFalhouRede()         { return falhouRede; }
39    public boolean isPulada()            { return pulada; }
40    public void marcarPulada()           { pulada = true; }
41    public String getMotivoFalha()       { return motivoFalha; }
42 }
```

ARQUIVO: diarios/coleta/Rodada.java

pacote diarios.coleta · 126 linhas · Uma coleta (ou descoberta) inteira; grava/relê uma linha em rodadas.txt.

```
1 package diarios.coleta;
2
3 import java.util.ArrayList;
4
5 public class Rodada {
6
7     private final String inicio;
8     private final String tipo;
9     private final int dias;
10    private final String desde;
11    private final String ate;
12    private final ArrayList<ResultadoColeta> resultados = new ArrayList<>();
13    private String fim = "";
14    private String observacao = "";
15
16    public Rodada(String inicio, String tipo, int dias, String desde, String ate) {
17        this.inicio = inicio;
18        this.tipo = tipo;
19        this.dias = dias;
20        this.desde = desde;
21        this.ate = ate;
22    }
23
24    public static Rodada daLinha(String linha) {
25        String[] c = linha.split(";", -1);
26        if (c.length < 11) {
27            return null;
28        }
29        Rodada r = new Rodada(c[0], c[1], Integer.parseInt(c[2]), c[3], c[4]);
30        r.fim = c[5];
31        r.observacao = c[10];
32        r.puladasSalvas = c.length >= 12 ? Integer.parseInt(c[11]) : 0;
33        r.resumoSalvo = new int[] {
34            Integer.parseInt(c[6]), Integer.parseInt(c[7]),
35            Integer.parseInt(c[8]), Integer.parseInt(c[9])
36        };
37        return r;
38    }
39
40    private int[] resumoSalvo = null;
41    private int puladasSalvas = 0;
42
43    public void adicionar(ResultadoColeta r) {
44        resultados.add(r);
45    }
46
47    public void finalizar(String quando, String observacao) {
48        this.fim = quando;
49        this.observacao = observacao;
50    }
51
52    public int fontes() {
53        return resumoSalvo != null ? resumoSalvo[0] : resultados.size();
54    }
55
56    public int baixados() {
57        if (resumoSalvo != null) {
58            return resumoSalvo[1];
59        }
60        int n = 0;
61        for (ResultadoColeta r : resultados) {
62            n += r.getBaixados();
63        }
64        return n;
65    }
66
67    public int jaExistiam() {
68        if (resumoSalvo != null) {
69            return resumoSalvo[2];
70        }
71        int n = 0;
72        for (ResultadoColeta r : resultados) {
73            n += r.getJaExistiam();
74        }
75        return n;
76    }
77
78    public int falhas() {
79        if (resumoSalvo != null) {
80            return resumoSalvo[3];
81        }
82        int n = 0;
83        for (ResultadoColeta r : resultados) {
84            if (r.isFalhouRede()) {
85                n++;
86            }
87        }
88    }
89 }
```

```
87         }
88         return n;
89     }
90
91     public int puladas() {
92         if (resumoSalvo != null) {
93             return puladasSalvas;
94         }
95         int n = 0;
96         for (ResultadoColeta r : resultados) {
97             if (r.isPulada()) {
98                 n++;
99             }
100         }
101         return n;
102     }
103
104     public ArrayList<String> registro() {
105         ArrayList<String> linhas = new ArrayList<>();
106         for (ResultadoColeta r : resultados) {
107             linhas.addAll(r.getRegistro());
108         }
109         return linhas;
110     }
111
112     public String paraLinha() {
113         return String.join(";", inicio, tipo, String.valueOf(dias), desde, ate, fim,
114             String.valueOf(fontes()), String.valueOf(baixados()),
115             String.valueOf(jaExistiam()), String.valueOf(falhas()),
116             observacao.replace(';', ' ').replace('\n', ' '), String.valueOf(puladas()));
117     }
118
119     public String getInicio()      { return inicio; }
120     public String getTipo()        { return tipo; }
121     public int getDias()           { return dias; }
122     public String getDesde()       { return desde; }
123     public String getAte()         { return ate; }
124     public String getFim()         { return fim; }
125     public String getObservacao() { return observacao; }
126 }
```

ARQUIVO: diarios/servidor/Servidor.java

pacote diarios.servidor · 224 linhas · Sobe o HttpServer do JDK em localhost e liga cada endereço a um método.

```
1 package diarios.servidor;
2
3 import com.sun.net.httpserver.HttpExchange;
4 import com.sun.net.httpserver.HttpServer;
5 import diarios.Aplicacao;
6 import java.awt.Desktop;
7 import java.io.IOException;
8 import java.io.OutputStream;
9 import java.net.BindException;
10 import java.net.InetSocketAddress;
11 import java.net.URI;
12 import java.nio.charset.StandardCharsets;
13
14 public class Servidor {
15
16     private static final int PORTA_PADRAO = 8080;
17     private static final int TENTATIVAS_DE_PORTA = 10;
18     private static final int DIAS_NA_PARTIDA = 1;
19
20     private final Aplicacao aplicacao;
21     private HttpServer http;
22     private int porta;
23
24     public Servidor(Aplicacao aplicacao) {
25         this.aplicacao = aplicacao;
26     }
27
28     public static void main(String[] args) {
29         boolean coletarNaPartida = true;
30         boolean abrirNavegador = true;
31         int porta = PORTA_PADRAO;
32         for (String a : args) {
33             if (a.equals("--sem-coleta")) {
34                 coletarNaPartida = false;
35             } else if (a.equals("--sem-navegador")) {
36                 abrirNavegador = false;
37             } else if (a.startsWith("--porta=")) {
38                 porta = Integer.parseInt(a.substring("--porta=".length()));
39             }
40         }
41
42         System.out.println("Diarios Oficiais - painel");
43         System.out.println("pasta de trabalho: " + System.getProperty("user.dir"));
44
45         Aplicacao aplicacao = new Aplicacao();
46         System.out.println("lendo os diarios que ja estao na pasta...");
47         aplicacao.processarPasta();
48
49         Servidor servidor = new Servidor(aplicacao);
50         if (!servidor.iniciar(porta)) {
51             return;
52         }
53
54         String url = "http://localhost:" + servidor.porta;
55         System.out.println();
56         System.out.println("PAINEL NO AR: " + url);
57         System.out.println("(para parar: botao vermelho do IntelliJ, ou Ctrl+C)");
58         System.out.println();
59
60         if (abrirNavegador) {
61             abrirNoNavegador(url);
62         }
63         if (coletarNaPartida) {
64             System.out.println("coletando " + DIAS_NA_PARTIDA + " dia(s) para tras "
65                 + "das fontes ligadas, em segundo plano...");
66             aplicacao.coletarEmSegundoPlano(DIAS_NA_PARTIDA);
67         }
68     }
69
70     public boolean iniciar(int portaDesejada) {
71         for (int p = portaDesejada; p < portaDesejada + TENTATIVAS_DE_PORTA; p++) {
72             try {
73                 http = HttpServer.create(new InetSocketAddress("localhost", p), 0);
74                 porta = p;
75                 break;
76             } catch (BindException e) {
77                 System.out.println("porta " + p + " ocupada, tentando a proxima...");
78             } catch (IOException e) {
79                 System.out.println("nao consegui abrir o servidor: " + e.getMessage());
80                 return false;
81             }
82         }
83         if (http == null) {
84             System.out.println("nenhuma porta livre entre " + portaDesejada + " e "
85                 + (portaDesejada + TENTATIVAS_DE_PORTA - 1));
86             return false;
87         }
88     }
89 }
```

```

87     }
88
89     http.createContext("/", this::tratarPainel);
90     http.createContext("/coleta", this::tratarPaginaColeta);
91     http.createContext("/coletar", this::tratarColetar);
92     http.createContext("/descobrir", this::tratarDescobrir);
93     http.createContext("/processar", this::tratarProcessar);
94     http.createContext("/fonte/alternar", this::tratarAlternar);
95     http.createContext("/fonte/todas", this::tratarTodas);
96     http.createContext("/diario", this::tratarDiario);
97     http.createContext("/ato", this::tratarAto);
98     http.createContext("/busca", this::tratarBusca);
99     http.setExecutor(null);
100    http.start();
101    return true;
102 }
103
104 public void parar() {
105     if (http != null) {
106         http.stop(0);
107     }
108 }
109
110 public int getPorta() {
111     return porta;
112 }
113
114 private void tratarPainel(HttpExchange troca) throws IOException {
115     if (!troca.getRequestURI().getPath().equals("/")) {
116         responder(troca, 404, PaginaDiario.paginaSimples("Nao encontrado",
117             "Endereco inexistente: " + troca.getRequestURI().getPath()));
118         return;
119     }
120     String aviso = Formulario.parametro(troca.getRequestURI().getRawQuery(), "aviso");
121     responder(troca, 200, new PainelWeb(aplicacao).montar(aviso));
122 }
123
124 private void tratarPaginaColeta(HttpExchange troca) throws IOException {
125     String aviso = Formulario.parametro(troca.getRequestURI().getRawQuery(), "aviso");
126     responder(troca, 200, new PainelWeb(aplicacao).montarColeta(aviso));
127 }
128
129 private void tratarColetar(HttpExchange troca) throws IOException {
130     if (!ehPost(troca)) {
131         redirecionar(troca, "/");
132         return;
133     }
134     String corpo = lerCorpo(troca);
135     int dias = Formulario.inteiro(corpo, "dias", 1);
136     boolean refazer = Formulario.parametro(corpo, "refazer").equals("sim");
137     boolean camecou = aplicacao.coletarEmSegundoPlano(dias, refazer);
138     redirecionar(troca, camecou ? "/coleta" : "/coleta?aviso=ocupado");
139 }
140
141 private void tratarDescobrir(HttpExchange troca) throws IOException {
142     if (!ehPost(troca)) {
143         redirecionar(troca, "/");
144         return;
145     }
146     boolean camecou = aplicacao.descobrirEmSegundoPlano();
147     redirecionar(troca, camecou ? "/coleta" : "/coleta?aviso=ocupado");
148 }
149
150 private void tratarProcessar(HttpExchange troca) throws IOException {
151     if (ehPost(troca) && !aplicacao.isOcupado()) {
152         aplicacao.processarPasta();
153     }
154     redirecionar(troca, "/coleta");
155 }
156
157 private void tratarAlternar(HttpExchange troca) throws IOException {
158     if (ehPost(troca)) {
159         aplicacao.alternarFonte(Formulario.parametro(lerCorpo(troca), "codigo"));
160     }
161     redirecionar(troca, "/coleta");
162 }
163
164 private void tratarTodas(HttpExchange troca) throws IOException {
165     if (ehPost(troca)) {
166         aplicacao.ligarTodas(Formulario.parametro(lerCorpo(troca), "ligar").equals("sim"));
167     }
168     redirecionar(troca, "/coleta");
169 }
170
171 private void tratarDiario(HttpExchange troca) throws IOException {
172     String arquivo = Formulario.parametro(troca.getRequestURI().getRawQuery(), "arquivo");
173     responder(troca, 200, new PaginaDiario(aplicacao).diario(arquivo));
174 }
175
176 private void tratarAto(HttpExchange troca) throws IOException {

```

```
177     String consulta = troca.getRequestURI().getRawQuery();
178     String arquivo = Formulario.parametro(consulta, "arquivo");
179     int indice = Formulario.inteiro(consulta, "indice", -1);
180     responder(troca, 200, new PaginaDiario(aplicacao).ato(arquivo, indice));
181 }
182
183 private void tratarBusca(HttpExchange troca) throws IOException {
184     String termo = Formulario.parametro(troca.getRequestURI().getRawQuery(), "q");
185     responder(troca, 200, new PaginaDiario(aplicacao).busca(termo));
186 }
187
188 private boolean ehPost(HttpExchange troca) {
189     return troca.getRequestMethod().equalsIgnoreCase("POST");
190 }
191
192 private String lerCorpo(HttpExchange troca) throws IOException {
193     return new String(troca.getRequestBody().readAllBytes(), StandardCharsets.UTF_8);
194 }
195
196 private void redirecionar(HttpExchange troca, String destino) throws IOException {
197     troca.getResponseHeaders().set("Location", destino);
198     troca.sendResponseHeaders(303, -1);
199     troca.close();
200 }
201
202 private void responder(HttpExchange troca, int status, String html) throws IOException {
203     byte[] bytes = html.getBytes(StandardCharsets.UTF_8);
204     troca.getResponseHeaders().set("Content-Type", "text/html; charset=utf-8");
205     troca.getResponseHeaders().set("Cache-Control", "no-store");
206     troca.sendResponseHeaders(status, bytes.length);
207     try (OutputStream saida = troca.getResponseBody()) {
208         saida.write(bytes);
209     }
210 }
211
212 private static void abrirNoNavegador(String url) {
213     try {
214         if (Desktop.isDesktopSupported()
215             && Desktop.getDesktop().isSupported(Desktop.Action.BROWSE)) {
216             Desktop.getDesktop().browse(URI.create(url));
217             return;
218         }
219     } catch (Exception e) {
220         System.out.println("nao consegui abrir o navegador sozinho: " + e.getMessage());
221     }
222     System.out.println("abra no navegador: " + url);
223 }
224 }
```

ARQUIVO: diarios/servidor/Formulario.java

pacote diarios.servidor · 48 linhas · Lê parâmetros de formulário/URL (split, URLDecoder).

```
1 package diarios.servidor;
2
3 import java.net.URLDecoder;
4 import java.net.URLEncoder;
5 import java.nio.charset.StandardCharsets;
6
7 public class Formulario {
8
9     public static String parametro(String dados, String chave) {
10         if (dados == null || dados.isEmpty()) {
11             return "";
12         }
13         for (String par : dados.split("&")) {
14             int igual = par.indexOf('=');
15             if (igual < 0) {
16                 continue;
17             }
18             if (decodificar(par.substring(0, igual)).equals(chave)) {
19                 return decodificar(par.substring(igual + 1)).trim();
20             }
21         }
22         return "";
23     }
24
25     public static int inteiro(String dados, String chave, int padrao) {
26         String valor = parametro(dados, chave);
27         if (valor.isEmpty()) {
28             return padrao;
29         }
30         try {
31             return Integer.parseInt(valor);
32         } catch (NumberFormatException e) {
33             return padrao;
34         }
35     }
36
37     public static String codificar(String s) {
38         return URLEncoder.encode(s == null ? "" : s, StandardCharsets.UTF_8);
39     }
40
41     private static String decodificar(String s) {
42         try {
43             return URLDecoder.decode(s, StandardCharsets.UTF_8);
44         } catch (IllegalArgumentException e) {
45             return s;
46         }
47     }
48 }
```

ARQUIVO: diarios/servidor/PainelWeb.java

pacote diarios.servidor · 421 linhas · Páginas "Painel" e "Coleta e fontes" (HTML montado em ArrayList<String>).

```

1 package diarios.servidor;
2
3 import diarios.Aplicacao;
4 import diarios.analise.Cobertura;
5 import diarios.analise.PainelContratos;
6 import diarios.coleta.EstadoFonte;
7 import diarios.coleta.Municipio;
8 import diarios.coleta.Rodada;
9 import diarios.etapas.PaginaHtml;
10 import diarios.modelo.Ato;
11 import diarios.modelo.Contrato;
12 import diarios.modelo.Diario;
13 import java.util.ArrayList;
14
15 public class PainelWeb {
16
17     private static final int SEGUNDOS_ENTRE_ATUALIZACOES = 3;
18
19     private final Aplicacao app;
20     private final PaginaHtml pagina = new PaginaHtml();
21
22     public PainelWeb(Aplicacao app) {
23         this.app = app;
24     }
25
26     public String montar(String aviso) {
27         ArrayList<Diario> diarios = app.getDiarios();
28         Cobertura cobertura = app.getCobertura();
29         PainelContratos painel = app.getPainel();
30
31         ArrayList<String> h = abrir("Painel");
32         menu(h, "painel");
33         pagina.cabecalho(h, diarios, cobertura, painel, app.getGrafo());
34         barraDeEstado(h, aviso);
35
36         if (diarios.isEmpty()) {
37             h.add("<section><h2>Nenhum diario lido ainda</h2><p class=\"nota\">Va em "
38                 + "<a href=\"/coleta\">Coleta e fontes</a>, ligue algumas fontes e clique em "
39                 + "Coletar agora.</p></section>");
40         } else {
41             pagina.secaoInterpretacao(h, app.getInterpretacao());
42             secaoPorMunicipio(h, diarios, painel);
43             pagina.secaoFornecedores(h, painel);
44             pagina.secaoGrafo(h, app.getGrafo());
45             secaoDiarios(h, diarios);
46             h.add("<h2 class=\"divisor\">Como o programa chegou nesses numeros</h2>");
47             pagina.secaoFunil(h, diarios, cobertura);
48             pagina.secaoPortao(h, cobertura);
49             h.add("<details class=\"log\"><summary>Ver a lista completa de contratacoes ("
50                 + painel.getContratos().size() + ")</summary>");
51             pagina.secaoTabela(h, painel);
52             h.add("</details>");
53         }
54         return fechar(h);
55     }
56
57     public String montarColeta(String aviso) {
58         ArrayList<String> h = abrir("Coleta e fontes");
59         menu(h, "coleta");
60         h.add("<header><h1>Coleta e fontes</h1><p class=\"sub\">De onde vem o dado, e o que "
61             + "aconteceu cada vez que o programa foi busca-lo.</p></header>");
62         barraDeEstado(h, aviso);
63         secaoColeta(h);
64         secaoFontes(h);
65         return fechar(h);
66     }
67
68     private ArrayList<String> abrir(String titulo) {
69         ArrayList<String> h = new ArrayList<>();
70         h.add("<!DOCTYPE html><html lang=\"pt-BR\"><head><meta charset=\"UTF-8\">");
71         h.add("<meta name=\"viewport\" content=\"width=device-width, initial-scale=1\">");
72         if (app.isOcupado()) {
73             h.add("<meta http-equiv=\"refresh\" content=\"" + SEGUNDOS_ENTRE_ATUALIZACOES + "\">");
74         }
75         h.add("<title>" + esc(titulo) + " - Diarios Oficiais</title>");
76         h.add("<style>" + pagina.estilo() + estiloDoPainel() + "</style>");
77         h.add("</head><body><div class=\"viz-root\">");
78         return h;
79     }
80
81     private String fechar(ArrayList<String> h) {
82         rodape(h);
83         h.add("</div></body></html>");
84         return String.join("\n", h);
85     }
86

```

```

87 static void menu(ArrayList<String> h, String ativo) {
88     h.add("<nav class=\"menu\">");
89     h.add("<a href=\"/\" + (ativo.equals(\"painel\") ? \" class=\"ativo\" : \"\") + \">Painel</a>");
90     h.add("<a href=\"/coleta\" + (ativo.equals(\"coleta\") ? \" class=\"ativo\" : \"\")");
91         + ">Coleta e fontes</a>");
92     h.add("<form method=\"get\" action=\"/busca\"><input type=\"text\" name=\"q\"");
93         + "placeholder=\"buscar nos diarios...\"></form>");
94     h.add("</nav>");
95 }
96
97 private void barraDeEstado(ArrayList<String> h, String aviso) {
98     if (app.isOcupado()) {
99         h.add("<div class=\"estado estado-ocupado\"><span class=\"girando\">&#9696;</span>");
100             + esc(app.getProgresso()) + " &mdash; a pagina se atualiza sozinha.</div>");
101     } else if (aviso.equals("ocupado")) {
102         h.add("<div class=\"estado estado-aviso\">Ja existe uma coleta em andamento.");
103             + "Espere ela terminar.</div>");
104     } else {
105         h.add("<div class=\"estado\">Ultima leitura da pasta:");
106             + esc(app.getUltimoProcessamento()) + "</div>");
107     }
108 }
109
110 private void secaoColeta(ArrayList<String> h) {
111     String desabilitar = app.isOcupado() ? " disabled" : "";
112     h.add("<section id=\"coleta\">");
113     h.add("<h2>Coleta</h2>");
114     h.add("<p class=\"nota\">Baixa, das fontes ligadas, os diarios publicados nos ultimos N");
115         + "dias. Fontes ja coletadas para o periodo (com dado, sem edicao ou so PDF)");
116         + "sao puladas; so coleta as que falharam ou nunca foram tentadas.");
117         + "Marque &quot;refazer todas&quot; para forcar. Diario ja baixado nunca e baixado de novo.</p>");
118     h.add("<div class=\"acoes\">");
119     h.add("<form method=\"post\" action=\"/coletar\">");
120     h.add("<label>dias para tras <input type=\"number\" name=\"dias\" value=\"1\" min=\"1\" max=\"\"");
121         + "Aplicacao.DIAS_MAXIMO + \"\"></label>");
122     h.add("<label title=\"Sem marcar, pula as fontes ja coletadas para esse periodo");
123         + "(com dado, sem edicao ou so PDF) e coleta so as que falharam ou nunca foram");
124         + "tentadas\"><input type=\"checkbox\" name=\"refazer\" value=\"sim\"> refazer todas</label>");
125     h.add("<button type=\"submit\" class=\"primario\" + desabilitar + ">Coletar agora</button>");
126     h.add("</form>");
127     h.add("<form method=\"post\" action=\"/descobrir\"><button type=\"submit\" + desabilitar");
128         + " title=\"Pergunta a API quais municipios publicaram texto nos ultimos 3 dias\">");
129         + "Descobrir municipios com publicacao recente</button></form>");
130     h.add("<form method=\"post\" action=\"/processar\"><button type=\"submit\" + desabilitar");
131         + ">Reler a pasta</button></form>");
132     h.add("</div>");
133
134     Rodada ultima = app.getUltimaRodada();
135     if (ultima != null && !ultima.registro().isEmpty()) {
136         h.add("<details class=\"log\"><summary>O que aconteceu na ultima rodada ("");
137             + ultima.registro().size() + " linhas)</summary><pre>");
138         for (String linha : ultima.registro()) {
139             h.add(esc(linha));
140         }
141         h.add("</pre></details>");
142     }
143
144     ArrayList<Rodada> rodadas = app.rodadasRecentes();
145     if (!rodadas.isEmpty()) {
146         h.add("<h3>Historico de rodadas</h3>");
147         h.add("<table><thead><tr><th>inicio</th><th>tipo</th><th>periodo</th>");
148             + "<th>num</th><th>fontes</th><th>num</th><th>puladas</th>");
149             + "<th>num</th><th>baixados</th>");
150             + "<th>num</th><th>ja tinha</th><th>num</th><th>falhas</th>");
151             + "<th>observacao</th></tr></thead><tbody>");
152         for (Rodada r : rodadas) {
153             h.add("<tr><td class=\"mono\"> + esc(r.getInicio()) + "</td>");
154                 + "<td> + esc(r.getTipo()) + "</td>");
155                 + "<td class=\"mono\"> + esc(r.getDesde()) + " a " + esc(r.getAte()) + "</td>");
156                 + "<td class=\"num\"> + r.fontes() + "</td>");
157                 + "<td class=\"num\"> + r.puladas() + "</td>");
158                 + "<td class=\"num\"> + r.baixados() + "</td>");
159                 + "<td class=\"num\"> + r.jaExistiam() + "</td>");
160                 + "<td class=\"num\" + (r.falhas() > 0 ? \" ruim\" : \"\") + \"> + r.falhas() + "</td>");
161                 + "<td> + esc(r.getObservacao()) + "</td></tr>");
162         }
163         h.add("</tbody></table>");
164     }
165     h.add("</section>");
166 }
167
168 private void secaoFontes(ArrayList<String> h) {
169     ArrayList<EstadoFonte> todas = app.getFontes().todas();
170     int ligadas = 0;
171     int comDado = 0;
172     for (EstadoFonte e : todas) {
173         if (e.isLigada()) {
174             ligadas++;
175         }
176         if (e.getStatus().equals(EstadoFonte.COM_DADO)) {

```

```

177         comDado++;
178     }
179 }
180
181 h.add("<section id=\"fontes\">");
182 h.add("<h2>Fontes</h2>");
183 h.add("<p class=\"nota\">" + ligadas + " de " + todas.size() + " ligadas &middot; "
184     + comDado + " com conteudo confirmado. So as ligadas entram na proxima coleta. "
185     + "Clique no botao da linha para ligar ou desligar. O estado fica salvo em "
186     + "<span class=\"mono\">dados/estado/fontes.txt</span>.</p>");
187 h.add("<div class=\"acoes\">");
188 h.add("<form method=\"post\" action=\"/fonte/todas\"><input type=\"hidden\" name=\"ligar\" "
189     + "value=\"sim\"><button type=\"submit\">ligar todas</button></form>");
190 h.add("<form method=\"post\" action=\"/fonte/todas\"><input type=\"hidden\" name=\"ligar\" "
191     + "value=\"nao\"><button type=\"submit\">desligar todas</button></form>");
192 h.add("</div>");
193
194 h.add("<div class=\"legenda\">");
195 legenda(h, "ok", "COM_DADO", "baixou ou ja tinha diario com texto");
196 legenda(h, "duv", "SEM_EDICAO / SO_PDF", "respondeu, mas sem texto no periodo");
197 legenda(h, "rej", "FALHOU", "a rede ou a API nao respondeu");
198 legenda(h, "neutro", "NUNCA_TENTADA", "ainda nao foi coletada");
199 h.add("</div>");
200
201 ArrayList<EstadoFonte> ligadasLista = new ArrayList<>();
202 ArrayList<EstadoFonte> desligadas = new ArrayList<>();
203 for (EstadoFonte e : todas) {
204     if (e.isLigada()) {
205         ligadasLista.add(e);
206     } else {
207         desligadas.add(e);
208     }
209 }
210 h.add("<h3>Ligadas (" + ligadasLista.size() + ")</h3>");
211 tabelaDeFontes(h, ligadasLista);
212 h.add("<details class=\"log\"><summary>Desligadas (" + desligadas.size()
213     + ") &mdash; clique para ver e ligar</summary>");
214 tabelaDeFontes(h, desligadas);
215 h.add("</details>");
216 h.add("</section>");
217 }
218
219 private void tabelaDeFontes(ArrayList<String> h, ArrayList<EstadoFonte> lista) {
220     if (lista.isEmpty()) {
221         h.add("<p class=\"nota\">nenhuma</p>");
222         return;
223     }
224     h.add("<table class=\"fontes\"><thead><tr><th></th><th>municipio</th><th>uf</th>"
225         + "<th>status</th><th>ultima tentativa</th><th>motivo</th>"
226         + "<th class=\"num\">no acervo</th></tr></thead><tbody>");
227     for (EstadoFonte e : lista) {
228         Municipio m = e.getMunicipio();
229         h.add("<tr class=\"\" + (e.isLigada() ? \"ligada\" : \"desligada\") + \">");
230         h.add("<td><form method=\"post\" action=\"/fonte/alternar\">"
231             + "<input type=\"hidden\" name=\"codigo\" value=\"\" + esc(m.getCodigo()) + \">"
232             + "<button type=\"submit\" class=\"chave \" + (e.isLigada() ? \"on\" : \"off\") + \">"
233             + (e.isLigada() ? "LIGADA" : "ligar") + "</button></form></td>");
234         h.add("<td>" + esc(m.getNome())
235             + (e.isDescoberta() ? " <span class=\"tag\">descoberta</span>" : "") + "</td>");
236         h.add("<td>" + esc(m.getUf()) + "</td>");
237         h.add("<td><span class=\"tag tag-\" + classeDoStatus(e.getStatus()) + \">"
238             + esc(e.getStatus()) + "</span></td>");
239         h.add("<td class=\"mono\">" + esc(e.getUltimaTentativa()) + "</td>");
240         h.add("<td class=\"motivo\">" + esc(PaginaHtml.cortar(e.getMotivo(), 90)) + "</td>");
241         h.add("<td class=\"num\">" + e.getDiariosNoAcervo() + "</td>");
242         h.add("</tr>");
243     }
244     h.add("</tbody></table>");
245 }
246
247 private void legenda(ArrayList<String> h, String classe, String nome, String explicacao) {
248     h.add("<span><span class=\"tag tag-\" + classe + \">" + nome + "</span>"
249         + esc(explicacao) + "</span>");
250 }
251
252 private void secaoPorMunicipio(ArrayList<String> h, ArrayList<Diario> diarios,
253     PainelContratos painel) {
254     ArrayList<String> municipios = new ArrayList<>();
255     for (Diario d : diarios) {
256         if (!municipios.contains(d.getMunicipio())) {
257             municipios.add(d.getMunicipio());
258         }
259     }
260
261     h.add("<section>");
262     h.add("<h2>Por municipio</h2>");
263     h.add("<p class=\"nota\">0s mesmos numeros do topo, separados por municipio. "
264         + "E aqui que as fontes podem ser comparadas entre si.</p>");
265     h.add("<table><thead><tr><th>municipio</th><th class=\"num\">diarios</th>"
266         + "<th class=\"num\">recusados</th><th class=\"num\">atos</th>");

```

```

267         + "<th class=\"num\">aceitos</th><th class=\"num\">contratacoes</th>"
268         + "<th class=\"num\">valor contratado</th></tr></thead><tbody>");
269     for (String m : municipios) {
270         int qtd = 0;
271         int recusados = 0;
272         int atos = 0;
273         int aceitos = 0;
274         for (Diario d : diarios) {
275             if (!d.getMunicipio().equals(m)) {
276                 continue;
277             }
278             qtd++;
279             if (!d.isUtilizavel()) {
280                 recusados++;
281             }
282             atos += d.getAtos().size();
283             aceitos += d.contar(Ato.ACEITO);
284         }
285         int contratos = 0;
286         double valor = 0;
287         for (Contrato c : painel.getContratos()) {
288             if (c.getMunicipio().equals(m)) {
289                 contratos++;
290                 valor += c.getValor();
291             }
292         }
293         h.add("<tr><td>" + esc(bonito(m)) + "</td><td class=\"num\">" + qtd + "</td>"
294             + "<td class=\"num\" + (recusados > 0 ? \" ruim\" : \"\") + "\">" + recusados + "</td>"
295             + "<td class=\"num\">" + atos + "</td><td class=\"num\">" + aceitos + "</td>"
296             + "<td class=\"num\">" + contratos + "</td>"
297             + "<td class=\"num\">" + Contrato.formatarReal(valor) + "</td></tr>");
298     }
299     h.add("</tbody></table>");
300     h.add("</section>");
301 }
302
303 private void secaoDiarios(ArrayList<String> h, ArrayList<Diario> diarios) {
304     h.add("<section>");
305     h.add("<h2>Diarios lidos</h2>");
306     h.add("<p class=\"nota\">Clique para abrir o diario: o texto como veio, o texto limpo, "
307         + "cada ato separado e o que o programa leu em cada um.</p>");
308     h.add("<table><thead><tr><th>arquivo</th><th>situacao</th><th class=\"num\">atos</th>"
309         + "<th class=\"num\">aceitos</th><th class=\"num\">duvidosos</th>"
310         + "<th class=\"num\">lixo removido</th></tr></thead><tbody>");
311     for (Diario d : diarios) {
312         String link = "<a href=\"/diario?arquivo=" + Formulario.codificar(d.getArquivo())
313             + "\">" + esc(d.getArquivo()) + "</a>";
314         if (!d.isUtilizavel()) {
315             h.add("<tr><td>" + link + "</td><td><span class=\"tag tag-rej\">RECUSADO</span> "
316                 + esc(d.getMotivoRecusa()) + "</td><td class=\"num\">-</td>"
317                 + "<td class=\"num\">-</td><td class=\"num\">-</td><td class=\"num\">-</td></tr>");
318             continue;
319         }
320         h.add("<tr><td>" + link + "</td><td><span class=\"tag tag-ok\">PROCESSADO</span></td>"
321             + "<td class=\"num\">" + d.getAtos().size() + "</td>"
322             + "<td class=\"num\">" + d.contar(Ato.ACEITO) + "</td>"
323             + "<td class=\"num\">" + d.contar(Ato.DUVIDOSO) + "</td>"
324             + "<td class=\"num\">" + d.getLinhasRemovidas() + "</td></tr>");
325     }
326     h.add("</tbody></table>");
327     h.add("</section>");
328 }
329
330 private void rodape(ArrayList<String> h) {
331     h.add("<footer>");
332     h.add("<p>Pagina montada a cada requisicao pelo proprio programa Java "
333         + "<span class=\"mono\">com.sun.net.httpserver</span>, que ja vem no JDK). "
334         + "Sem biblioteca externa, sem JavaScript.</p>");
335     h.add("<p>Deterministico: a mesma pasta de diarios produz sempre os mesmos numeros "
336         + "e as mesmas frases. Nenhuma interpretacao usa IA.</p>");
337     h.add("<p>CPFs publicados nos diarios aparecem mascarados. Dados: Querido Diario, "
338         + "Open Knowledge Brasil.</p>");
339     h.add("</footer>");
340 }
341
342 static String classeDoStatus(String status) {
343     if (status.equals(EstadoFonte.COM_DADO)) {
344         return "ok";
345     }
346     if (status.equals(EstadoFonte.FALHOU)) {
347         return "rej";
348     }
349     if (status.equals(EstadoFonte.SEM_EDICAO) || status.equals(EstadoFonte.SO_PDF)) {
350         return "duv";
351     }
352     return "neutro";
353 }
354
355 static String bonito(String apelido) {
356     StringBuilder nome = new StringBuilder();

```

```

357     for (String palavra : apelido.split("-")) {
358         if (palavra.isEmpty()) {
359             continue;
360         }
361         nome.append(nome.length() > 0 ? " " : "")
362             .append(Character.toUpperCase(palavra.charAt(0))).append(palavra.substring(1));
363     }
364     return nome.toString();
365 }
366
367 private static String esc(String s) {
368     return PaginaHtml.escapar(s);
369 }
370
371 static String estiloDoPainel() {
372     return String.join("\n",
373         ".estado{font-size:13px;color:var(--ink2);padding:10px 14px;border-radius:8px;",
374         "    border:1px solid var(--grade);background:var(--surface);margin-bottom:18px}",
375         ".estado-ocupado{border-left:3px solid var(--serie);color:var(--ink)}",
376         ".estado-aviso{border-left:3px solid var(--duv)}",
377         ".girando{display:inline-block;animation:gira 1s linear infinite}",
378         "@keyframes gira{to{transform:rotate(360deg)}}",
379         ".acoes{display:flex;flex-wrap:wrap;gap:10px;align-items:center;margin-bottom:12px}",
380         ".acoes form{display:flex;gap:8px;align-items:center;margin:0}",
381         "button{font:inherit;font-size:13px;padding:6px 12px;border-radius:6px;cursor:pointer;",
382         "    border:1px solid var(--eixo);background:var(--surface);color:var(--ink)}",
383         "button:hover{border-color:var(--ink3)}",
384         "button:disabled{opacity:.45;cursor:not-allowed}",
385         "button.primario{background:var(--serie);border-color:var(--serie);color:#fff}",
386         "input[type=number]{width:64px}",
387         "input{font:inherit;font-size:13px;padding:5px 8px;border-radius:6px;",
388         "    border:1px solid var(--eixo);background:var(--plano);color:var(--ink)}",
389         ".campo{flex:1;min-width:220px}",
390         ".label{font-size:13px;color:var(--ink2);display:flex;gap:6px;align-items:center}",
391         ".chave{min-width:86px;font-size:11px;letter-spacing:.05em;padding:3px 8px}",
392         ".chave.on{background:var(--ok);border-color:var(--ok);color:#fff}",
393         ".chave.off{color:var(--ink3)}",
394         "tr.desligada td{color:var(--ink3)}",
395         ".fontes td{padding:4px 8px}",
396         ".tag-ok{border-color:var(--ok);color:var(--ok)}",
397         ".tag-duv{border-color:var(--duv);color:var(--ink2)}",
398         ".tag-rej{border-color:var(--rej);color:var(--rej)}",
399         ".tag-neutro{color:var(--ink3)}",
400         ".legenda{display:flex;flex-wrap:wrap;gap:14px;font-size:12px;color:var(--ink3);",
401         "    margin-bottom:8px}",
402         "td.ruim{color:var(--rej);font-weight:600}",
403         ".log{margin:6px 0 4px}",
404         ".log summary{cursor:pointer;font-size:13px;color:var(--ink2)}",
405         ".log pre{font-size:11.5px;color:var(--ink2);white-space:pre-wrap;max-height:260px;",
406         "    overflow:auto;background:var(--plano);padding:10px;border-radius:6px}",
407         "a{color:var(--serie)}",
408         ".menu{display:flex;gap:6px;align-items:center;margin:-8px 0 22px;padding:8px;",
409         "    border:1px solid var(--grade);border-radius:10px;background:var(--surface);",
410         "    position:sticky;top:8px;z-index:10}",
411         ".menu a{padding:6px 12px;border-radius:6px;text-decoration:none;color:var(--ink2);",
412         "    font-size:14px}",
413         ".menu a.ativo{background:var(--serie);color:#fff}",
414         ".menu form{margin-left:auto}",
415         ".menu input{width:230px}",
416         ".divisor{margin:34px 0 12px;font-size:15px;color:var(--ink3);text-transform:uppercase;",
417         "    letter-spacing:.06em}",
418         "td.motivo{font-size:11.5px;color:var(--ink3);max-width:320px}",
419         "details.log>section{margin-top:10px}");
420 }
421 }

```

ARQUIVO: diarios/servidor/PaginaDiario.java

pacote diarios.servidor · 281 linhas · Páginas de um diário, de um ato (com prova de cada fato) e a busca.

```

1 package diarios.servidor;
2
3 import diarios.Aplicacao;
4 import diarios.analise.Assunto;
5 import diarios.analise.LeituraDoAto;
6 import diarios.etapas.PaginaHtml;
7 import diarios.etapas.Privacidade;
8 import diarios.extratores.Extrator;
9 import diarios.modelo.Ato;
10 import diarios.modelo.Diario;
11 import diarios.modelo.Fato;
12 import java.util.ArrayList;
13
14 public class PaginaDiario {
15
16     private static final int LIMITE_TEXTO_BRUTO = 60000;
17     private static final int LIMITE_RESULTADOS = 200;
18     private static final int MARGEM_DA_PROVA = 70;
19
20     private final Aplicacao app;
21
22     public PaginaDiario(Aplicacao app) {
23         this.app = app;
24     }
25
26     public String diario(String arquivo) {
27         Diario d = app.diarioDoArquivo(arquivo);
28         ArrayList<String> h = abrir(arquivo);
29         h.add("<a class=\"voltar\" href=\"/\">&larr; painel</a>");
30         if (d == null) {
31             h.add("<p>Esse arquivo nao esta entre os diarios lidos.</p>");
32             return fechar(h);
33         }
34
35         h.add("<h1>" + esc(arquivo) + "</h1>");
36         if (!d.isUtilizavel()) {
37             h.add("<div class=\"estado estado-aviso\">RECUSADO: " + esc(d.getMotivoRecusa())
38                 + "</div>");
39             h.add("<p class=\"nota\">0 texto abaixo e exatamente o que veio da fonte. Repare que "
40                 + "quase so tem cabecalho e rodape de pagina - o conteudo esta em imagem.</p>");
41             h.add("<pre class=\"texto\">" + esc(Privacidade.mascararCpf(
42                 recortar(d.getTextoBruto(), LIMITE_TEXTO_BRUTO))) + "</pre>");
43             return fechar(h);
44         }
45
46         h.add("<p class=\"nota\">" + d.getAto().size() + " atos &mdot; "
47             + d.getTitulosCandidatos() + " titulos candidatos, " + d.getTitulosDescartados()
48             + " descartados sem corpo &mdot; " + d.getLinhasRemovidas()
49             + " linhas de lixo removidas de " + d.getLinhasOriginais() + "</p>");
50
51         LeituraDoAto leitor = app.leitor();
52         h.add("<h2>Ato e o que cada um faz</h2>");
53         h.add("<table><thead><tr><th>#</th><th>decisao</th><th>assunto</th><th>titulo</th>"
54             + "<th>leitura</th></tr></thead><tbody>");
55         for (int i = 0; i < d.getAto().size(); i++) {
56             Ato a = d.getAto().get(i);
57             h.add("<tr><td class=\"mono\">" + i + "</td>"
58                 + "<td><span class=\"tag tag-\" + classeDaDecisao(a.getDecisao()) + \">"
59                 + a.getDecisao() + "</span></td>"
60                 + "<td class=\"mono\">" + Assunto.de(a) + "</td>"
61                 + "<td><a href=\"" + linkDoAto(arquivo, i) + "\">"
62                 + esc(PaginaHtml.cortar(a.getTitulo(), 60)) + "</a></td>"
63                 + "<td class=\"leitura-curta\">" + esc(leitor.ler(a, d.getData())) + "</td></tr>");
64         }
65         h.add("</tbody></table>");
66
67         h.add("<details class=\"log\"><summary>Texto como veio da fonte (bruto)</summary>");
68         h.add("<pre class=\"texto\">" + esc(Privacidade.mascararCpf(
69             recortar(d.getTextoBruto(), LIMITE_TEXTO_BRUTO))) + "</pre></details>");
70         h.add("<details class=\"log\"><summary>Texto depois da limpeza</summary>");
71         h.add("<pre class=\"texto\">" + esc(Privacidade.mascararCpf(
72             recortar(d.getTextoLimpo(), LIMITE_TEXTO_BRUTO))) + "</pre></details>");
73         return fechar(h);
74     }
75
76     public String ato(String arquivo, int indice) {
77         Diario d = app.diarioDoArquivo(arquivo);
78         ArrayList<String> h = abrir("ato " + indice);
79         h.add("<a class=\"voltar\" href=\"/diario?arquivo=" + Formulario.codificar(arquivo)
80             + "\">&larr; " + esc(arquivo) + "</a>");
81         if (d == null || indice < 0 || indice >= d.getAto().size()) {
82             h.add("<p>Ato nao encontrado.</p>");
83             return fechar(h);
84         }
85
86         Ato ato = d.getAto().get(indice);

```

```

87     h.add("<h1>" + esc(Privacidade.mascararCpf(ato.getTitulo())) + "</h1>");
88     h.add("<div class=\"leitura-ato\"><div class=\"rotulo-leitura\">0 que este ato faz</div>"
89         + esc(app.leitor().ler(ato, d.getData())) + "</div>");
90
91     h.add("<table class=\"ficha\"><tbody>");
92     linha(h, "identidade", "<span class=\"mono\">" + esc(ato.getIdentidade()) + "</span>");
93     linha(h, "assunto", Assunto.de(ato));
94     linha(h, "decisao", "<span class=\"tag tag-\" + classeDaDecisao(ato.getDecisao()) + \"\">"
95         + ato.getDecisao() + "</span>" + esc(ato.getMotivo()));
96     linha(h, "posicao no diario", "caracteres " + ato.getPosicao() + " a "
97         + (ato.getPosicao() + ato.getTexto().length()));
98     h.add("</tbody></table>");
99
100    h.add("<h2>Fatos e a prova de cada um</h2>");
101    h.add("<p class=\"nota\">Cada fato guarda a posicao exata onde foi achado. O trecho "
102        + "abaixo e recortado do texto original a partir dessa posicao - e a prova de "
103        + "que o programa nao inventou nada.</p>");
104    if (ato.getFatos().isEmpty()) {
105        h.add("<p class=\"nota\">Nenhum fato extraido deste ato.</p>");
106    }
107    for (Fato f : ato.getFatos()) {
108        h.add("<div class=\"fato\"><span class=\"tag\">" + f.getTipo() + "</span>"
109            + "<strong>" + esc(Privacidade.mascararCpf(f.getValor())) + "</strong>"
110            + "<span class=\"mono pos\">[" + f.getInicio() + ".." + f.getFim() + "]"</span>"
111            + "<div class=\"prova\">&hellip;" + esc(Privacidade.mascararCpf(
112                f.provar(d.getTextoLimpo(), MARGEM_DA_PROVA)) + "&hellip;</div></div>");
113    }
114
115    h.add("<h2>Texto do ato, com os fatos marcados</h2>");
116    h.add("<pre class=\"texto\">" + destacar(ato) + "</pre>");
117    return fechar(h);
118 }
119
120 public String busca(String termo) {
121     ArrayList<String> h = abrir("busca");
122     h.add("<a class=\"voltar\" href=\"\"/>&larr; painel</a>");
123     h.add("<h1>Busca</h1>");
124     h.add("<form method=\"get\" action=\"\"/busca\" class=\"acoes\">"
125         + "<input type=\"text\" name=\"q\" class=\"campo\" value=\"\" + esc(termo) + \"\">"
126         + "<button type=\"submit\">Buscar</button></form>");
127     if (termo.length() < 2) {
128         h.add("<p class=\"nota\">Digite ao menos 2 letras.</p>");
129         return fechar(h);
130     }
131
132     String alvo = Extrator.semAcento(termo.toLowerCase());
133     int achados = 0;
134     ArrayList<String> linhas = new ArrayList<>();
135
136     for (Diario d : app.getDiarios()) {
137         if (!d.isUtilizavel()) {
138             continue;
139         }
140         String texto = d.getTextoLimpo();
141         String base = Extrator.semAcento(texto.toLowerCase());
142         int p = base.indexOf(alvo);
143         while (p >= 0 && achados < LIMITE_RESULTADOS) {
144             achados++;
145             int ato = atoNaPosicao(d, p);
146             String trecho = Privacidade.mascararCpf(
147                 texto.substring(Math.max(0, p - 80), Math.min(texto.length(), p + alvo.length() + 80)))
148                 .replace('\n', ' ');
149             String destino = ato >= 0 ? linkDoAto(d.getArquivo(), ato)
150                 : "/diario?arquivo=" + Formulario.codificar(d.getArquivo());
151             linhas.add("<div class=\"fato\"><a href=\"\" + destino + \"\">" + esc(d.getArquivo())
152                 + (ato >= 0 ? " &middot;" : " ") + ato : " &middot;" + fora de ato) + "</a>"
153                 + "<div class=\"prova\">&hellip;" + esc(trecho) + "&hellip;</div></div>");
154             p = base.indexOf(alvo, p + alvo.length());
155         }
156     }
157
158     h.add("<p class=\"nota\">" + achados + (achados >= LIMITE_RESULTADOS ? "+" : "")
159         + " ocorrencia(s) de <strong>" + esc(termo) + "</strong>.</p>");
160     h.addAll(linhas);
161     return fechar(h);
162 }
163
164 public static String paginaSimples(String titulo, String mensagem) {
165     ArrayList<String> h = abrir(titulo);
166     h.add("<a class=\"voltar\" href=\"\"/>&larr; painel</a>");
167     h.add("<h1>" + esc(titulo) + "</h1><p>" + esc(mensagem) + "</p>");
168     return fechar(h);
169 }
170
171 private String destacar(Ato ato) {
172     String texto = ato.getTexto();
173     ArrayList<int[]> trechos = new ArrayList<>();
174     for (Fato f : ato.getFatos()) {
175         trechos.add(new int[] {f.getInicio() - ato.getPosicao(), f.getFim() - ato.getPosicao()});
176     }

```

```

177         ordenar(trechos);
178
179         StringBuilder s = new StringBuilder();
180         int cursor = 0;
181         for (int[] t : trechos) {
182             int ini = Math.max(0, Math.min(t[0], texto.length()));
183             int fim = Math.max(ini, Math.min(t[1], texto.length()));
184             if (ini < cursor) {
185                 continue;
186             }
187             s.append(esc(Privacidade.mascararCpf(texto.substring(cursor, ini))));
188             s.append("<mark>").append(esc(Privacidade.mascararCpf(texto.substring(ini, fim))))
189                 .append("</mark>");
190             cursor = fim;
191         }
192         s.append(esc(Privacidade.mascararCpf(texto.substring(cursor))));
193         return s.toString();
194     }
195
196     private void ordenar(ArrayList<int[]> trechos) {
197         for (int i = 1; i < trechos.size(); i++) {
198             int[] atual = trechos.get(i);
199             int j = i - 1;
200             while (j >= 0 && trechos.get(j)[0] > atual[0]) {
201                 trechos.set(j + 1, trechos.get(j));
202                 j--;
203             }
204             trechos.set(j + 1, atual);
205         }
206     }
207
208     private int atoNaPosicao(Diario d, int posicao) {
209         for (int i = 0; i < d.getAto().size(); i++) {
210             Ato a = d.getAto().get(i);
211             if (posicao >= a.getPosicao() && posicao < a.getPosicao() + a.getTexto().length()) {
212                 return i;
213             }
214         }
215         return -1;
216     }
217
218     private static String linkDoAto(String arquivo, int indice) {
219         return "/ato?arquivo=" + Formulario.codificar(arquivo) + "&indice=" + indice;
220     }
221
222     private void linha(ArrayList<String> h, String rotulo, String valor) {
223         h.add("<tr><th>" + esc(rotulo) + "</th><td>" + valor + "</td></tr>");
224     }
225
226     private String classeDaDecisao(String decisao) {
227         if (decisao.equals(Ato.ACEITO)) {
228             return "ok";
229         }
230         return decisao.equals(Ato.REJEITADO) ? "rej" : "duv";
231     }
232
233     private String recortar(String s, int max) {
234         return s.length() <= max ? s : s.substring(0, max) + "\n\n[... cortado para exibicao ...]";
235     }
236
237     private static ArrayList<String> abrir(String titulo) {
238         ArrayList<String> h = new ArrayList<>();
239         h.add("<!DOCTYPE html><html lang=\"pt-BR\"><head><meta charset=\"UTF-8\">");
240         h.add("<meta name=\"viewport\" content=\"width=device-width, initial-scale=1\">");
241         h.add("<title>" + esc(titulo) + " - Diarios Oficiais</title>");
242         h.add("<style>" + new PaginaHtml().estilo() + PaineWeb.estiloDoPaine() + estiloExtra()
243             + "</style>");
244         h.add("</head><body><div class=\"viz-root\">");
245         PaineWeb.menu(h, "");
246         return h;
247     }
248
249     private static String fechar(ArrayList<String> h) {
250         h.add("</div></body></html>");
251         return String.join("\n", h);
252     }
253
254     private static String esc(String s) {
255         return PaginaHtml.escapar(s);
256     }
257
258     private static String estiloExtra() {
259         return String.join("\n",
260             ".voltar{display:inline-block;margin-bottom:14px;color:var(--ink2);text-decoration:none;",
261             "    font-size:13px}",
262             ".voltar:hover{text-decoration:underline}",
263             "h1{font-size:21px;margin-bottom:12px}",
264             "h2{margin-top:26px}",
265             ".texto{white-space:pre-wrap;word-break:break-word;font-size:12.5px;line-height:1.6;",
266             "    background:var(--surface);border:1px solid var(--grade);border-radius:8px;",

```

```
267         " padding:14px;max-height:620px;overflow:auto}",
268         "mark{background:#fde68a;color:#111;border-radius:2px;padding:0 1px}",
269         ".fato{border:1px solid var(--grade);border-radius:8px;padding:9px 12px;",
270         " margin-bottom:7px;background:var(--surface);font-size:13px}",
271         ".fato .pos{margin-left:6px;font-size:11px;color:var(--ink3)}",
272         ".prova{margin-top:5px;font-size:12.5px;color:var(--ink2)}",
273         ".leitura-ato{font-size:16px;line-height:1.5;padding:14px 16px;border-radius:8px;",
274         " border:1px solid var(--grade);border-left:3px solid var(--serie);",
275         " background:var(--surface);margin-bottom:16px}",
276         ".rotulo-leitura{font-size:11px;letter-spacing:.06em;text-transform:uppercase;",
277         " color:var(--ink3);margin-bottom:4px}",
278         ".leitura-curta{font-size:12.5px;color:var(--ink2);max-width:380px}",
279         ".ficha th{width:170px;text-transform:none;letter-spacing:0;font-size:12.5px}");
280     }
281 }
```

ARQUIVO: diarios/Aplicacao.java

pacote diarios · 306 linhas · O "cérebro": processarPasta(), coletar(), descobrir() e a coleta em segundo plano.

```
1 package diarios;
2
3 import diarios.analise.Cobertura;
4 import diarios.analise.GrafoNormas;
5 import diarios.analise.Interpretacao;
6 import diarios.analise.LeituraDoAto;
7 import diarios.analise.PainelContratos;
8 import diarios.coleta.Coletor;
9 import diarios.coleta.EstadoFonte;
10 import diarios.coleta.Fontes;
11 import diarios.coleta.Municipio;
12 import diarios.coleta.ResultadoColeta;
13 import diarios.coleta.Rodada;
14 import diarios.erros.ColetaException;
15 import diarios.erros.DiarioIlegivelException;
16 import diarios.etapas.Extracao;
17 import diarios.etapas.Fragmentacao;
18 import diarios.etapas.Ingestao;
19 import diarios.etapas.Limpeza;
20 import diarios.etapas.Portao;
21 import diarios.modelo.Diario;
22 import java.io.IOException;
23 import java.nio.charset.StandardCharsets;
24 import java.nio.file.Files;
25 import java.nio.file.Path;
26 import java.nio.file.Paths;
27 import java.nio.file.StandardOpenOption;
28 import java.time.LocalDate;
29 import java.time.LocalDateTime;
30 import java.time.format.DateTimeFormatter;
31 import java.util.ArrayList;
32
33 public class Aplicacao {
34
35     public static final String PASTA_DIARIOS = "dados/diarios";
36     public static final String PASTA_SAIDA = "dados/saida";
37     public static final String PASTA_ESTADO = "dados/estado";
38
39     public static final int DIAS_MAXIMO = 90;
40     private static final int DIAS_DESCOBERTA = 3;
41     private static final int RODADAS_NA_TELA = 12;
42     private static final int FALHAS_SEGUIDAS_PARA_PARAR = 3;
43
44     private final Fontes fontes;
45     private final Coletor coletor = new Coletor(PASTA_DIARIOS);
46     private final Path arquivoRodadas = Paths.get(PASTA_ESTADO, "rodadas.txt");
47
48     private ArrayList<Diario> diarios = new ArrayList<>();
49     private Cobertura cobertura = new Cobertura();
50     private PainelContratos painel = new PainelContratos();
51     private GrafoNormas grafo = new GrafoNormas();
52     private Interpretacao interpretacao = new Interpretacao();
53     private ArrayList<String> falhasDeLeitura = new ArrayList<>();
54     private String ultimoProcessamento = "-";
55
56     private final ArrayList<Rodada> rodadas = new ArrayList<>();
57     private Rodada ultimaRodada = null;
58
59     private volatile boolean ocupado = false;
60     private volatile String progresso = "";
61
62     public Aplicacao() {
63         criarPastas();
64         fontes = new Fontes(PASTA_ESTADO);
65         carregarRodadas();
66         fontes.atualizarAcervo(PASTA_DIARIOS);
67     }
68
69     public void processarPasta() {
70         ArrayList<String> falhas = new ArrayList<>();
71         ArrayList<Diario> lidos = new Ingestao(PASTA_DIARIOS).lerTodos(falhas);
72
73         ArrayList<Diario> novos = new ArrayList<>();
74         Cobertura novaCobertura = new Cobertura();
75         Limpeza limpeza = new Limpeza();
76         Extracao extracao = new Extracao();
77         Portao portao = new Portao();
78
79         for (Diario d : lidos) {
80             try {
81                 limpeza.limpar(d);
82             } catch (DiarioIlegivelException e) {
83                 novaCobertura.registrarRecusa(e.getArquivo(), e.getMessage(), e.getEvidencia());
84                 novos.add(d);
85                 continue;
86             }
87         }
88     }
89 }
```

```

87         new Fragmentacao().fragmentar(d);
88         extracao.extrair(d);
89         portao.avaliar(d);
90         novaCobertura.registrarProcessado(d);
91         novos.add(d);
92     }
93
94     PainelContratos novoPainel = new PainelContratos();
95     novoPainel.processar(novos);
96     GrafoNormas novoGrafo = new GrafoNormas();
97     novoGrafo.processar(novos);
98     Interpretacao novaInterpretacao = new Interpretacao();
99     novaInterpretacao.interpretar(novos, novaCobertura, novoPainel, novoGrafo);
100
101     synchronized (this) {
102         diarios = novos;
103         cobertura = novaCobertura;
104         painel = novoPainel;
105         grafo = novoGrafo;
106         interpretacao = novaInterpretacao;
107         falhasDeLeitura = falhas;
108         ultimoProcessamento = agora();
109     }
110     fontes.atualizarAcervo(PASTA_DIARIOS);
111 }
112
113 public Rodada coletar(int dias) {
114     return coletar(dias, false);
115 }
116
117 public Rodada coletar(int dias, boolean refazerTudo) {
118     int n = Math.max(1, Math.min(dias, DIAS_MAXIMO));
119     LocalDate hoje = LocalDate.now();
120     String desde = hoje.minusDays(n).toString();
121     String ate = hoje.toString();
122
123     ArrayList<EstadoFonte> alvo = fontes.ligadas();
124     Rodada rodada = new Rodada(agora(), "coleta", n, desde, ate);
125
126     int i = 0;
127     int falhasSeguidas = 0;
128     String interrompida = "";
129     for (EstadoFonte estado : alvo) {
130         i++;
131         if (!refazerTudo && estado.jaCobre(desde, ate)) {
132             ResultadoColeta pulada = new ResultadoColeta(estado.getMunicipio());
133             pulada.marcarPulada();
134             pulada.anotar(estado.getMunicipio().getNome() + ": pulada, ja coletada ("
135                 + estado.getStatus() + " de " + estado.getCoberturaDesde() + " a "
136                 + estado.getCoberturaAte() + ")");
137             rodada.adicionar(pulada);
138             continue;
139         }
140         progresso = "coletando " + estado.getMunicipio().getNome()
141             + " (" + i + " de " + alvo.size() + ")";
142         ResultadoColeta r = coletor.coletar(estado.getMunicipio(), desde, ate);
143         estado.registrarResultado(agora(), r, desde, ate);
144         rodada.adicionar(r);
145
146         falhasSeguidas = r.isFalhouRede() ? falhasSeguidas + 1 : 0;
147         if (falhasSeguidas >= FALHAS_SEGUIDAS_PARA_PARAR && i < alvo.size()) {
148             interrompida = "interrompida depois de " + FALHAS_SEGUIDAS_PARA_PARAR
149                 + " falhas seguidas; " + (alvo.size() - i)
150                 + " fonte(s) ficaram para a proxima rodada";
151             break;
152         }
153     }
154     fontes.salvar();
155
156     progresso = "lendo e interpretando os diarios da pasta";
157     processarPasta();
158
159     String obs = alvo.isEmpty() ? "nenhuma fonte ligada" : interrompida;
160     rodada.finalizar(agora(), obs);
161     registrarRodada(rodada);
162     return rodada;
163 }
164
165 public Rodada descobrir() {
166     LocalDate hoje = LocalDate.now();
167     String desde = hoje.minusDays(DIAS_DESCOBERTA).toString();
168     String ate = hoje.toString();
169     Rodada rodada = new Rodada(agora(), "descoberta", DIAS_DESCOBERTA, desde, ate);
170
171     progresso = "perguntando a API quem publicou de " + desde + " a " + ate;
172     try {
173         ArrayList<Municipio> achados = coletor.descobrir(desde, ate);
174         int novas = fontes.adicionarDescobertas(achados, agora());
175         rodada.finalizar(agora(), achados.size() + " municipio(s) com texto publicado, "
176             + novas + " novo(s) na lista");

```

```

177         } catch (ColetaException e) {
178             rodada.finalizar(agora(), "falhou: " + e.getMessage());
179         }
180         fontes.atualizarAcervo(PASTA_DIARIOS);
181         registrarRodada(rodada);
182         return rodada;
183     }
184
185     public synchronized boolean coletarEmSegundoPlano(int dias) {
186         return coletarEmSegundoPlano(dias, false);
187     }
188
189     public synchronized boolean coletarEmSegundoPlano(int dias, boolean refazerTudo) {
190         if (ocupado) {
191             return false;
192         }
193         ocupado = true;
194         Thread t = new Thread(() -> {
195             try {
196                 coletar(dias, refazerTudo);
197             } finally {
198                 ocupado = false;
199                 progresso = "";
200             }
201         }, "coleta");
202         t.setDaemon(true);
203         t.start();
204         return true;
205     }
206
207     public synchronized boolean descobrirEmSegundoPlano() {
208         if (ocupado) {
209             return false;
210         }
211         ocupado = true;
212         Thread t = new Thread(() -> {
213             try {
214                 descobrir();
215             } finally {
216                 ocupado = false;
217                 progresso = "";
218             }
219         }, "descoberta");
220         t.setDaemon(true);
221         t.start();
222         return true;
223     }
224
225     public void alternarFonte(String codigo) {
226         fontes.alternar(codigo);
227     }
228
229     public void ligarTodas(boolean ligar) {
230         fontes.ligarTodas(ligar);
231     }
232
233     public synchronized LeituraDoAto leitor() {
234         return new LeituraDoAto(painel, grafo);
235     }
236
237     public synchronized Diario diarioDoArquivo(String arquivo) {
238         for (Diario d : diarios) {
239             if (d.getArquivo().equals(arquivo)) {
240                 return d;
241             }
242         }
243         return null;
244     }
245
246     public synchronized ArrayList<Rodada> rodadasRecentes() {
247         ArrayList<Rodada> lista = new ArrayList<>();
248         for (int i = rodadas.size() - 1; i >= 0 && lista.size() < RODADAS_NA_TELA; i--) {
249             lista.add(rodadas.get(i));
250         }
251         return lista;
252     }
253
254     private synchronized void registrarRodada(Rodada rodada) {
255         rodadas.add(rodada);
256         ultimaRodada = rodada;
257         try {
258             Files.writeString(arquivoRodadas, rodada.paraLinha() + System.lineSeparator(),
259                             StandardCharsets.UTF_8, StandardOpenOption.CREATE, StandardOpenOption.APPEND);
260         } catch (IOException e) {
261             System.out.println("aviso: nao consegui registrar a rodada: " + e.getMessage());
262         }
263     }
264
265     private void carregarRodadas() {
266         if (!Files.exists(arquivoRodadas)) {

```

```

267         return;
268     }
269     try {
270         for (String linha : Files.readAllLines(arquivoRodadas, StandardCharsets.UTF_8)) {
271             Rodada r = Rodada.daLinha(linha);
272             if (r != null) {
273                 rodadas.add(r);
274             }
275         }
276     } catch (IOException | NumberFormatException e) {
277         System.out.println("aviso: historico de rodadas ilegível, começando vazio.");
278     }
279 }
280
281 private void criarPastas() {
282     try {
283         Files.createDirectories(Paths.get(PASTA_DIARIOS));
284         Files.createDirectories(Paths.get(PASTA_SAIDA));
285         Files.createDirectories(Paths.get(PASTA_ESTADO));
286     } catch (IOException e) {
287         System.out.println("aviso: nao consegui criar as pastas: " + e.getMessage());
288     }
289 }
290
291 public static String agora() {
292     return LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));
293 }
294
295 public synchronized ArrayList<Diario> getDiarios() { return diarios; }
296 public synchronized Cobertura getCobertura() { return cobertura; }
297 public synchronized PaineLContratos getPaineL() { return paineL; }
298 public synchronized GrafoNormas getGrafo() { return grafo; }
299 public synchronized Interpretacao getInterpretacao() { return interpretacao; }
300 public synchronized ArrayList<String> getFalhasDeLeitura() { return falhasDeLeitura; }
301 public synchronized String getUltimoProcessamento() { return ultimoProcessamento; }
302 public synchronized Rodada getUltimaRodada() { return ultimaRodada; }
303 public Fontes getFontes() { return fontes; }
304 public boolean isOcupado() { return ocupado; }
305 public String getProgresso() { return progresso; }
306 }

```

ARQUIVO: diarios/Principal.java

pacote diarios · 486 linhas · Menu de terminal (Scanner + switch); chama a Aplicacao.

```
1 package diarios;
2
3 import diarios.coleta.EstadoFonte;
4 import diarios.coleta.Rodada;
5 import diarios.etapas.PaginaHtml;
6 import diarios.etapas.Relatorio;
7 import diarios.etapas.Visualizador;
8 import diarios.modelo.Achado;
9 import diarios.modelo.Ato;
10 import diarios.modelo.Contrato;
11 import diarios.modelo.Diario;
12 import diarios.modelo.Fato;
13 import diarios.modelo.Relacao;
14 import diarios.servidor.Servidor;
15 import java.io.IOException;
16 import java.util.ArrayList;
17 import java.util.Scanner;
18
19 public class Principal {
20
21     private final Aplicacao app = new Aplicacao();
22     private final Relatorio relatorio = new Relatorio(Aplicacao.PASTA_SAIDA);
23     private Servidor servidor = null;
24
25     public static void main(String[] args) {
26         Principal p = new Principal();
27         p.app.processarPasta();
28         if (args.length > 0 && args[0].equals("--tudo")) {
29             p.mostrarInterpretacao();
30             p.mostrarCobertura();
31             p.mostrarPainel();
32             p.mostrarGrafo();
33             p.gravarTudo();
34             return;
35         }
36         p.menu();
37     }
38
39     private void menu() {
40         Scanner teclado = new Scanner(System.in);
41         boolean rodando = true;
42
43         while (rodando) {
44             System.out.println();
45             System.out.println("=====");
46             System.out.println("  DIARIOS OFICIAIS - coleta, leitura e analise");
47             System.out.println("=====");
48             System.out.println("  1 - Coletar das fontes ligadas (escolher dias);");
49             System.out.println("  2 - Fontes: ver, ligar e desligar");
50             System.out.println("  3 - Descobrir municipios com publicacao recente");
51             System.out.println("  4 - Reler a pasta de diarios");
52             System.out.println("  5 - INTERPRETACAO: o que esses dados dizem");
53             System.out.println("  6 - Ver os arquivos (entrada e saida)");
54             System.out.println("  7 - Painel de contratacoes");
55             System.out.println("  8 - Grafo de normas");
56             System.out.println("  9 - Relatorio de cobertura");
57             System.out.println(" 10 - Consultar uma norma no grafo");
58             System.out.println(" 11 - Conferir um fato no texto original");
59             System.out.println(" 12 - Gravar relatorios em saida/");
60             System.out.println(" 13 - Abrir o painel no navegador (localhost)");
61             System.out.println(" 0 - Sair");
62             System.out.print("  escolha: ");
63
64             String escolha = teclado.hasNextLine() ? teclado.nextLine().trim() : "0";
65
66             switch (escolha) {
67                 case "1": coletar(teclado); break;
68                 case "2": gerenciarFontes(teclado); break;
69                 case "3": descobrir(); break;
70                 case "4": reler(); break;
71                 case "5": mostrarInterpretacao(); break;
72                 case "6": verArquivos(teclado); break;
73                 case "7": mostrarPainel(); break;
74                 case "8": mostrarGrafo(); break;
75                 case "9": mostrarCobertura(); break;
76                 case "10": consultarNorma(teclado); break;
77                 case "11": conferirFato(teclado); break;
78                 case "12": gravarTudo(); break;
79                 case "13": abrirPainel(); break;
80                 case "0": rodando = false; break;
81                 default: System.out.println(" opcao invalida.");
82             }
83         }
84         if (servidor != null) {
85             servidor.parar();
86         }
87     }
88 }
```

```

87         System.out.println(" ate mais.");
88         teclado.close();
89     }
90
91     private void coletar(Scanner teclado) {
92         ArrayList<EstadoFonte> ligadas = app.getFontes().ligadas();
93         if (ligadas.isEmpty()) {
94             System.out.println(" nenhuma fonte ligada. Use a opcao 2.");
95             return;
96         }
97         System.out.print(" quantos dias para tras [1]: ");
98         int dias = lerInteiro(teclado, 1);
99         System.out.println();
100        System.out.println(" coletando " + ligadas.size() + " fonte(s), " + dias + " dia(s)...");
101
102        Rodada r = app.coletar(dias);
103        System.out.println(" puladas (ja coletadas): " + r.puladas());
104        for (String linha : r.registro()) {
105            System.out.println(" " + linha);
106        }
107        System.out.println();
108        System.out.println(" baixados: " + r.baixados() + " ja existiam: " + r.jaExistiam()
109            + " falhas de rede: " + r.falhas());
110        reler();
111    }
112
113    private void gerenciarFontes(Scanner teclado) {
114        while (true) {
115            ArrayList<EstadoFonte> todas = app.getFontes().todas();
116            System.out.println();
117            for (int i = 0; i < todas.size(); i++) {
118                EstadoFonte e = todas.get(i);
119                System.out.printf(" %2d [%s] %-24s %-3s %-14s acervo=%d%n", i,
120                    e.isLigada() ? "ON" : "off", corta(e.getMunicipio().getNome(), 24),
121                    e.getMunicipio().getUf(), e.getStatus(), e.getDiariosNoAcervo());
122            }
123            System.out.print(" numero para ligar/desligar, 't' liga todas, 'n' desliga todas "
124                + "(enter volta): ");
125            String entrada = teclado.hasNextLine() ? teclado.nextLine().trim() : "";
126            if (entrada.isEmpty()) {
127                return;
128            }
129            if (entrada.equalsIgnoreCase("t") || entrada.equalsIgnoreCase("n")) {
130                app.ligarTodas(entrada.equalsIgnoreCase("t"));
131                continue;
132            }
133            try {
134                int i = Integer.parseInt(entrada);
135                if (i >= 0 && i < todas.size()) {
136                    app.alternarFonte(todas.get(i).getMunicipio().getCodigo());
137                } else {
138                    System.out.println(" fora da lista.");
139                }
140            } catch (NumberFormatException e) {
141                System.out.println(" isso nao e um numero.");
142            }
143        }
144    }
145
146    private void descobrir() {
147        System.out.println();
148        System.out.println(" perguntando a API quais municipios publicaram recentemente...");
149        Rodada r = app.descobrir();
150        System.out.println(" " + r.getObservacao());
151        System.out.println(" as novas entram desligadas. Use a opcao 2 para ligar.");
152    }
153
154    private void reler() {
155        app.processarPasta();
156        System.out.println();
157        for (String f : app.getFalhasDeLeitura()) {
158            System.out.println(" ! " + f);
159        }
160        for (Diario d : app.getDiarios()) {
161            if (d.isUtilizavel()) {
162                System.out.println(" " + d.getArquivo() + " ... " + d.getAtos().size() + " atos ("
163                    + d.contar(Ato.ACEITO) + " aceitos, " + d.contar(Ato.DUVIDOSO)
164                    + " duvidosos, " + d.contar(Ato.REJEITADO) + " rejeitados)");
165            } else {
166                System.out.println(" " + d.getArquivo() + " ... RECUSADO ("
167                    + d.getMotivoRecusa() + ")");
168            }
169        }
170        System.out.println();
171        System.out.println(" pronto. " + app.getCobertura().getUtilizaveis() + " utilizavel(is), "
172            + app.getCobertura().getRecusados() + " recusado(s).");
173    }
174
175    private void abrirPainel() {
176        if (servidor == null) {

```

```

177         servidor = new Servidor(app);
178         if (!servidor.iniciar(8080)) {
179             servidor = null;
180             return;
181         }
182     }
183     System.out.println(" painel em http://localhost:" + servidor.getPorta()
184         + " (fica no ar enquanto o menu estiver aberto)");
185 }
186
187 private void mostrarInterpretacao() {
188     System.out.println();
189     System.out.println("O QUE ESSES DADOS DIZEM");
190     System.out.println("=====");
191     System.out.println();
192     System.out.println(" " + app.getInterpretacao().getResumo());
193
194     mostrarAchados("LEITURA DO PERIODO", app.getInterpretacao().doNivel(Achado.LEITURA));
195     mostrarAchados("ONDE VALE A PENA OLHAR", app.getInterpretacao().doNivel(Achado.ATENCAO));
196     mostrarAchados("LIMITES DESTA LEITURA", app.getInterpretacao().doNivel(Achado.LIMITE));
197
198     System.out.println();
199     System.out.println(" Cada frase acima sai de uma regra no codigo, sobre numero");
200     System.out.println(" medido no texto. Nenhuma foi escrita a mao.");
201 }
202
203 private void mostrarAchados(String secao, ArrayList<Achado> lista) {
204     if (lista.isEmpty()) {
205         return;
206     }
207     System.out.println();
208     System.out.println(" " + secao);
209     System.out.println(" " + "-".repeat(secao.length()));
210     for (Achado a : lista) {
211         System.out.println();
212         System.out.println(" " + a.getTitulo());
213         for (String linha : quebrarTexto(a.getFrase(), 72)) {
214             System.out.println(" " + linha);
215         }
216         System.out.println(" -> " + a.getEvidencia());
217     }
218 }
219
220 private ArrayList<String> quebrarTexto(String texto, int largura) {
221     ArrayList<String> linhas = new ArrayList<>();
222     StringBuilder atual = new StringBuilder();
223     for (String palavra : texto.split(" ")) {
224         if (atual.length() + palavra.length() + 1 > largura) {
225             linhas.add(atual.toString());
226             atual = new StringBuilder();
227         }
228         atual.append(atual.length() > 0 ? " " : "").append(palavra);
229     }
230     if (atual.length() > 0) {
231         linhas.add(atual.toString());
232     }
233     return linhas;
234 }
235
236 private void verArquivos(Scanner teclado) {
237     System.out.println();
238     System.out.println(" 1 - diarios/ (o que entrou)");
239     System.out.println(" 2 - saida/ (o que o programa produziu)");
240     System.out.print(" escolha: ");
241
242     String escolha = teclado.hasNextLine() ? teclado.nextLine().trim() : "";
243     Visualizador visual = new Visualizador(teclado);
244
245     if (escolha.equals("1")) {
246         visual.abrir(Aplicacao.PASTA_DIARIOS, app.getDiarios());
247     } else if (escolha.equals("2")) {
248         visual.abrir(Aplicacao.PASTA_SAIDA, new ArrayList<>());
249     }
250 }
251
252 private void mostrarCobertura() {
253     System.out.println();
254     for (String l : app.getCobertura().montar()) {
255         System.out.println(l);
256     }
257 }
258
259 private void mostrarPainel() {
260     ArrayList<Contrato> contratos = app.getPainel().getContratos();
261     System.out.println();
262     System.out.println("PAINEL DE CONTRATAÇÕES");
263     System.out.println("=====");
264     System.out.println();
265
266     if (contratos.isEmpty()) {

```

```

267         System.out.println(" nenhuma contratacao encontrada.");
268         return;
269     }
270
271     System.out.println(" contratacoes : " + contratos.size());
272     System.out.println(" valor total : " + Contrato.formatarReal(app.getPainel().total()));
273     System.out.println();
274     System.out.printf(" %-20s %-38s %18s %6s%n", "CNPJ", "CONTRATADA", "TOTAL", "VEZES");
275     System.out.println(" " + "-".repeat(84));
276
277     ArrayList<String[]> ranking = app.getPainel().ranking();
278     int mostrar = Math.min(15, ranking.size());
279     for (int i = 0; i < mostrar; i++) {
280         String[] r = ranking.get(i);
281         System.out.printf(" %-20s %-38s %18s %6s%n", r[0], corta(r[1], 38), r[2], r[3]);
282     }
283     if (ranking.size() > mostrar) {
284         System.out.println(" ... e mais " + (ranking.size() - mostrar) + " fornecedor(es)");
285     }
286 }
287
288 private void mostrarGrafo() {
289     System.out.println();
290     System.out.println("GRAFO DE NORMAS");
291     System.out.println("=====");
292     System.out.println();
293     System.out.println(" relacoes : " + app.getGrafo().getArestas().size());
294     System.out.println(" ALTERA : " + app.getGrafo().doTipo(Relacao.ALTERA).size());
295     System.out.println(" REVOGA : " + app.getGrafo().doTipo(Relacao.REVOGA).size());
296     System.out.println(" PRORROGA : " + app.getGrafo().doTipo(Relacao.PRORROGA).size());
297     System.out.println(" CITA : " + app.getGrafo().doTipo(Relacao.CITA).size());
298
299     ArrayList<String[]> mexidas = app.getGrafo().maisAlteradas();
300     if (!mexidas.isEmpty()) {
301         System.out.println();
302         System.out.println(" NORMAS MAIS MEXIDAS");
303         int mostrar = Math.min(10, mexidas.size());
304         for (int i = 0; i < mostrar; i++) {
305             System.out.printf(" %2sx %s%n", mexidas.get(i)[1], mexidas.get(i)[0]);
306         }
307     }
308 }
309
310 private void consultarNorma(Scanner teclado) {
311     System.out.print(" norma (ex: lei:11065): ");
312     String norma = teclado.hasNextLine() ? teclado.nextLine().trim() : "";
313     if (norma.isEmpty()) {
314         return;
315     }
316
317     System.out.println();
318     System.out.println(" QUEM MEXEU EM " + norma);
319     boolean achou = false;
320     for (Relacao r : app.getGrafo().getArestas()) {
321         if (r.getAlvo().equals(norma)) {
322             System.out.println(" " + r);
323             achou = true;
324         }
325     }
326     if (!achou) {
327         System.out.println(" nenhuma relacao com essa norma.");
328         return;
329     }
330
331     ArrayList<String> vizinhos = app.getGrafo().vizinhanca(norma, 2);
332     System.out.println();
333     System.out.println(" VIZINHANCA EM 2 NIVEIS: " + vizinhos.size() + " nos");
334     int mostrar = Math.min(12, vizinhos.size());
335     for (int i = 0; i < mostrar; i++) {
336         System.out.println(" " + vizinhos.get(i));
337     }
338 }
339
340 private void conferirFato(Scanner teclado) {
341     Diario escolhido = null;
342     for (Diario d : app.getDiarios()) {
343         if (d.isUtilizavel() && !d.getAtos().isEmpty()) {
344             escolhido = d;
345             break;
346         }
347     }
348     if (escolhido == null) {
349         System.out.println(" nenhum diario utilizavel.");
350         return;
351     }
352
353     System.out.println();
354     System.out.println(" atos de " + escolhido.getArquivo() + ":");
355     int mostrar = Math.min(15, escolhido.getAtos().size());
356     for (int i = 0; i < mostrar; i++) {

```

```

357         System.out.printf("    %2d - %5s\n", i, escolhido.getAtos().get(i));
358     }
359     System.out.print(" numero do ato: ");
360
361     int indice = lerInteiro(teclado, -1);
362     if (indice < 0 || indice >= escolhido.getAtos().size()) {
363         System.out.println(" fora da lista.");
364         return;
365     }
366
367     Ato ato = escolhido.getAtos().get(indice);
368     System.out.println();
369     System.out.println(" " + ato.getTitulo());
370     System.out.println(" leitura : " + app.leitor().ler(ato, escolhido.getData()));
371     System.out.println(" identidade : " + ato.getIdentidade());
372     System.out.println(" decisao : " + ato.getDecisao() + " - " + ato.getMotivo());
373     System.out.println();
374     System.out.println(" FATOS E A PROVA DE CADA UM NO TEXTO ORIGINAL:");
375
376     int limite = Math.min(12, ato.getFatos().size());
377     for (int i = 0; i < limite; i++) {
378         Fato f = ato.getFatos().get(i);
379         System.out.println(" " + f);
380         System.out.println(" ..." + f.provar(escolhido.getTextoLimpo(), 45) + "...");
381     }
382 }
383
384 private void gravarTudo() {
385     try {
386         System.out.println();
387         System.out.println(" gravado em:");
388         System.out.println(" " + relatorio.gravar("interpretacao.txt", linhasDeInterpretacao()));
389         System.out.println(" " + relatorio.gravar("cobertura.txt", app.getCobertura().montar()));
390         System.out.println(" " + relatorio.gravar("contratos.txt", linhasDeContratos()));
391         System.out.println(" " + relatorio.gravar("grafo.txt", linhasDeGrafo()));
392         System.out.println(" " + relatorio.gravar("atos.txt", linhasDeAtos()));
393         System.out.println(" " + new PaginaHtml(relatorio).gerar(app.getDiarios(),
394             app.getCobertura(), app.getPainel(), app.getGrafo(), app.getInterpretacao()));
395     } catch (IOException e) {
396         System.out.println(" falha ao gravar: " + e.getMessage());
397     }
398 }
399
400 private ArrayList<String> linhasDeInterpretacao() {
401     ArrayList<String> l = new ArrayList<>();
402     l.add("O QUE ESSES DADOS DIZEM");
403     l.add("=====");
404     l.add("");
405     l.add(app.getInterpretacao().getResumo());
406     for (Achado a : app.getInterpretacao().getAchados()) {
407         l.add("");
408         l.add("[ " + a.getNivel() + " ] " + a.getTitulo());
409         l.add(" " + a.getFrase());
410         l.add(" evidencia: " + a.getEvidencia());
411     }
412     return l;
413 }
414
415 private ArrayList<String> linhasDeContratos() {
416     ArrayList<String> l = new ArrayList<>();
417     l.add("CONTRATACOES");
418     l.add("municipio;data;cnpj;contratada;valor;processo;ato");
419     for (Contrato c : app.getPainel().getContratos()) {
420         l.add(String.join(";", c.getMunicipio(), c.getData(), c.getCnpj(),
421             c.getContratada().replace(';', ','),
422             String.format(java.util.Locale.ROOT, "%.2f", c.getValor()), c.getProcesso(), c.getIdentidadeAto()));
423     }
424     l.add("");
425     l.add("TOTAL;" + Contrato.formatarReal(app.getPainel().total()));
426     l.add("");
427     l.add("RANKING DE FORNECEDORES");
428     l.add("cnpj;contratada;total;vezes");
429     for (String[] r : app.getPainel().ranking()) {
430         l.add(String.join(";", r[0], r[1].replace(';', ','), r[2], r[3]));
431     }
432     return l;
433 }
434
435 private ArrayList<String> linhasDeGrafo() {
436     ArrayList<String> l = new ArrayList<>();
437     l.add("GRAFO DE NORMAS");
438     l.add("origem;acao;alvo;posicao");
439     for (Relacao r : app.getGrafo().getArestas()) {
440         l.add(String.join(";", r.getOrigem(), r.getAcao(), r.getAlvo(),
441             String.valueOf(r.getPosicao())));
442     }
443     l.add("");
444     l.add("NORMAS MAIS MEXIDAS");
445     l.add("norma;vezes");
446     for (String[] m : app.getGrafo().maisAlteradas()) {

```

```
447         l.add(m[0] + ";" + m[1]);
448     }
449     return l;
450 }
451
452 private ArrayList<String> linhasDeAto() {
453     ArrayList<String> l = new ArrayList<>();
454     l.add("ATOS");
455     l.add("identidade;especie;decisao;motivo;fatos;leitura;titulo");
456     for (Diario d : app.getDiarios()) {
457         for (Ato a : d.getAto()) {
458             l.add(String.join(";", a.getIdentidade(), a.getEspecie(), a.getDecisao(),
459                 a.getMotivo().replace(';', ' '), String.valueOf(a.getFatos().size()),
460                 app.leitor().ler(a, d.getData()).replace(';', ' '),
461                 a.getTitulo().replace(';', ' ')));
462         }
463     }
464     return l;
465 }
466
467 private int lerInteiro(Scanner teclado, int padrao) {
468     String entrada = teclado.hasNextLine() ? teclado.nextLine().trim() : "";
469     if (entrada.isEmpty()) {
470         return padrao;
471     }
472     try {
473         return Integer.parseInt(entrada);
474     } catch (NumberFormatException e) {
475         System.out.println(" numero invalido, usando " + padrao + ".");
476         return padrao;
477     }
478 }
479
480 private static String corta(String s, int max) {
481     if (s == null) {
482         return "";
483     }
484     return s.length() <= max ? s : s.substring(0, max - 1) + ".";
485 }
486 }
```

ARQUIVO: test/diarios/Testes.java

pacote diarios · 430 linhas · 85 verificações com um mini-framework próprio (sem JUnit).

```
1 package diarios;
2
3 import diarios.analise.GrafoNormas;
4 import diarios.analise.LeituraDoAto;
5 import diarios.analise.PainelContratos;
6 import diarios.coleta.EstadoFonte;
7 import diarios.coleta.Fonte;
8 import diarios.coleta.Fontes;
9 import diarios.coleta.Json;
10 import diarios.coleta.Municipio;
11 import diarios.coleta.ResultadoColeta;
12 import diarios.coleta.Rodada;
13 import diarios.erros.DiarioIllegivelException;
14 import diarios.etapas.Extracao;
15 import diarios.etapas.Fragmentacao;
16 import diarios.etapas.Ingestao;
17 import diarios.etapas.Limpeza;
18 import diarios.etapas.Portao;
19 import diarios.etapas.Privacidade;
20 import diarios.servidor.Formulario;
21 import java.io.IOException;
22 import java.nio.file.Files;
23 import java.nio.file.Path;
24 import diarios.extratores.ExtratorCnpj;
25 import diarios.extratores.ExtratorReferencia;
26 import diarios.extratores.ExtratorValor;
27 import diarios.modelo.Ato;
28 import diarios.modelo.Diario;
29 import diarios.modelo.Fato;
30 import java.util.ArrayList;
31
32 public class Testes {
33
34     private static int passou = 0;
35     private static int falhou = 0;
36
37     public static void main(String[] args) {
38         System.out.println("TESTES");
39         System.out.println("=====");
40         System.out.println();
41
42         java.util.Locale.setDefault(java.util.Locale.forLanguageTag("pt-BR"));
43         conferir("formata real em Windows portugues", "R$ 8.323.312,07",
44             diarios.modelo.Contrato.formatarReal(8323312.07));
45         testarExtratorValor();
46         testarExtratorCnpj();
47         testarExtratorReferencia();
48         testarIngestao();
49         testarFonte();
50         testarJson();
51         testarPapeis();
52         testarFontes();
53         testarFormulario();
54         testarPrivacidade();
55         testarRodada();
56         testarPipelineCompleto();
57
58         System.out.println();
59         System.out.println("passou: " + passou + "   falhou: " + falhou);
60         if (falhou > 0) {
61             System.exit(1);
62         }
63     }
64
65     private static void testarExtratorValor() {
66         System.out.println("-- ExtratorValor");
67         ExtratorValor e = new ExtratorValor();
68
69         ArrayList<Fato> achados = e.extrair("valor global de R$ 184.500,00 para o exercicio");
70         conferir("acha um valor", 1, achados.size());
71         conferir("valor certo", "R$ 184.500,00", achados.get(0).getValor());
72         conferir("converte para numero", 184500.0, ExtratorValor.paraNumero("R$ 184.500,00"));
73         conferir("ignora R$ sem centavos", 0, e.extrair("multa de R$ 18 por dia").size());
74
75         String texto = "abcde R$ 1.000,00 fim";
76         Fato f = e.extrair(texto).get(0);
77         conferir("span aponta certo", "R$ 1.000,00",
78             texto.substring(f.getInicio(), f.getFim()));
79     }
80
81     private static void testarExtratorCnpj() {
82         System.out.println("-- ExtratorCnpj");
83         ExtratorCnpj e = new ExtratorCnpj();
84
85         String real = "Deferidos: AKASULO, CNPJ: 48.356.311/0001-14; FLB ATACADO LTDA";
86         ArrayList<Fato> achados = e.extrair(real);
```

```

87         conferir("acha o CNPJ", 1, achados.size());
88         conferir("CNPJ certo", "48.356.311/0001-14", achados.get(0).getValor());
89         conferir("nao confunde com CPF", 0, e.extrair("CPF 069.555.516-20").size());
90     }
91
92     private static void testarExtratorReferencia() {
93         System.out.println("-- ExtratorReferencia");
94         ExtratorReferencia e = new ExtratorReferencia();
95
96         conferir("lei simples", "lei:11065",
97             e.extrair("da Lei nº 11.065, de 2017").get(0).getValor());
98
99         ArrayList<Fato> comSigla =
100             e.extrair("0 Art. 2º da Portaria SMC nº 073/2025 passa a vigorar");
101         conferir("portaria com sigla de orgao", 1, comSigla.size());
102         conferir("numero da portaria", "portaria:073", comSigla.get(0).getValor());
103
104         conferir("grafias diferentes, mesma norma",
105             e.extrair("Lei nº 11.065").get(0).getValor(),
106             e.extrair("LEI MUNICIPAL 11065").get(0).getValor());
107     }
108
109     private static void testarIngestao() {
110         System.out.println("-- Ingestao");
111         conferir("le o municipio do nome", "belo-horizonte",
112             Ingestao.municipioDe("belo-horizonte_2026-03-25.txt"));
113         conferir("le a data do nome", "2026-03-25",
114             Ingestao.dataDe("belo-horizonte_2026-03-25.txt"));
115     }
116
117     private static void testarFonte() {
118         System.out.println("-- Fonte");
119         Fonte f = new Fonte();
120         conferir("27 capitais", 27, f.todos().size());
121         conferir("acha por nome", "3106200", f.porNome("belo").getCodigo());
122         conferir("acha por codigo", "Recife", f.porNome("2611606").getNome());
123         conferir("nome de arquivo sem acento", "belo-horizonte",
124             f.porNome("3106200").getApelido());
125         conferir("sao paulo vira sao-paulo", "sao-paulo", f.porNome("3550308").getApelido());
126         conferir("conjunto padrao tem 5", 5, f.padrao().size());
127         conferir("municipio inexistente devolve nulo", true, f.porNome("xyz123") == null);
128     }
129
130     private static void testarJson() {
131         System.out.println("-- Json");
132         String resposta = "{\"total_gazettes\":2,\"gazettes\":["
133             + "{\"date\":\"2026-03-25\",\"txt_url\":\"http://a/1.txt\",\"edition\":\"7466\"},\"
134             + "{\"date\":\"2026-03-26\",\"txt_url\":null}}]";
135
136         ArrayList<String> objetos = Json.objetos(resposta, "gazettes");
137         conferir("separa as duas edicoes", 2, objetos.size());
138         conferir("le a data", "2026-03-25", Json.campo(objetos.get(0), "date"));
139         conferir("le a url", "http://a/1.txt", Json.campo(objetos.get(0), "txt_url"));
140         conferir("url nula vira vazio", "", Json.campo(objetos.get(1), "txt_url"));
141         conferir("chave ausente vira vazio", "", Json.campo(objetos.get(1), "edition"));
142
143         String comAcento = "{\"gazettes\": [{\"territory_name\": \"Macei\\u00f3\"}]";
144         conferir("decodifica acento escapado", "Maceió",
145             Json.campo(Json.objetos(comAcento, "gazettes").get(0), "territory_name"));
146
147         String semAspas = "{\"territory_id\": 3106200, \"is_extra_edition\": false, \"x\": null}";
148         conferir("le numero sem aspas", "3106200", Json.campo(semAspas, "territory_id"));
149         conferir("le booleano", "false", Json.campo(semAspas, "is_extra_edition"));
150         conferir("null sem aspas vira vazio", "", Json.campo(semAspas, "x"));
151     }
152
153     private static void testarFontes() {
154         System.out.println("-- Fontes: ligar, desligar e lembrar");
155         Path pasta;
156         try {
157             pasta = Files.createTempDirectory("fontes-teste");
158         } catch (IOException e) {
159             marcar(false, "criar pasta temporaria", e.getMessage());
160             return;
161         }
162
163         Fontes primeira = new Fontes(pasta.toString());
164         conferir("comeca com as 27 capitais", 27, primeira.todas().size());
165         conferir("sem arquivo, liga as 5 do padrao", 5, primeira.ligadas().size());
166
167         primeira.alternar("3550308");
168         conferir("ligar Sao Paulo soma uma", 6, primeira.ligadas().size());
169         primeira.alternar("3106200");
170         conferir("desligar BH tira uma", 5, primeira.ligadas().size());
171
172         ArrayList<Municipio> achados = new ArrayList<>();
173         achados.add(new Municipio("3509502", "Campinas", "SP"));
174         achados.add(new Municipio("3550308", "Sao Paulo", "SP"));
175         conferir("descoberta so conta o que e novo", 1,
176             primeira.adicionarDescobertas(achados, "agora"));

```

```

177         conferir("descoberta entra desligada", false, primeira.doCodigo("3509502").isLigada());
178         conferir("descoberta vira COM_DADO", EstadoFonte.COM_DADO,
179             primeira.doCodigo("3509502").getStatus());
180
181         Fontes reaberta = new Fontes(pasta.toString());
182         conferir("ao reabrir lembra quantas estao ligadas", 5, reaberta.ligadas().size());
183         conferir("ao reabrir lembra que SP esta ligada", true, reaberta.doCodigo("3550308").isLigada());
184         conferir("ao reabrir lembra que BH esta desligada", false,
185             reaberta.doCodigo("3106200").isLigada());
186         conferir("ao reabrir lembra a descoberta", 28, reaberta.todas().size());
187
188         reaberta.ligarTodas(false);
189         conferir("desligar todas zero", 0, new Fontes(pasta.toString()).ligadas().size());
190
191         ResultadoColeta falha = new ResultadoColeta(new Municipio("1", "X", "XX"));
192         falha.marcarFalhaDeRede("sem rede");
193         EstadoFonte e = new EstadoFonte(new Municipio("1", "X", "XX"), true, false);
194         e.registrarResultado("agora", falha, "2026-03-20", "2026-03-25");
195         conferir("falha de rede vira FALHOU", EstadoFonte.FALHOU, e.getStatus());
196
197         ResultadoColeta vazio = new ResultadoColeta(new Municipio("1", "X", "XX"));
198         e.registrarResultado("agora", vazio, "2026-03-20", "2026-03-25");
199         conferir("sem edicao vira SEM_EDICAO", EstadoFonte.SEM_EDICAO, e.getStatus());
200
201         System.out.println("-- Fontes: quando pular e quando coletar");
202         conferir("sem edicao no periodo: pula o mesmo periodo", true,
203             e.jaCobre("2026-03-20", "2026-03-25"));
204         conferir("sem edicao: pula periodo menor, dentro do coberto", true,
205             e.jaCobre("2026-03-24", "2026-03-25"));
206         conferir("periodo maior que o coberto: coleta", false,
207             e.jaCobre("2026-03-10", "2026-03-25"));
208         conferir("dia seguinte (periodo novo): coleta", false,
209             e.jaCobre("2026-03-21", "2026-03-26"));
210
211         ResultadoColeta baixou = new ResultadoColeta(new Municipio("1", "X", "XX"));
212         baixou.contarBaixado();
213         e.registrarResultado("agora", baixou, "2026-03-20", "2026-03-25");
214         conferir("com dado: pula", true, e.jaCobre("2026-03-20", "2026-03-25"));
215
216         ResultadoColeta soPdf = new ResultadoColeta(new Municipio("1", "X", "XX"));
217         soPdf.contarSemTexto();
218         e.registrarResultado("agora", soPdf, "2026-03-20", "2026-03-25");
219         conferir("so PDF conta como sem conteudo: pula", true, e.jaCobre("2026-03-20", "2026-03-25"));
220
221         e.registrarResultado("agora", falha, "2026-03-20", "2026-03-25");
222         conferir("falhou: nao pula, coleta de novo", false, e.jaCobre("2026-03-20", "2026-03-25"));
223
224         EstadoFonte nunca = new EstadoFonte(new Municipio("2", "Y", "YY"), true, false);
225         conferir("nunca tentada: nao pula", false, nunca.jaCobre("2026-03-20", "2026-03-25"));
226
227         Path pasta2;
228         try {
229             pasta2 = Files.createTempDirectory("fontes-cobertura");
230         } catch (IOException ex) {
231             marcar(false, "criar pasta temporaria", ex.getMessage());
232             return;
233         }
234         Fontes antes = new Fontes(pasta2.toString());
235         antes.doCodigo("3106200").registrarResultado("agora", baixou, "2026-03-20", "2026-03-25");
236         antes.salvar();
237         Fontes depois = new Fontes(pasta2.toString());
238         conferir("a cobertura sobrevive a fechar e abrir o programa", true,
239             depois.doCodigo("3106200").jaCobre("2026-03-20", "2026-03-25"));
240         conferir("fonte nao coletada continua sem cobertura", false,
241             depois.doCodigo("2611606").jaCobre("2026-03-20", "2026-03-25"));
242     }
243
244     private static void testarFormulario() {
245         System.out.println("-- Formulario");
246         conferir("le parametro", "7", Formulario.parametro("dias=7&x=1", "dias"));
247         conferir("decodifica espaco e acento", "sao joao",
248             Formulario.parametro("q=sao+jo%C3%A3o", "q").replace("ã", "a"));
249         conferir("inteiro invalido cai no padrao", 1, Formulario.inteiro("dias=abc", "dias", 1));
250         conferir("parametro ausente vira vazio", "", Formulario.parametro("a=1", "b"));
251         conferir("codifica e decodifica o nome do arquivo", "belo-horizonte_2026-03-25.txt",
252             Formulario.parametro("arquivo=" + Formulario.codificar(
253                 "belo-horizonte_2026-03-25.txt"), "arquivo"));
254     }
255
256     private static void testarPrivacidade() {
257         System.out.println("-- Privacidade");
258         String original = "servidor, CPF 069.555.516-20, lotado";
259         String mascarado = Privacidade.mascararCpf(original);
260         conferir("mascara o CPF", "servidor, CPF ***.***.***-20, lotado", mascarado);
261         conferir("mantem o tamanho (os spans continuam valendo)", original.length(),
262             mascarado.length());
263         conferir("nao mexe em CNPJ", "48.356.311/0001-14",
264             Privacidade.mascararCpf("48.356.311/0001-14"));
265     }
266
267     private static void testarRodada() {

```

```

267     System.out.println("-- Rodada: historico gravado e relido");
268     Rodada r = new Rodada("2026-03-25 10:00:00", "coleta", 3, "2026-03-22", "2026-03-25");
269     ResultadoColeta rc = new ResultadoColeta(new Municipio("1", "X", "XX"));
270     rc.contarBaixado();
271     rc.contarBaixado();
272     rc.contarJaExistia();
273     r.adicionar(rc);
274     r.finalizar("2026-03-25 10:01:00", "ok; tudo certo");
275
276     Rodada relida = Rodada.daLinha(r.paraLinha());
277     conferir("relida nao e nula", true, relida != null);
278     conferir("mantem os baixados", 2, relida.baixados());
279     conferir("mantem os que ja existiam", 1, relida.jaExistiam());
280     conferir("ponto e virgula da observacao nao quebra a linha", "ok, tudo certo",
281             relida.getObservacao());
282 }
283
284 private static void testarPapeis() {
285     System.out.println("-- PaineisContratos: papel de cada CNPJ");
286
287     String trechoReal = "NOTIFICADO: ACACIA COMERCIO DE MEDICAMENTOS LTDA., CNPJ N "
288         + "03.945.035/0001-91\nNOTIFICANTE: HOSPITAL METROPOLITANO ODILON BEHRENS, "
289         + "sito na Rua Formiga, n. 50, Bairro Sao Cristovao, Belo Horizonte/MG, "
290         + "inscrito no CNPJ sob o n\n16.692.121/0001-81, neste ato representado. "
291         + "Multa de R$ 2.461,28 devida.";
292
293     Diario d = new Diario("teste_2026-03-25.txt", "teste", "2026-03-25", trechoReal);
294     d.registrarLimpeza(trechoReal, 0);
295     d.adicionarAto(new Ato("teste", "2026-03-25", "TERMO", "1",
296         "TERMO", trechoReal, 0));
297     new Extracao().extrair(d);
298
299     ArrayList<Diario> lista = new ArrayList<>();
300     lista.add(d);
301
302     PaineisContratos p = new PaineisContratos();
303     p.processar(lista);
304
305     conferir("o orgao que multa nao vira fornecedor", 1,
306             p.contarDescartadosDoPapel(PaineisContratos.LADO_MUNICIPIO));
307     conferir("a empresa multada nao vira fornecedor", 1,
308             p.contarDescartadosDoPapel(PaineisContratos.NOTIFICADO));
309     conferir("nenhum contrato neste trecho", 0, p.getContratos().size());
310     conferir("multa nao entra no total", 0.0, p.total());
311 }
312
313 private static void testarPipelineCompleto() {
314     System.out.println("-- pipeline completo, sobre os diarios reais");
315
316     ArrayList<String> falhas = new ArrayList<>();
317     ArrayList<Diario> lidos = new Ingestao(Aplicacao.PASTA_DIARIOS).lerTodos(falhas);
318     conferirMaiorQue("leu diarios da pasta", lidos.size(), 1);
319
320     Limpeza limpeza = new Limpeza();
321     Extracao extracao = new Extracao();
322     Portao portao = new Portao();
323
324     int recusados = 0;
325     ArrayList<Diario> ok = new ArrayList<>();
326
327     for (Diario d : lidos) {
328         try {
329             limpeza.limpar(d);
330         } catch (DiarioIllegivelException ex) {
331             recusados++;
332             continue;
333         }
334         new Fragmentacao().fragmentar(d);
335         extracao.extrair(d);
336         portao.avaliar(d);
337         ok.add(d);
338     }
339
340     conferir("recusa o diario escaneado", 1, recusados);
341     conferirMaiorQue("processa diario bom", ok.size(), 0);
342
343     Diario bh = ok.get(0);
344     conferirMaiorQue("encontrou atos", bh.getAtos().size(), 50);
345     conferirMaiorQue("descartou titulos sem corpo", bh.getTitulosDescartados(), 50);
346     conferirMaiorQue("extraiu fatos", contarFatos(bh), 500);
347     conferirMaiorQue("aceitou atos", bh.contar(Ato.ACEITO), 30);
348
349     boolean todosDecididos = true;
350     for (Ato a : bh.getAtos()) {
351         if (a.getMotivo().isEmpty() || a.getDecisao().isEmpty()) {
352             todosDecididos = false;
353         }
354     }
355     conferir("todo ato tem decisao e motivo", true, todosDecididos);
356

```

```

357     ArrayList<String> identidades = new ArrayList<>();
358     boolean repetida = false;
359     for (Ato a : bh.getAtos()) {
360         if (identidades.contains(a.getIdentidade())) {
361             repetida = true;
362         }
363         identidades.add(a.getIdentidade());
364     }
365     conferir("nenhuma identidade repetida no mesmo diario", false, repetida);
366
367     boolean spansValidos = true;
368     String texto = bh.getTextoLimpo();
369     for (Ato a : bh.getAtos()) {
370         for (Fato f : a.getFatos()) {
371             if (f.getInicio() < 0 || f.getFim() > texto.length()
372                 || f.getInicio() >= f.getFim()) {
373                 spansValidos = false;
374             }
375         }
376     }
377     conferir("todo span cabe no texto", true, spansValidos);
378
379     GrafoNormas grafo = new GrafoNormas();
380     grafo.processar(ok);
381     conferirMaiorQue("montou o grafo", grafo.getArestas().size(), 50);
382
383     PannelContratos painel = new PannelContratos();
384     painel.processar(ok);
385     LeituraDoAto leitor = new LeituraDoAto(painel, grafo);
386     int comLeitura = 0;
387     int contratosLidos = 0;
388     for (Ato a : bh.getAtos()) {
389         String leitura = leitor.ler(a, bh.getData());
390         if (!leitura.isEmpty()) {
391             comLeitura++;
392         }
393         if (leitura.startsWith("Contrata ")) {
394             contratosLidos++;
395         }
396     }
397     conferir("todo ato ganha uma leitura", bh.getAtos().size(), comLeitura);
398     conferirMaiorQue("leitura reconhece contratacoes", contratosLidos, 5);
399     conferir("mesma entrada, mesma leitura (deterministico)",
400         leitor.ler(bh.getAtos().get(0), bh.getData()),
401         new LeituraDoAto(painel, grafo).ler(bh.getAtos().get(0), bh.getData()));
402 }
403
404 private static int contarFatos(Diario d) {
405     int n = 0;
406     for (Ato a : d.getAtos()) {
407         n += a.getFatos().size();
408     }
409     return n;
410 }
411
412 private static void conferir(String nome, Object esperado, Object obtido) {
413     marcar(esperado.equals(obtido), nome,
414         "esperado <" + esperado + ">, obtido <" + obtido + ">");
415 }
416
417 private static void conferirMaiorQue(String nome, int obtido, int minimo) {
418     marcar(obtido > minimo, nome, "esperado > " + minimo + ", obtido " + obtido);
419 }
420
421 private static void marcar(boolean ok, String nome, String detalhe) {
422     if (ok) {
423         passou++;
424         System.out.println("    ok    " + nome);
425     } else {
426         falhou++;
427         System.out.println("    FALHOU    " + nome + " - " + detalhe);
428     }
429 }
430 }

```

ARQUIVO: README.md

raiz do projeto · 383 linhas · README do projeto: como rodar, o pipeline e as 11 decisões de projeto (o porquê).

```

1  # Diários Oficiais municipais – coleta, leitura e análise
2
3  Java puro, sem nenhuma biblioteca externa. Só o JDK (17 ou mais novo).
4
5  O programa sobe um painel no navegador**, coleta diários oficiais de
6  municípios brasileiros, separa cada diário em atos individuais, lê o que cada
7  ato faz e produz informação que não está escrita em nenhum dos documentos**:
8
9  1. quanto cada município contratou, com quem, e quais fornecedores se repetem;
10 2. um grafo de quais normas foram alteradas, revogadas ou prorrogadas;
11 3. uma leitura, em português, do que cada ato faz;
12 4. um relatório do que o programa não conseguiu ler**, e por quê.
13
14 Tudo é determinístico: a mesma pasta de diários produz sempre os mesmos
15 números e as mesmas frases. Nenhuma interpretação usa IA.
16
17 ---
18
19 > Quer entender o código? Leia o [GUIA_DE_ESTUDO.md](GUIA_DE_ESTUDO.md): a ordem
20 > certa de leitura, com os trechos reais e um plano de estudo.
21
22 ## Como rodar
23
24 ### No IntelliJ (recomendado)
25
26 1. File → Open e escolha a pasta projeto-diarios (a que tem o pom.xml).
27 2. Se perguntar, escolha "Open as Project" / "Trust Project". O IntelliJ
28 reconhece o Maven e configura tudo sozinho (espere a barra de progresso
29 embaixo terminar).
30 3. Se aparecer "Project JDK is not defined", clique em Setup SDK e escolha
31 um JDK 17 ou mais novo.
32 4. Abra src/main/java/diarios/servidor/Servidor.java, clique na setinha ►
33 verde ao lado de public static void main e escolha Run.
34
35 O navegador abre sozinho em http://localhost:8080 e a coleta de 1 dia para
36 trás começa em segundo plano. Para parar: o botão vermelho ■.
37
38 Para rodar os testes: abra src/test/java/diarios/Testes.java e aperte ► do
39 mesmo jeito. Para o menu de terminal: src/main/java/diarios/Principal.java.
40
41 > Se o IntelliJ marcar o código todo em vermelho, clique com o botão direito
42 > no pom.xml → Maven → Reload Project.
43
44 ### Pelo cmd (sem IntelliJ)
45
46 Precisa só do JDK 17+ (javac -version para conferir).
47
48 ```
49 rodar.bat      abre o painel
50 testar.bat     roda os 75 testes
51 ```
52
53 ### Pelo Maven (se tiver instalado)
54
55 ```
56 mvn package          gera target/diarios-oficiais-1.0.jar
57 java -jar target/diarios-oficiais-1.0.jar  abre o painel
58 ```
59
60 Opções da linha de comando: --sem-coleta, --sem-navegador, --porta=9090.
61
62 ---
63
64 ## O que tem no painel
65
66 | seção | o que faz |
67 |---|---|
68 | Coleta | escolhe quantos dias para trás (1 a 90) e coleta das fontes ligadas |
69 | Descobrir municípios | pergunta à API quais municípios publicaram texto nos últimos 3 dias e os adiciona à lista |
70 | Histórico de rodadas | cada coleta e cada descoberta: quando, período, quantos baixados, quantas falhas |
71 | Fontes | todas as fontes com botão de ligar/desligar, status da última tentativa e quantos diários já estão no acervo |
72 | Busca | procura uma palavra em todos os diários e leva direto ao ato |
73 | 0 que esses dados dizem | a interpretação do período, em frases geradas por regra |
74 | Por município | os números separados por município, para comparar fontes |
75 | Diários lidos | cada diário, clicável: atos, leitura de cada um, texto bruto e texto limpo |
76 | Página do ato | o que o ato faz, os fatos extraídos, a prova de cada um recortada do texto original, e o texto com os fa
77 >> tos marcados |
78 | funil, portão, fornecedores, grafo, contratos | as análises que já existiam |
79
80 ### Status de uma fonte
81
82 | status | significa |
83 |---|---|
84 | COM_DADO | baixou diário com texto, ou já tinha na pasta |
85 | SEM_EDICAO | a API respondeu, mas não houve edição no período |
86 | SO_PDF | houve edição, mas só em PDF escaneado – sem texto |

```

```

86 | `FALHOU` | a rede ou a API não respondeu (o motivo aparece ao passar o mouse) |
87 | `NUNCA_TENTADA` | ainda não foi coletada |
88
89 O estado das fontes (ligada/desligada, último status) fica em
90 `dados/estado/fontes.txt`, e o histórico em `dados/estado/rodadas.txt` – os dois são
91 texto, dá para abrir e conferir.
92
93 ---
94
95 ## A coleta
96
97 Fonte: Querido Diário(https://queridodiario.ok.org.br) (API em `api.queridodiario.org.br`, endereço novo desde set/2026)
98 >> , projeto aberto da
99 Open Knowledge Brasil que agrega diários oficiais de milhares de municípios, com
100 API pública e gratuita.
101
102 A lista começa com as 27 capitais, com 5 ligadas (Belo Horizonte, Recife,
103 Fortaleza, Curitiba, Aracaju). Nem todo município tem diário com texto no
104 Querido Diário – por isso existe o botão Descobrir municípios com publicação
105 recente: em vez de adivinhar quais fontes têm conteúdo, o programa pergunta à
106 API quem publicou texto nos últimos dias e acrescenta esses municípios à lista
107 (desligados; você liga os que quiser).
108
109 Cuidados que estão no código:
110
111 - 1 segundo entre requisições – o servidor é público e gratuito;
112 - User-Agent identificando o trabalho, em vez de fingir ser navegador;
113 - diário que já está na pasta não é baixado de novo;
114 - falha de uma fonte não derruba a rodada: ela fica marcada `FALHOU` com o
115 motivo, e as outras continuam;
116 - uma coleta por vez: clicar de novo durante uma coleta mostra um aviso.
117
118 ---
119
120 ## Duas peças que não foram dadas em aula
121
122 O professor ensinou até a Aula 05. Duas peças do JDK foram usadas além disso,
123 porque sem elas o programa não teria como buscar o dado real nem mostrar o
124 painel:
125
126 | peça | para quê | por que ela |
127 |---|---|---|
128 | `java.net.http.HttpClient` | baixar os diários da API | vem no JDK; nenhuma biblioteca externa |
129 | `com.sun.net.httpserver.HttpServer` | servir o painel em `localhost` | vem no JDK; evita Spring, Tomcat, Maven |
130
131 O resto – ler o JSON da API, ler os formulários do painel, montar o HTML – é
132 feito à mão com `indexOf`, `substring`, `split` e `StringBuilder`, que são as
133 Aulas 03 e 04. O painel não tem JavaScript: cada botão é um formulário HTML
134 comum; o Java recebe o clique, faz a ação e devolve a página de novo.
135
136 A coleta roda numa `Thread` separada só para a página não ficar travada
137 enquanto baixa.
138
139 ---
140
141 ## Arquitetura: um cérebro, duas portas
142
143 ```
144 navegador → [servidor/Servidor]
145
146 terminal → [Principal (menu)] → Aplicacao → coleta · etapas · analise
147
148 ```
149
150
151 `Aplicacao` concentra todo o trabalho (coletar, ler, interpretar). O menu de
152 console e o painel web só chamam os métodos dela – por isso os números são
153 sempre os mesmos nos dois.
154
155 ---
156
157 ## O pipeline
158
159 ```
160 [0] Coleta          API do Querido Diário → diários/<municipio>_<data>.txt
161 [1] Ingestão        lista a pasta, lê, guarda o bruto
162 [2] Limpeza         remove lixo da conversão de PDF
163                    RECUSA o diário sem texto aproveitável
164 [3] Fragmentação    corta o diário em atos individuais
165                    só corta com evidência positiva de corpo
166 [4] Extração        datas, valores, CNPJ, processos, normas
167                    cada fato guarda a POSIÇÃO no texto
168 [5] Portão          ACEITO · DUVIDOSO · REJEITADO, com motivo
169 [6] Análise         contratos · grafo · interpretação · leitura de cada ato
170
171
172 ---
173
174 ## Sobre os dados pessoais

```

```
175
176 Diários oficiais publicam **CPF completo** de servidores nomeados, exonerados,
177 licenciados. O painel mostra esses CPFs mascarados (`***.***.***-20`). A máscara
178 troca caractere por caractere, então o texto mantém o mesmo tamanho e as
179 posições dos fatos continuam apontando para o lugar certo.
180
181 ---
182
183 ## 0 que o programa produz
184
185 Sobre os dois diários do corpus que vem junto:
186
187 ...
188 aracaju_2026-03-11.txt          RECUSADO
189     motivo: PDF sem texto extraível (provavelmente escaneado)
190     evidência: 1 linha de corpo em 1.715 linhas
191
192 belo-horizonte_2026-03-25.txt  PROCESSADO
193     limpeza      : 16.338 linhas → 5.478 removidas como lixo
194     fragmentação : 225 títulos candidatos → 129 descartados → 96 atos
195     portão       : ACEITO 66 · DUVIDOSO 30 · REJEITADO 0
196     contratações : 19, somando R$ 8.323.312,07
197     grafo        : 6 alterações reais sobre 4 portarias
198 ...
199
200 Exemplos de leitura de ato, gerados por regra:
201
202 ...
203 Contrata AÇÕES PARA A CIDADANIA (13.630.795/0001-63) por R$ 700.000,00.
204     Processo 31.00204095/2026-45.
205 Ato normativo que altera portaria:074. Vigora desde a publicacao (2026-03-25).
206 Ato de pessoal: exoneraçao. Leitura com ressalva: poucos fatos extraídos (1).
207 Notificacao ou multa envolvendo 3 CNPJ(s); nao conta como contratacao.
208 ...
209
210 ---
211
212 ## As decisões de projeto
213
214 Esta seção é a documentação do **porquê**. O código não tem comentários: as
215 razões estão aqui, onde dá para ler de uma vez.
216
217 ### 1. O texto bruto nunca é alterado
218
219 `Diario` guarda `textoBruto` e `textoLimpo` lado a lado. Se a limpeza ou a
220 fragmentação tiverem defeito, conserta-se o código e roda de novo. Alterar o
221 original é a única perda que nenhum conserto futuro alcança.
222
223 ### 2. Recusar é melhor que fingir que leu
224
225 O diário de Aracaju tem **269 páginas** e o texto extraído tem **1 linha de
226 corpo** – é um PDF escaneado, o conteúdo está em imagem.
227
228 Sem a recusa explícita, o programa devolveria "0 atos", que é indistinguível de
229 "dia sem publicação". São coisas diferentes, e `DiarioIlegivelException` carrega
230 o motivo **e a evidência numérica** que sustenta a recusa.
231
232 ### 3. Descartar, nunca remontar
233
234 A marca d'água vertical do PDF ("Documento assinado digitalmente") sai picotada,
235 uma ou duas letras por linha, no meio do texto – **5.478 das 16.338 linhas** do
236 diário de BH.
237
238 A tentação é colar os fragmentos de volta em palavras. Foi testado e **piora**:
239 colar junta o título do ato no parágrafo anterior, e o programa passa a perder
240 atos que antes achava. Descartar linha curta é seguro – o corpo do documento
241 está intacto por baixo.
242
243 ### 4. Só fragmenta com evidência positiva
244
245 O diário de Aracaju tem isto:
246
247 ...
248 DECRETO NUMERADOS - 8512
249     DECRETO SIMPLES - 2026-02-23
250     DECRETO SIMPLES - 2026-02-24
251     ...
252 ...
253
254 Parece lista de atos. É o **sumário**. Um fragmentador que corta sempre que vê
255 um título cria 30 atos vazios ali, e a partir daí toda contagem do programa
256 mente.
257
258 A regra: título só vira ato se houver corpo de texto depois dele. No diário de
259 BH isso descarta **129 dos 225** títulos candidatos.
260
261 Perder um ato é ruim; inventar 30 é pior, porque o erro não aparece – vira
262 número no relatório.
263
264 ### 5. Todo fato guarda a posição no texto
```

```

265
266 Não é "achei a data 24/03/2026". É "achei `24/03/2026` entre os caracteres
267 4.502 e 4.512". O método `Fato.provar()` recorta o trecho original em volta, e
268 a opção 9 do menu mostra isso ato por ato.
269
270 Um extrator sem posição pede para acreditar nele. Um extrator com posição
271 mostra a prova.
272
273 ### 6. Três saídas no portão, nunca duas
274
275 | erro | acontece o quê |
276 |---|---|
277 | deixa passar lixo | **aparece** – alguém lê a saída e vê |
278 | descarta coisa boa | **não aparece nunca** |
279
280 Com duas saídas, o caso duvidoso é empurrado para um dos lados e some. Com a
281 terceira, ele fica contável – os 30 DUVIDOSO do BH são a lista do que o
282 programa ainda não trata.
283
284 `ACEITO` exige sinal positivo. Nenhum caminho leva a "aceita porque não achei
285 nada errado".
286
287 ### 7. Procurar a palavra não é classificar
288
289 No diário de BH a palavra "altera" aparece **145 vezes** e "prorrog" **125**.
290 Quase nenhuma fala de norma:
291
292 ```
293 "Não haverá alteração dos preços ora registrados"           → contrato
294 "As demais condições permanecem inalteradas"                → contrato
295 "prorrogação da vigência da Ata de Registro de Preços"      → contrato
296 "revoga as contratações dos candidatos"                     → pessoal
297 ```
298
299 Um classificador que só procura a palavra criaria **270 relações falsas** num
300 diário que tem **5 verdadeiras**. Por isso `GrafoNormas.classificar()` começa
301 eliminando: citação de fundamento legal ("nos termos do Decreto X") e forma
302 não-dispositiva viram apenas `CITA`.
303
304 A ordem das regras também importa. Em `O Art. 2º da Portaria SMC nº 073/2025
305 passa a vigorar`, o trecho começa com "Art.", que é marca de fundamento – mas o
306 "passa a vigorar" logo depois é evidência mais forte. Na ordem inversa, a única
307 alteração real do diário virava menção.
308
309 ### 8. Sigla de órgão no meio da citação
310
311 `Portaria SMC nº 073/2025` não era encontrada porque o extrator não passava
312 pela sigla do órgão – e essa era a **única** alteração de norma do diário
313 inteiro. `ExtratorReferencia` aceita a sigla, mas só em MAIÚSCULA no texto
314 original: aceitar minúscula faria "lei que trata do artigo 5" virar citação a
315 uma lei número 5.
316
317 ### 9. Agrupar por CNPJ, não por nome
318
319 A mesma empresa aparece escrita de várias formas no diário. O CNPJ é o único
320 campo estável, então o ranking soma por ele.
321
322 ---
323
324 ### 10. Identidade única dentro do diário
325
326 Várias portarias de BH começam com a mesma frase ("Lei Federal nº 13.019/2014 e
327 ao Decreto Municipal nº 16.746/2017...") e a fragmentação tirava o número
328 dessa linha. Resultado: dois atos diferentes com a mesma identidade
329 `lei:13019`, e a leitura de um herdava a alteração do outro – o painel dizia
330 "altera portaria:073" num ato que, no texto logo abaixo, alterava a **074**.
331
332 Achado olhando a página do ato, não num teste. Agora, dentro do mesmo diário,
333 uma identidade repetida ganha `#2`, `#3`, e há um teste que trava isso.
334
335 ### 11. A janela depois da citação
336
337 Em `Portaria SMC Nº 074/2025, publicada no DOM em 25 de novembro de 2025,
338 passa a vigorar`, o verbo fica a ~60 caracteres da citação. Com a janela de
339 60 a alteração se perdia; com 90 ela aparece, e as 6 alterações encontradas
340 foram conferidas uma a uma no texto (a busca do painel faz isso em segundos).
341
342 ---
343
344 ## Onde está cada conteúdo de aula
345
346 | pacote | o que faz | conteúdo |
347 |---|---|---|
348 | `modelo/` | Diário, Ato, Fato, Contrato, Relacao, Achado | classes, encapsulamento, `equals()` |
349 | `extratores/` | família de extratores | **herança e polimorfismo** |
350 | `etapas/` | as etapas do pipeline | `String`, `ArrayList`, arquivos |
351 | `analise/` | contratos, grafo, interpretação, leitura do ato | agregação, ordenação, percurso em grafo |
352 | `coleta/` | API, JSON, fontes, rodadas | `String`, `ArrayList`, arquivos, exceções |
353 | `servidor/` | o painel web | `String`, `StringBuilder` (+ `HttpServer`, ver acima) |
354 | `erros/` | duas exceções próprias | **exceção personalizada** |

```

```

355 | `Principal.java` | menu de console | `Scanner`, `switch` |
356
357 ---
358
359 ## Estrutura
360
361 ```
362 projeto-diarios/
363 |___ pom.xml                configuracao do Maven (Java 17, sem dependencias)
364 |___ README.md
365 |___ rodar.bat · testar.bat  rodar pelo cmd do Windows
366 |___ dados/
367 |   |___ diarios/           entrada: um .txt por diario
368 |   |___ estado/           fontes ligadas e historico de rodadas (gerado)
369 |   |___ saida/            relatorios gravados pelo menu (gerado)
370 |___ src/
371 |   |___ main/java/diarios/
372 |   |   |___ Aplicacao.java  o "cerebro": coletar, ler, interpretar
373 |   |   |___ Principal.java  menu de terminal
374 |   |   |___ servidor/      painel web: Servidor, PaineWeb, PaginaDiario, Formulario
375 |   |   |___ coleta/        API, JSON, fontes, rodadas
376 |   |   |___ etapas/        ingestao, limpeza, fragmentacao, extracao, portao
377 |   |   |___ extratores/     data, valor, CNPJ, processo, norma (heranca)
378 |   |   |___ analise/        contratos, grafo, interpretacao, leitura do ato
379 |   |   |___ modelo/        Diario, Ato, Fato, Contrato, Relacao, Achado
380 |   |   |___ erros/         excecoes proprias
381 |   |___ test/java/diarios/
382 |   |   |___ Testes.java     75 testes sobre dados reais
383 |___ ```

```

ARQUIVO: GUIA_DE_ESTUDO.md

raiz do projeto · 296 linhas · Guia de estudo original: ordem de leitura do código e plano de estudo.

```

1  # Guia de estudo – como ler este projeto
2
3  Este guia é pra você entender o código na ordem certa, sem se perder. Cada
4  passo diz o que abrir, o que procurar, e o que perguntar a si mesmo.
5
6  ---
7
8  ## Parte 0 – O mapa em uma imagem
9
10 O projeto faz UMA coisa: transforma um arquivo de texto gigante (o diário
11 oficial) em informação organizada.
12
13 ```
14     diário (.txt)                                o que sai
15
16     16.338 linhas misturadas    →    96 atos separados
17                                     |
17                                     | cada um com datas, valores, CNPJs
18                                     | cada um com uma nota (aceito/duvidoso/rejeitado)
19                                     | e uma frase dizendo o que ele faz
20 ```
21
22 Tudo que existe no código serve a uma destas cinco perguntas:
23
24 | pergunta | pasta | exemplo |
25 |---|---|---|
26 | o que é o dado? | `modelo/` | `Ato`, `Fato`, `Diario` |
27 | de onde vem o dado? | `coleta/` | `Coletor`, `Fontes` |
28 | como limpar e separar o dado? | `etapas/` | `Limpeza`, `Fragmentacao` |
29 | o que achar no texto? | `extratores/` | `ExtratorCnpj`, `ExtratorData` |
30 | o que isso significa? | `analise/` | `GrafoNormas`, `Interpretacao` |
31
32 E a pasta `servidor/` só mostra o resultado no navegador.
33
34 > Regra pra não se perder: se você está lendo a pasta `servidor/` e não
35 > entende algo, PARE. Ela é só apresentação. O coração do projeto está em
36 > `etapas/` e `extratores/`.
37
38 ---
39
40 ## Parte 1 – Comece pelas classes de dados (15 minutos)
41
42 Abra `modelo/Fato.java`. É a classe mais simples e a mais importante do projeto.
43
44 ```java
45 public class Fato {
46     private final String tipo;    // "CNPJ", "DATA", "VALOR"...
47     private final String valor;    // "48.356.311/0001-14"
48     private final int inicio;    // em que caractere do texto começa
49     private final int fim;    // em que caractere termina
50 }
51
52 o que entender: um `Fato` não é só "achei um CNPJ". É "achei este CNPJ,
53 entre o caractere 4502 e o 4520". Guardar a posição é a ideia central do
54 trabalho. Com ela o método `provar()` consegue recortar o trecho original e
55 mostrar de onde o dado saiu.
56
57 Conceitos de aula aqui: atributos `private`, construtor, getters
58 (encapsulamento), `equals()` e `hashCode()` (comparar conteúdo, não referência).
59
60 Depois leia `modelo/Ato.java`. Um `Ato` é um decreto, uma portaria, um extrato
61 de contrato: uma unidade de trabalho do projeto. Ele tem:
62 - um `texto` (o trecho do diário que é dele),
63 - uma lista de `Fato` (`ArrayList<Fato>`),
64 - uma `decisao` (ACEITO / DUVIDOSO / REJEITADO) e o `motivo`.
65
66 Pergunta pra si mesmo: por que a lista de fatos fica dentro do `Ato` e não
67 solta? *(Porque cada fato pertence a um ato; o ato é quem decide se os fatos
68 bastam.)*
69
70 Por último, `modelo/Diario.java`: o arquivo inteiro de um dia, com o texto
71 bruto e o texto limpo lado a lado.
72
73 ---
74
75 ## Parte 2 – Herança e polimorfismo (20 minutos)
76
77 Abra `extratores/Extrator.java`. É uma classe abstrata: não se cria um
78 `Extrator`, só se criam as filhas.
79
80 ```java
81 public abstract class Extrator {
82     public abstract ArrayList<Fato> extrair(String texto);    // cada filha implementa
83     protected Fato criar(String valor, int inicio, int fim) // atalho reaproveitado
84     protected static boolean casaMolde(String texto, int posicao, String molde)
85 }
86

```

```

87 Agora abra `extratores/ExtratorCnpj.java` (tem só 25 linhas):
88
89 ```java
90 public class ExtratorCnpj extends Extrator {
91     private static final String MOLDE = "99.999.999/9999-99";
92
93     @Override
94     public ArrayList<Fato> extrair(String texto) {
95         ArrayList<Fato> achados = new ArrayList<>();
96         for (int i = 0; i + MOLDE.length() <= texto.length(); i++) {
97             if (casaMolde(texto, i, MOLDE)) {
98                 achados.add(criar(texto.substring(i, i + MOLDE.length()),
99                                 i, i + MOLDE.length()));
100                 i += MOLDE.length() - 1;
101             }
102         }
103         return achados;
104     }
105 }
106 ```
107
108 **Como funciona o `MOLDE`: cada `9` significa "qualquer dígito"; qualquer
109 outro caractere tem que ser igual. O método `casaMolde` (em `Extrator.java`)
110 anda caractere por caractere comparando. Nenhuma expressão regular: é só
111 `charAt`, `substring`, `for` (Aula 04).
112
113 **Exercício: abra `ExtratorData.java` e `ExtratorValor.java`. Todos têm o
114 mesmo método `extrair`, e cada um faz do seu jeito. Isso é **polimorfismo**
115 (Aula 02, o exemplo do `FazerSom()` do Cachorro e do Gato).
116
117 Onde o polimorfismo aparece? Em `etapas/Extracao.java`:
118
119 ```java
120 extratores.add(new ExtratorData());
121 extratores.add(new ExtratorValor());
122 extratores.add(new ExtratorCnpj());
123 ...
124 for (Extrator e : extratores) {
125     for (Fato f : e.extrair(ato.getTexto())) { ... }
126 }
127 ```
128
129 O laço chama `e.extrair(...)` sem saber qual filha é. Cada uma responde do seu
130 jeito. **Se você quiser achar CEPs, cria uma classe filha e acrescenta uma
131 linha. Nada mais muda.**
132
133 ---
134
135 ## Parte 3 – O pipeline, que é o esqueleto do projeto (30 minutos)
136
137 Abra `Aplicacao.java` e ache o método `processarPasta()`. **Este método é o
138 projeto inteiro em 30 linhas.** Leia devagar:
139
140 ```java
141 ArrayList<Diario> lidos = new Ingestao(PASTA_DIARIOS).lerTodos(falhas); // 1
142
143 for (Diario d : lidos) {
144     try {
145         limpeza.limpar(d); // 2
146     } catch (DiarioIlegivelException e) {
147         novaCobertura.registrarRecusa(...); // 2b
148         novos.add(d);
149         continue;
150     }
151     new Fragmentacao().fragmentar(d); // 3
152     extracao.extrair(d); // 4
153     portao.avaliar(d); // 5
154     ...
155 }
156 novoPainel.processar(novos); // 6
157 novoGrafo.processar(novos); // 7
158 novaInterpretacao.interpretar(...); // 8
159 ```
160
161 Siga cada número até a classe correspondente:
162
163 ### 1. `etapas/Ingestao.java` – ler a pasta
164 Lista os `*.txt` de `dados/diarios/` e cria um `Diario` para cada um. O nome do
165 arquivo (`belo-horizonte_2026-03-25.txt`) carrega município e data.
166 **Aula 05:** `Files`, `DirectoryStream`, `try-with-resources`, `IOException`.
167
168 ### 2. `etapas/Limpeza.java` – limpar e recusar
169 Leia o método `limpar`. Duas ideias:
170 - linha com 3 caracteres ou menos é **lixo** (a marca d'água do PDF sai
171 picotada: `D`, `oc`, `um...`) e é **descartada**;
172 - se sobram menos de 10 linhas longas, o diário é **ilegível** e lança
173 `DiarioIlegivelException`.
174
175 **Por que lançar exceção, em vez de devolver um número? Porque "diário
176 ilegível" não é um erro do programa: é um resultado possível que precisa ser

```

```

177 tratado. Por isso é uma exceção própria (Aula 05). Veja em
178 `erros/DiarioIlegivelException.java` que ela carrega o motivo e a
179 evidência.
180
181 ### 3. `etapas/Fragmentacao.java` – cortar em atos
182 A parte mais difícil. Leia `temCorpo`:
183
184 ```java
185 private boolean temCorpo(String bloco, String titulo) {
186     ...
187     if (semTitulo.trim().length() < CORPO_MINIMO) return false;
188     for (String l : semTitulo.split("\n")) {
189         if (l.trim().length() > PARAGRAFO_MINIMO) return true;
190     }
191     return false;
192 }
193 ```
194
195 **A ideia:** uma linha que começa com "DECRETO" só vira ato se tiver texto de
196 verdade depois. Senão é item de sumário. Na dúvida, não corta.
197
198 ### 4. `etapas/Extracao.java` – chamar os extratores
199 Já vimos na Parte 2.
200
201 ### 5. `etapas/Portao.java` – dar a nota
202 Leia `avaliarAto`. É só uma sequência de `if`:
203
204 ```
205 texto curto demais?           → REJEITADO (com o motivo)
206 poucos fatos?                 → DUVIDOSO (com o motivo)
207 sem número/valor/norma?      → DUVIDOSO
208 sem nenhuma data?            → DUVIDOSO
209 passou em tudo                → ACEITO
210 ```
211
212 **Repare:** só chega em ACEITO quem passou por todos os `if`. **Nenhum caminho
213 leva a "aceito" por omissão. E toda decisão grava o `motivo`.
214
215 ### 6, 7, 8. `analise/` – o que significa
216 Depois de ter os atos com fatos, `analise/` responde perguntas:
217 - `PainelContratos`: quem foi contratado, por quanto?
218 - `GrafoNormas`: qual ato altera qual norma?
219 - `Interpretacao`: as frases do "o que esses dados dizem".
220
221 ---
222
223 ## Parte 4 – Os trechos mais difíceis (leia por último)
224
225 Quando o resto estiver claro, estes três são os que mais custam:
226
227 **`analise/GrafoNormas.classificar()`** – decide se um ato ALTERA, REVOGA,
228 PRORROGA ou só CITA uma norma. A dificuldade: a palavra "altera" aparece 145
229 vezes no diário de BH e quase todas são cláusula de contrato ("alteração dos
230 preços"), não norma. Por isso o método segue uma ordem, e a ordem
231 importa: (1) olha o trecho depois da citação ("...passa a vigorar" →
232 ALTERA); (2) descarta "fundamento legal" ("nos termos do Decreto X" → só
233 CITA); (3) só então procura o verbo antes da citação ("revoga o...").
234 Se nada bater, é só CITA – na dúvida, a relação mais fraca.
235
236 **`analise/PainelContratos.papelENomeAntesDe()`** – descobre se um CNPJ é
237 fornecedor, ou é o órgão que está multando (`NOTIFICANTE`), ou a empresa
238 multada (`NOTIFICADO`). Nasceu de um erro real: o programa somava multas como
239 contratos.
240
241 **`coleta/Coletor.consultarApi()`** – tenta de novo com espera crescente
242 (5s, 15s, 45s) quando a API recusa por excesso de pedidos.
243
244 ---
245
246 ## Parte 5 – Como ver o código funcionando (o melhor jeito de aprender)
247
248 No IntelliJ, use o Debug:
249
250 1. Abra `Aplicacao.java`, vá ao método `processarPasta()`.
251 2. Clique na margem esquerda ao lado da linha `limpeza.limpar(d);` –
252 aparece uma bolinha vermelha (breakpoint).
253 3. Rode `Servidor.java` com o botão inseto <U+1F41E> (Debug), não o ▶.
254 4. O programa para nessa linha. Embaixo aparece a aba Variables:
255 abra `d` e veja o texto bruto do diário.
256 5. Aperte F8 (Step Over) para avançar uma linha por vez, e F7
257 (Step Into) para entrar dentro do método.
258
259 Faça isso com `limpeza.limpar(d)`, depois `fragmentar(d)`, depois
260 `extracao.extrair(d)`. Em cada passo olhe `d.getAtos()` crescer: primeiro vazio,
261 depois com 96 atos, depois cada ato com seus fatos.
262
263 **Atalhos que salvam a vida:**
264 - `Ctrl + clique` num nome → vai pra onde ele foi escrito
265 - `Alt + ←` → volta
266 - `Ctrl + Shift + F` → busca um texto no projeto inteiro

```

```
267
268 ---
269
270 ## Parte 6 – Como responder na apresentação
271
272 | se o professor perguntar... | aponte para... |
273 |---|---|
274 | "Mostra o uso de herança." | `Extrator` + `ExtratorCnpj` (Parte 2) |
275 | "Mostra tratamento de exceção." | `Limpeza.limpar()` lança, `Aplicacao.processarPasta()` captura |
276 | "Onde você usa arquivos?" | `Ingestao` (lê), `Relatorio` (escreve), ambos com `try-with-resources` |
277 | "Onde você usa ArrayList?" | quase em todo lugar: `Ato.fatos`, `Diario.atos`... |
278 | "Como você sabe que não inventou o dado?" | `Fato.provar()` e a página de um ato no painel |
279 | "Por que não usou banco?" | não foi dado; arquivo texto dá pra abrir e conferir |
280 | "O que é essa parte de servidor?" | só mostra o resultado; o trabalho está em `etapas/` |
281
282 ---
283
284 ## Um plano de estudo realista
285
286 | dia | o que fazer | tempo |
287 |---|---|---|
288 | 1 | Partes 0 e 1: as classes de `modelo/` | 30 min |
289 | 1 | Parte 2: `Extrator` e as filhas | 30 min |
290 | 2 | Parte 3: `processarPasta()` + `Limpeza` + `Portao` | 1 h |
291 | 2 | Parte 5: debugar `processarPasta()` | 30 min |
292 | 3 | `Fragmentacao` e `Ingestao` | 1 h |
293 | 3 | Parte 4: `GrafoNormas` | 1 h |
294
295 **Se sobrar pouco tempo antes da arguição:** domine a Parte 2 (herança) e
296 a Parte 3 (o pipeline). Com essas duas você responde a maioria das perguntas.
```

ARQUIVO: PREPARO_ARGUICAO.md

raiz do projeto · 233 linhas · Preparo original para a arguição: perguntas por classe e exercícios.

```

1  # Preparo para a arguição do código
2
3  0 professor vai **abrir o código e perguntar**. Este material é pra treinar
4  isso. Não decore: entenda, e ensaie dizendo em voz alta.
5
6  ---
7
8  ## 1. Três regras que valem mais que saber tudo
9
10 1. **Nunca invente.** Dizer "essa parte foi além do que o senhor deu, eu
11   entendo o que ela faz, mas não domino o detalhe" é muito melhor que
12   inventar e ser pego. Isso vale principalmente para a Seção 4.
13 2. **Responda em duas frases:** *o que faz* + *por que foi feito assim*.
14   Exemplo: "`temCorpo` só deixa um título virar ato se tiver texto depois
15   dele. Sem isso o sumário do diário virava 30 atos vazios."
16 3. **Aponte o arquivo e a linha** enquanto fala. Quem mostra o código
17   transmite que conhece o código.
18
19 ---
20
21 ## 2. Quem defende o quê (sugestão para 4 pessoas)
22
23 | pessoa | pacote | o que dominar |
24 |---|---|---|
25 | 1 | `modelo/` + `extratores/` | `Fato`, `Ato`, `Extrator` e as filhas (herança) |
26 | 2 | `etapas/` | `Limpeza`, `Fragmentacao`, `Portao`, `Ingestao`, `Relatorio` |
27 | 3 | `analise/` | `PainelContratos`, `GrafoNormas`, `Interpretacao` |
28 | 4 | `Aplicacao` + `coleta/` + `servidor/` | o fluxo e a parte "fora da aula" |
29
30 **Todos** precisam saber explicar `Aplicacao.processarPasta()`.
31
32 ---
33
34 ## 3. Perguntas por classe
35
36 ### `Fato.java`
37 - **0 que é um Fato?** Um dado achado no texto (CNPJ, data, valor) junto com a
38   posição exata onde foi achado (`inicio` e `fim`).
39 - **Por que guardar a posição?** Para provar de onde saiu. O método `provar()`
40   recorta o trecho original em volta. Sem posição, o programa pediria pra
41   acreditar nele.
42 - **Por que `equals` e `hashCode`?** Dois fatos são iguais se tiverem o mesmo
43   tipo e valor, ignorando a posição. Comparar conteúdo, não referência (Aula 04).
44 - **Por que os atributos são `private final`?** Encapsulamento: ninguém altera
45   um fato depois de criado.
46
47 ### `Ato.java`
48 - **0 que é um Ato?** Um decreto, portaria ou extrato: a unidade de trabalho.
49 - **Como nasce a identidade?** `getIdentidade()` monta
50   `municipio:especie:numero:data` a partir do conteúdo.
51 - **Por que não um contador (1, 2, 3)?** Com contador, rodar de novo geraria
52   identificadores diferentes.
53 - **E se dois atos tiverem a mesma identidade?** `desambiguar()` acrescenta
54   `#2`, `#3`. Foi um bug real: duas portarias começavam com a mesma linha.
55
56 ### `Extrator` e as filhas (herança e polimorfismo)
57 - **Onde está a herança?** `ExtratorCnpj extends Extrator`.
58 - **Por que `Extrator` é abstrata?** Não faz sentido criar um "extrator
59   genérico". Ela define o contrato (`extrair`) e reaproveita `criar` e
60   `casaMolde`.
61 - **Onde está o polimorfismo?** Em `Extracao`: uma lista de `Extrator` com
62   cinco classes diferentes, e o laço chama `e.extrair()` sem saber qual é qual.
63 - **0 que é o `@Override`?** Indica que o método sobrescreve o da classe mãe;
64   o compilador avisa se a assinatura estiver errada.
65 - **Como o `casaMolde` funciona?** `9` = qualquer dígito; o resto tem que ser
66   idêntico. Anda caractere por caractere com `charAt`.
67
68 ### `Limpeza.java`
69 - **0 que faz?** Descarta linha com até 3 caracteres (lixo da marca d'água do
70   PDF) e recusa o diário que não tem texto suficiente.
71 - **Por que descartar e não remontar as letras?** Foi testado: remontar colava
72   o título de um ato no parágrafo anterior e o programa perdia atos.
73 - **Por que lançar exceção?** "Diário ilegível" é um resultado possível, não
74   um bug. `DiarioIlegivelException` (exceção própria, Aula 05) carrega o
75   motivo e a evidência.
76 - **Por que 10 linhas longas como mínimo?** O diário de Aracaju (PDF
77   escaneado) tem 1. 0 de BH tem mais de 1.000. O limite separa os dois casos.
78
79 ### `Fragmentacao.java`
80 - **0 que faz?** Corta o diário em atos.
81 - **Qual o problema principal?** O sumário parece uma lista de atos.
82 - **Como resolve?** `temCorpo`: o título só vira ato se houver texto de verdade
83   depois. **Na dúvida, não corta.**
84 - **Por que `LEI COMPLEMENTAR` vem antes de `LEI` na lista?** Porque a busca
85   para no primeiro que casa; se `LEI` viesse antes, `LEI COMPLEMENTAR` seria
86   lida como `LEI`.

```

```

87
88 ### `Portao.java`
89 - **Por que três saídas e não duas?** Com duas, o caso duvidoso é empurrado pra
90   um lado e some. Com três, fica contável.
91 - **Como um ato chega em ACEITO?** Passando por todos os `if`. Nenhum caminho
92   leva a aceito por omissão.
93 - **E o motivo?** Toda decisão grava o motivo, inclusive as rejeitadas.
94
95 ### `Ingestao.java` e `Relatorio.java` (Aula 05)
96 - **Onde usa arquivo?** `Ingestao` lê (`Files.readString`, `DirectoryStream`),
97   `Relatorio` grava (`BufferedWriter`).
98 - **Por que `try-with-resources`?** Fecha o arquivo sozinho, mesmo se der erro.
99 - **E se a pasta não existir?** `Ingestao` captura `IOException`, anota a falha
100   e devolve lista vazia; o programa não cai.
101
102 ### `PainelContratos.java`
103 - **Como calcula o ranking?** Agrupa por CNPJ (`indexOf` num `ArrayList`) e
104   ordena por **seleção**: acha o maior, marca como usado, repete.
105 - **Qual a complexidade?**  $O(n^2)$ . Com dezenas de fornecedores é instantâneo.
106   Com milhares, valeria outra estrutura (`HashMap`, ainda não dada).
107 - **Por que agrupar por CNPJ e não por nome?** O nome vem escrito de várias
108   formas; o CNPJ não.
109 - **O que são os papéis (NOTIFICANTE, CONTRATADA)?** Descubrem se o CNPJ é
110   fornecedor ou é o órgão que está multando. Nasceu de um erro real.
111
112 ### `GrafoNormas.java`
113 - **O que é o grafo?** Uma lista de arestas: *este ato ALTERA/REVOGA/PRORROGA/
114   CITA aquela norma*.
115 - **Por que não procurar só a palavra "altera"?** Ela aparece 145 vezes no
116   diário de BH e quase todas são cláusula de contrato.
117 - **Qual a ordem das regras?** (1) verbo depois da citação, (2) fundamento
118   legal, (3) verbo antes. Na dúvida, a relação mais fraca (`CITA`).
119
120 ### `Coletor.java`
121 - **Como baixa?** `HttpClient` faz um GET na API do Querido Diário; o JSON é
122   lido à mão por `Json.java` com `indexOf` e `substring`.
123 - **E se a API falhar?** Tenta de novo com espera crescente (5s, 15s, 45s) e,
124   depois de 3 fontes seguidas com falha, a rodada para.
125 - **Por que `Json.java` à mão?** Não usar biblioteca externa.
126
127 ---
128
129 ## 4. A zona vermelha: o que está além da aula
130
131 O professor já sabe que o scraper é extra. Estes são os outros pontos que ele
132 pode atacar. **Saiba onde estão e tenha a resposta honesta pronta.**
133
134 | o quê | onde | resposta curta e honesta |
135 |---|---|---|
136 | `Thread` | `Aplicacao.coletarEmSegundoPlano` | "A coleta demora, então roda numa linha de execução separada pra página não t
137   >> ravar." |
138 | `synchronized` | `Aplicacao` (vários métodos) | "Duas linhas de execução mexem nos mesmos dados; `synchronized` deixa uma de
139   >> cada vez. Entendo o motivo, não domino o detalhe." |
140 | `volatile` | `Aplicacao.ocupado` | "Garante que a outra linha enxergue o valor atualizado." |
141 | `()` -> `(lambda)` | `new Thread(() -> {...})` | "É um jeito curto de passar um bloco de código. Equivale a uma classe que imp
142   >> lementa `Runnable`." |
143 | `this::tratarPainel` | `Servidor.iniciar` | "Referência a um método: diz ao servidor qual método chamar em cada endereço." |
144 | `HttpServer` | `Servidor` | "Vem no JDK; recebe pedido do navegador e devolve a página." |
145 | `Locale.ROOT` | `Contrato.formatarReal` | "No Windows em português o `String.format` usa vírgula; foi um erro real que só ap
146   >> arecia lá." |
147 | Maven (`pom.xml`) | raiz | "Formato que o IntelliJ entende; não tem dependência nenhuma." |
148
149 **Se ele perguntar "por que usou isso se não foi dado?" a resposta é:
150 **Sem `HttpClient` não haveria como buscar o dado real, e sem `HttpServer` não
151 haveria o painel. Usamos só o que já vem no JDK, sem biblioteca externa.**
152
153 ---
154
155 ## 5. Exercícios: "altere o código aqui na minha frente"
156
157 Muito professor pede isso. **Treinem os três até fazer sem olhar.**
158
159 ### Exercício A – achar um tipo novo de dado (CEP)
160 1. Em `Fato.java`, acrescente `public static final String CEP = "CEP";`
161 2. Crie `extratores/ExtratorCep.java`:
162 ```java
163 public class ExtratorCep extends Extrator {
164     private static final String MOLDE = "99999-999";
165
166     public ExtratorCep() { super(Fato.CEP); }
167
168     @Override
169     public ArrayList<Fato> extrair(String texto) {
170         ArrayList<Fato> achados = new ArrayList<>();
171         for (int i = 0; i + MOLDE.length() <= texto.length(); i++) {
172             if (casaMolde(texto, i, MOLDE)) {
173                 achados.add(criar(texto.substring(i, i + MOLDE.length()),
174                                     i, i + MOLDE.length()));
175                 i += MOLDE.length() - 1;
176             }
177         }
178     }
179 }

```

```
173     }
174     return achados;
175 }
176 },
177 },
178 3. Em `Extracao.java`, no construtor: `extratores.add(new ExtratorCep());`
179
180 **Testado:** no diário de BH acha 4 CEPs reais (ex.: `03333-050`), e os testes
181 continuam passando.
182
183 **0 que isso prova:** polimorfismo. Só acrescentou uma classe e uma linha;
184 nada mais mudou. (Cuidado: o molde pode pegar pedaço de outro número; se ele
185 notar, diga que é uma limitação e que o passo seguinte seria checar o que vem
186 antes e depois.)
187
188 ### Exercício B – deixar o portão mais rígido
189 Em `Portao.java`, mude `FATOS_MINIMOS = 2` para `3`. **Resultado medido:**
190 o diário de BH passa de **66 aceitos / 30 duvidosos** para **58 aceitos / 38
191 duvidosos**. Mostre no painel ("Reler a pasta", depois de recompilar). *Por que isso é bom de mostrar:*
192 comprova que o limite é um parâmetro, e que o número de "duvidosos" responde
193 a ele.
194
195 ### Exercício C – reconhecer outro tipo de ato
196 Em `Fragmentacao.java`, acrescente `"CONVENIO"` à lista `ESPECIES`. Atenção à
197 ordem: nomes mais longos antes dos curtos.
198
199 ---
200
201 ## 6. Perguntas gerais que costumam aparecer
202
203 - **Por que `ArrayList` e não `HashMap`?* `HashMap` não foi dado. Com dezenas
204 ou centenas de itens a diferença de tempo não aparece.
205 - **Por que não usou banco de dados?* Não foi dado; arquivo texto pode ser
206 aberto e conferido por qualquer pessoa.
207 - **0 que acontece se o diário vier vazio?** `Limpeza` lança
208 `DiarioIllegivelException`, o diário é recusado com motivo e o programa segue.
209 - **Como você sabe que o programa acerta?* 85 testes sobre diários reais, mais
210 a página do ato, que mostra cada fato com o trecho original.
211 - **Quais são as limitações?* Só foi validado com diários de texto; PDF
212 escaneado não é lido (precisaria de OCR); alguns nomes de empresa não são
213 identificados; 31% dos atos ficam em dúvida, e isso aparece no painel.
214 - **Por que um projeto de diários oficiais?* Dado público, real e bagunçado,
215 que exige decisões de programação, não só ler arquivo.
216
217 ---
218
219 ## 7. Autoteste (10 minutos, em voz alta)
220
221 1. Explique o que `Aplicacao.processarPasta()` faz, em ordem.
222 2. O que é um `Fato` e por que guarda a posição?
223 3. Mostre herança no projeto e diga onde está o polimorfismo.
224 4. O que a `Limpeza` faz e quando lança exceção?
225 5. Por que o `Portao` tem três saídas?
226 6. Como o `Fragmentacao` evita contar o sumário como atos?
227 7. Por que `ranking()` é  $O(n^2)$  e por que isso não é problema aqui?
228 8. Faça o Exercício A sem olhar.
229 9. Explique o que `synchronized` faz e onde aparece.
230 10. O que acontece se a API estiver fora do ar?
231
232 Se você errou mais de 3, releia as Partes 2 e 3 do `GUIA_DE_ESTUDO.md` e
233 debug o `processarPasta()` no IntelliJ.
```